

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM

Logo_UIT_updated.jpg

KHÓA LUẬN TỐT NGHIỆP
KỸ SƯ / CỬ NHÂN NGÀNH KỸ THUẬT PHẦN MỀM
MÃ HỌC PHẦN: SE505.Q21

**PHÁT TRIỂN HỆ THỐNG QUẢN LÝ BÁN LẺ ĐA CHI
NHÁNH DỰA TRÊN KIẾN TRÚC MICROSERVICES VÀ
HỆ GỢI Ý THÔNG MINH LAI**

*Development of a Multi-branch Retail Management System based on Microservices
Architecture and Hybrid Recommender*

GIẢNG VIÊN HƯỚNG DẪN:
TS. Nguyễn Thị Xuân Hương

SINH VIÊN THỰC HIỆN:

Họ và tên	Mã số SV	Ngành
Nguyễn Trương Tiến Phát	23521148	Kỹ thuật Phần mềm
Đỗ Minh Đức	23520303	Kỹ thuật Phần mềm

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM KHÓA LUẬN TỐT NGHIỆP

Hội đồng chấm khóa luận tốt nghiệp, thành lập theo Quyết định số:
ngày . tháng . năm 2026 của Hiệu trưởng Trường Đại học Công nghệ Thông tin, ĐHQG-HCM.

Thành viên Hội đồng:

- 1. Chủ tịch: Cơ quan:
- 2. Thư ký: Cơ quan:
- 3. Phản biện 1: Cơ quan:
- 4. Phản biện 2: Cơ quan:
- 5. Ủy viên: Cơ quan:

Khóa luận tốt nghiệp được thực hiện tại: **Khoa Công nghệ Phần mềm, Trường Đại học Công nghệ Thông tin, ĐHQG-HCM.**

Giảng viên hướng dẫn: **TS. Nguyễn Thị Xuân Hương**

TP. Hồ Chí Minh, ngày tháng năm 2026

CHỦ TỊCH HỘI ĐỒNG

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến **Tiến sĩ Nguyễn Thị Xuân Hương** - Giảng viên Khoa Công nghệ Phần mềm, Trường Đại học Công nghệ Thông tin, ĐHQG-HCM. Cô đã luôn tận tình định hướng, chỉ bảo khoa học, theo sát và hỗ trợ chúng em trong suốt quá trình nghiên cứu, xây dựng hệ thống và hoàn thiện khóa luận tốt nghiệp này. Những góp ý chuyên môn sâu sắc của Cô về kiến trúc phân tán, phương pháp nghiên cứu hệ gợi ý và tính nghiêm ngặt trong thực nghiệm đã giúp chúng em nâng cao năng lực tư duy khoa học và giải quyết các bài toán kỹ thuật phức tạp.

Chúng em cũng xin trân trọng cảm ơn Ban Giám hiệu Trường Đại học Công nghệ Thông tin, cùng toàn thể quý Thầy Cô Khoa Công nghệ Phần mềm đã tạo mọi điều kiện thuận lợi, truyền đạt những kiến thức chuyên môn quý báu trong suốt những năm học tập tại trường.

Cuối cùng, chúng em xin bày tỏ lòng biết ơn vô hạn đến gia đình, người thân và bạn bè đã luôn động viên, chia sẻ và sát cánh cùng chúng em trong suốt quá trình học tập và thực hiện đề tài.

Dù đã rất nỗ lực đầu tư nghiên cứu nghiêm túc, song khóa luận khó tránh khỏi những thiếu sót nhất định. Nhóm sinh viên kính mong nhận được những ý kiến đóng góp, chỉ dẫn quý báu từ quý Thầy Cô trong Hội đồng để khóa luận được hoàn thiện hơn nữa.

TP. Hồ Chí Minh, tháng 06 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Trương Tiến Phát - Đỗ Minh Đức

TÓM TẮT KHÓA LUẬN

Đặt vấn đề

Trong kỷ nguyên bán lẻ đa kênh (Omnichannel Retailing), các chuỗi cửa hàng tiện lợi và siêu thị mini đối mặt với bài toán phức tạp: vừa phải bảo đảm tốc độ xử lý giao dịch tức thời tại quầy bán lẻ (POS Terminal), vừa phải xử lý lượng truy cập khổng lồ và cá nhân hóa trải nghiệm cho khách hàng mua sắm trực tuyến (Storefront). Các hệ thống nguyên khối (Monolithic) bộc lộ sự quá tải cục bộ, chi phí hạ tầng đắt đỏ và thiếu khả năng hỗ trợ thông minh từ AI. Khóa luận tập trung nghiên cứu, thiết kế và phát triển nền tảng **POSMART** - một hệ sinh thái quản trị bán lẻ phân tán thể hệ mới dựa trên kiến trúc Microservices kết hợp hệ gợi ý thông minh lai.

Phương pháp và Giải pháp kiến trúc

Hệ thống được phân rã thành **9 Microservices** độc lập, giao tiếp với nhau qua API Gateway (Nginx) và Message Broker (RabbitMQ):

- **Row-Level Multi-Tenancy:** Tối ưu hóa chi phí hạ tầng máy chủ thông qua mô hình chia sẻ CSDL (Shared Database, Shared Schema) trên PostgreSQL 16 (1 instance, 8 logical databases), cô lập vùng dữ liệu an toàn cho từng chi nhánh thông qua trường định danh `store_id` được ký mã hóa trong JWT Payload.
- **Tính nhất quán phân tán & Saga Pattern:** Triển khai Saga Choreography giữa Payment Service, Order Service và Inventory Service thông qua RabbitMQ Topic Exchange, kết hợp mẫu thiết kế **Transactional Outbox** và **Idempotency Guard** (`processed_events`) để đảm bảo nguyên tắc xử lý chính xác một lần.
- **Trợ lý tác vụ AI Chatbot (Action Assistant v2.0):** Nâng cấp Chatbot từ mô hình hỏi đáp chỉ đọc lên trợ lý tác vụ hai chiều. Nhờ thiết kế *Action Response Protocol* và chốt chặn xác nhận (*Confirmation Gate*) bảo mật 7 lớp, Chatbot có thể trực tiếp tra cứu tồn kho chi nhánh, quản lý giỏ hàng và tạo đơn hàng mà Backend vẫn duy trì trạng thái phi trạng thái (Stateless).
- **Mô hình Gợi ý lai Wide & Deep Two-Tower ('ai-service-v2'):** Kết hợp nhánh Wide (Luật kết hợp Apriori trên giỏ hàng mua sắm tự nhiên) và nhánh Deep Two-Tower (User Tower + Item Tower tích hợp Content Projection text/category/price) huấn luyện qua hàm mất mát Bayesian Personalized Ranking (BPR Loss). Xây dựng quy trình Benchmark khoa học có khả năng tái lập (Reproducible Protocol) với đánh giá toàn danh mục (Full-catalog ranking $C_u = I \setminus H_u^{\text{seen}}$).

Kết quả đạt được

Hệ thống đã hiện thực hóa trọn vẹn cả 3 phân hệ giao diện: POS Terminal tại quầy (đăng nhập mã PIN Bcrypt, quét mã vạch, đơn hàng chờ Hold order, thanh toán VNPAY QR động), Admin & Manager Dashboard (quản trị kho bãi, lô hàng, công nợ NCC và thống kê Redis Caching), và

Customer Storefront (mua sắm trực tuyến và trợ lý ảo AI). Kiểm thử tải thực tế tại API Gateway xác nhận độ trễ trung bình P95 < 200ms cho các API nghiệp vụ và P95 < 500ms cho toàn bộ chuỗi pipeline AI. Toàn bộ 59/59 testcase kiểm thử chức năng và phi chức năng đều đạt 100% PASS.

ABSTRACT

Title: Development of a Multi-branch Retail Management System based on Microservices Architecture and Hybrid Recommender

Students: Nguyen Truong Tien Phat, Do Minh Duc

Supervisor: Ph.D. Nguyen Thi Xuan Huong

Major: Software Engineering, University of Information Technology, VNU-HCM

In modern retail, chain-level mini-marts and convenience stores face significant architectural challenges in balancing high-throughput in-store transactions at POS terminals with scalable, personalized online shopping experiences. Traditional monolithic enterprise resource planning systems inevitably suffer from performance bottlenecks, single points of failure, and infrastructure cost bloat.

This graduation thesis presents the design and realization of **POSMART**, an advanced multi-tenant retail management platform. The backend is partitioned into 9 independent microservices managed via an Nginx API Gateway and coordinated asynchronously through a RabbitMQ event bus. A cost-effective Row-Level Multi-tenancy architecture is established across 8 logical PostgreSQL databases, strictly enforcing data isolation between store branches via cryptographically signed `store_id` claims. Distributed transaction consistency across order placement, stock deduction, and electronic payments (VNPay) is guaranteed through Saga Choreography augmented with Transactional Outbox and Idempotent Consumer patterns.

Furthermore, the platform introduces two major AI advancements: (1) An interactive Action Assistant (v2.0) that moves beyond read-only RAG pipelines to execute bidirectional state-changing actions via a structured Action Response Protocol and a 7-layer Confirmation Gate; and (2) An independent scientific research recommender engine (`ai-service-v2`) integrating a decoupled Wide-and-Deep two-tower architecture. The Wide branch captures co-purchase regularities mined by Apriori association rules on organic baskets, while the Deep branch leverages User and Item towers optimized with Bayesian Personalized Ranking (BPR) loss and additive Z-score fusion. The recommendation protocol enforces reproducible full-catalog evaluation ($C_u = I \setminus H_u^{\text{seen}}$) without data leakage. Extensive empirical evaluations demonstrate that POSMART achieves sub-200ms P95 API response times, sub-500ms AI pipeline latencies, and 100% test pass rates across all functional and non-functional requirements.

Keywords: Microservices, Multi-tenancy, Saga Pattern, Retail Management, Recommender Systems, Two-Tower Models, Apriori, BPR Loss, Full-catalog Ranking, Action Assistant.

Mục lục

LỜI CẢM ƠN	1
TÓM TẮT KHÓA LUẬN	2
ABSTRACT	4
DANH MỤC TỪ VIẾT TẮT	12
1 TỔNG QUAN ĐỀ TÀI	14
1.1 Động lực nghiên cứu và lý do chọn đề tài	14
1.1.1 Bối cảnh chuyển dịch mô hình bán lẻ và sự cấp thiết của đề tài	14
1.1.2 Thách thức kỹ thuật cốt lõi trong kiến trúc hệ thống bán lẻ phân tán	14
1.1.3 Động lực nâng cấp từ Đồ án 2 lên Khóa luận tốt nghiệp	15
1.2 Khảo sát hiện trạng và Hệ thống đề xuất	15
1.2.1 Hiện trạng các nền tảng bán lẻ và thương mại điện tử	15
1.2.2 Phân tích các hạn chế kỹ thuật phổ biến	16
1.2.3 Ma trận phân tích khoảng cách năng lực, điểm nghẽn kiến trúc và nợ kỹ thuật	16
1.2.4 Hệ thống kiến trúc đề xuất (POSMART Enterprise Solution)	16
1.3 Đối tượng và Phạm vi nghiên cứu	18
1.3.1 Đối tượng nghiên cứu	18
1.3.2 Phạm vi nghiên cứu	18
1.4 Mục tiêu đề tài và Các yêu cầu hệ thống	19
1.4.1 Yêu cầu chức năng hệ thống (Functional Requirements)	19
1.4.2 Yêu cầu dữ liệu (Data Requirements)	20
1.4.3 Yêu cầu giao diện, phần cứng, phần mềm	21
1.4.4 Yêu cầu phi chức năng (Quality Attributes)	21
2 Cơ sở lý thuyết	23
2.1 Những ứng dụng và công trình nghiên cứu liên quan	23
2.2 Công nghệ áp dụng	24
2.2.1 Node.js	24
2.2.2 Express.js	25
2.2.3 PostgreSQL	25
2.2.4 RabbitMQ	26
2.2.5 Redis	27
2.2.6 Hugging Face Inference API	27
2.2.7 Nginx	28

2.2.8	Docker & Docker Compose	29
2.2.9	Các thuật toán và kiến trúc gợi ý sản phẩm	29
3	PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	35
3.1	Yêu cầu hệ thống	35
3.1.1	Yêu cầu chức năng	35
3.1.2	Yêu cầu phi chức năng	36
3.2	Kiến trúc hệ thống	37
3.2.1	Sơ đồ kiến trúc tổng thể	37
3.2.2	Chi tiết Backend Architecture	38
3.2.3	Chi tiết Frontend Architecture	40
3.3	Thiết kế use case	46
3.3.1	Biểu đồ use case tổng quan	46
3.3.2	Biểu đồ use case phân rã	46
3.3.3	Đặc tả use case	46
3.4	Thiết kế Cơ sở dữ liệu	73
3.4.1	Sơ đồ dữ liệu	73
3.4.2	Bảng CSDL	74
3.5	Thiết kế Giao diện	84
3.5.1	Danh sách các giao diện	84
3.5.2	Chi tiết các giao diện	85
4	XÂY DỰNG ỨNG DỤNG VÀ KIỂM THỬ CHƯƠNG TRÌNH	115
4.1	Tổ chức thư mục của dự án	115
4.2	Kiểm thử chương trình	116
4.2.1	Kiểm thử các chức năng đã thực hiện	116
4.2.2	Kiểm thử các yêu cầu phi chức năng	124
4.3	Kết quả Thực nghiệm và Đo lường Mô hình Gợi ý Thông minh Lai (‘ai-service-v2’)	125
4.3.1	Thiết lập Môi trường Thực nghiệm và Dữ liệu Kiểm thử v5.1	125
4.3.2	Bảng Kết quả Đánh giá So sánh trên Toàn bộ Danh mục	125
	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	127
	TÀI LIỆU THAM KHẢO	129

Danh sách hình vẽ

2.1	Logo Node.js	24
2.2	Logo Express.js	25
2.3	Logo PostgreSQL	25
2.4	Logo RabbitMQ	26
2.5	Logo Redis	27
2.6	Logo Hugging Face	27
2.7	Logo Nginx	28
2.8	Logo Docker	29
3.1	Sơ đồ kiến trúc tổng thể hệ thống POSMART	38
3.2	Chi tiết phân rã 8 Microservices trong Backend	40
3.3	Sơ đồ kiến trúc cấu trúc các tầng logic trên Frontend	41
3.4	Sơ đồ luồng xử lý RAG	42
3.5	Biểu đồ use case tổng quan	46
3.6	Sơ đồ cơ sở dữ liệu phân tán của hệ thống POSMART	74
3.7	Giao diện Trang chủ Storefront (UI-01)	85
3.8	Giao diện Danh sách sản phẩm (UI-02)	87
3.9	Giao diện Chi tiết sản phẩm (UI-03)	88
3.10	Giao diện Giỏ hàng (UI-04)	89
3.11	Giao diện Thanh toán đơn hàng (UI-05)	90
3.12	Giao diện Lịch sử đơn hàng (UI-06)	91
3.13	Giao diện Đăng nhập Email/Password (UI-07)	92
3.14	Giao diện Đăng nhập POS nhanh (UI-08)	93
3.15	Giao diện Đăng ký tài khoản (UI-09)	94
3.16	Giao diện Dashboard Super Admin (UI-10)	95
3.17	Giao diện Quản lý tài khoản (UI-12)	96
3.18	Giao diện Quản lý phân quyền (UI-13)	97
3.19	Giao diện Cấu hình hệ thống (UI-14)	98
3.20	Giao diện Dashboard Manager (UI-15)	99
3.21	Giao diện Quản lý danh mục sản phẩm (UI-16)	100
3.22	Giao diện Quản lý sản phẩm (UI-17)	101
3.23	Giao diện Quản lý nhà cung cấp (UI-18)	102
3.24	Giao diện Quản lý đơn nhập hàng (UI-19)	103
3.25	Giao diện Nhập kho hàng hóa (UI-20)	104
3.26	Giao diện Quản lý tồn kho (UI-21)	105
3.27	Giao diện Sơ đồ kho bãi (UI-22)	106
3.28	Giao diện Quản lý lô hàng (UI-23)	107
3.29	Giao diện Phiếu xuất kho (UI-24)	108
3.30	Giao diện Quản lý khách hàng (UI-25)	109

3.31	Giao diện Báo cáo doanh thu (UI-26)	110
3.32	Giao diện Màn hình POS chính (UI-27)	111
3.33	Giao diện Quản lý đơn hàng chờ (UI-28)	112
3.34	Giao diện Thanh toán VNPay QR Code (UI-29)	113
3.35	Giao diện AI Chatbot Assistant (UI-30)	114

Danh sách bảng

1.1	Ma trận phân tích khoảng cách năng lực, điểm nghẽn kiến trúc và nợ kỹ thuật của các giải pháp hiện hành	17
2.1	So sánh giữa kiến trúc Hybrid Ensemble và Pure Deep Learning	30
3.1	Phân tích độ phức tạp Batch Job cho Collaborative Filtering	44
3.2	Các cluster ngữ cảnh mua sắm trong Session Context Boost	45
3.3	Tổng hợp độ phức tạp và hiệu năng của hệ thống gợi ý	46
3.4	Đặc tả Usecase: Đăng nhập	47
3.5	Đặc tả Usecase: Tạo hóa đơn (Create order)	47
3.6	Đặc tả Usecase: Lưu hóa đơn chờ (Hold order)	48
3.7	Đặc tả Usecase: Quản lý đơn hàng chờ (Management hold order)	48
3.8	Đặc tả Usecase: Truy vấn hóa đơn (Query order)	49
3.9	Đặc tả Usecase: Return order	49
3.10	Đặc tả Usecase: Đăng nhập	50
3.11	Đặc tả Usecase: Manage Product Category	50
3.12	Đặc tả Usecase: Tìm kiếm Category	51
3.13	Đặc tả Usecase: Manage Product	51
3.14	Đặc tả Usecase: Tìm kiếm Product	52
3.15	Đặc tả Usecase: Manage supplier	52
3.16	Đặc tả Usecase: Tìm kiếm supplier	53
3.17	Đặc tả Usecase: View Inventory	54
3.18	Đặc tả Usecase: Manage Movement	54
3.19	Đặc tả Usecase: Manage Batch	55
3.20	Đặc tả Usecase: Tạo chỉnh sửa tồn kho	56
3.21	Đặc tả Usecase: Quản lý hủy lô hàng	56
3.22	Đặc tả Usecase: View Disposal History	57
3.23	Đặc tả Usecase: Quản lý Role	57
3.24	Đặc tả Usecase: Tìm kiếm Role	59
3.25	Đặc tả Usecase: Quản lý Tài khoản	59
3.26	Đặc tả Usecase: Tìm kiếm Account	61
3.27	Đặc tả Usecase: Quản lý Đơn nhập hàng (Purchase Order)	62
3.28	Đặc tả Usecase: Nhập kho hàng hóa (Receive Stock)	62
3.29	Đặc tả Usecase: Quản lý Phiếu Xuất Kho (Stock Out)	63
3.30	Đặc tả Usecase: Thanh toán qua cổng VNPAY	64
3.31	Đặc tả Usecase: Tương tác với Trợ lý AI (Chatbot)	65
3.32	Đặc tả Usecase: Quản lý Cấu hình hệ thống (System Settings)	65
3.33	Đặc tả Usecase: Xem Báo cáo thống kê (View Reports)	66
3.34	Đặc tả Usecase: Quản lý Khách hàng (Manage Customer)	67

3.35	Đặc tả Usecase: Quản lý Vị trí Kho bãi (Warehouse Locations)	68
3.36	Đặc tả Usecase: Cấp quyền và Bảo mật POS (Manage POS Auth)	68
3.37	Đặc tả Usecase: Quản lý chi nhánh (Manage Store)	69
3.38	Đặc tả Usecase: Khách hàng tự đăng ký (Customer Self-Registration)	70
3.39	Đặc tả Usecase: Điều chuyển hàng lên kệ (Move Stock to Shelf)	71
3.40	Đặc tả Usecase: Xem lịch sử biến động giá (View Price History)	72
3.41	Đặc tả Usecase: Hoàn tiền giao dịch (Refund Payment)	72
3.42	Bảng store	74
3.43	Bảng permission	75
3.44	Bảng role	75
3.45	Bảng role_permission	75
3.46	Bảng user_account	75
3.47	Bảng employee	76
3.48	Bảng customer	76
3.49	Bảng auth_tokens	76
3.50	Bảng pos_auth	76
3.51	Bảng category	77
3.52	Bảng product	77
3.53	Bảng product_price_history	77
3.54	Bảng sale_order	78
3.55	Bảng sale_order_detail	78
3.56	Bảng security_settings	78
3.57	Bảng sales_settings	79
3.58	Bảng settings_history	79
3.59	Bảng supplier	79
3.60	Bảng purchase_order	79
3.61	Bảng purchase_order_detail	80
3.62	Bảng product_batch	80
3.63	Bảng warehouse_block	81
3.64	Bảng location	81
3.65	Bảng inventory_item	81
3.66	Bảng inventory_movement	81
3.67	Bảng stock_out_order	82
3.68	Bảng stock_out_detail	82
3.69	Bảng payment	82
3.70	Bảng vnpay_transaction	83
3.71	Bảng chat_session	83
3.72	Bảng chat_message	84
3.73	Danh sách các giao diện của hệ thống POSMART	86
4.1	Testcase TC-RAG-01: RRF Fusion	117
4.2	Testcase TC-RAG-02: Cold-start sản phẩm mới	117
4.3	Testcase TC-RAG-03: Giới hạn Top 5	117
4.4	Testcase TC-APR-01: Tính toán Apriori	118
4.5	Testcase TC-APR-02: Edge Case Division by Zero	118
4.6	Testcase TC-APR-03: Lọc tương quan ngẫu nhiên	119
4.7	Testcase TC-CF-01: Cosine Similarity cao	119
4.8	Testcase TC-CF-02: Pruning cặp ít dữ liệu	119

4.9	Testcase TC-CF-03: Cold-start User Redistribution	120
4.10	Testcase TC-ENS-01: Ensemble Score User VIP	120
4.11	Testcase TC-ENS-02: Apriori quyết định thứ hạng	120
4.12	Testcase TC-SES-01: Nhận diện cluster lẫu bò	121
4.13	Testcase TC-SES-02: Không boost khi duyệt tổng quát	121
4.14	Testcase TC-WL-01: Cập nhật trọng số qua Exponential Smoothing	122
4.15	Testcase TC-WL-02: Guard Rail MIN_FEEDBACK_COUNT	122
4.16	Testcase TC-WL-03: Guard Rail Clamping	122
4.17	Testcase TC-E2E-01: Kịch bản tổng hợp lẫu bò	123
4.18	Testcase TC-E2E-02: Phễu Feedback Loop 5 bước	123
4.19	Kết quả kiểm thử hiệu suất từng thành phần	124
4.20	Kết quả kiểm thử cách ly dữ liệu Dual-Tracking	125
4.21	Kết quả đánh giá so sánh các mô hình trên tập kiểm thử Dataset Snapshot v5.1 .	126

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh đầy đủ	Ý nghĩa / Khái niệm tiếng Việt
2FA	Two-Factor Authentication	Xác thực hai yếu tố (Bảo mật đa tầng)
ACID	Atomicity, Consistency, Isolation, Durability	Bốn thuộc tính toàn vẹn của giao dịch CSDL
API	Application Programming Interface	Giao diện lập trình ứng dụng
BOPIS	Buy Online, Pick Up In Store	Mô hình mua hàng trực tuyến và nhận tại quầy
BPR	Bayesian Personalized Ranking	Thuật toán tối ưu hóa xếp hạng cá nhân hóa
CF	Collaborative Filtering	Lọc cộng tác (Gợi ý dựa trên người dùng tương đồng)
CQRS	Command Query Responsibility Segregation	Mẫu kiến trúc tách biệt luồng Ghi và luồng Đọc
DDD	Domain-Driven Design	Phương pháp thiết kế phần mềm hướng miền nghiệp vụ
FTS	Full-Text Search	Tìm kiếm toàn văn trên cơ sở dữ liệu
GAUC	Group Area Under Curve	Độ đo phân loại xếp hạng theo từng nhóm người dùng
GIN	Generalized Inverted Index	Cấu trúc chỉ mục đảo tổng quát trong PostgreSQL
HNSW	Hierarchical Navigable Small World	Cấu trúc đồ thị chỉ mục vector tiệm cận gần đúng
HR@K	Hit Ratio at rank K	Tỷ lệ dự đoán trúng mục tiêu trong top K gợi ý
IPN	Instant Payment Notification	Cơ chế Webhook thông báo giao dịch tức thời
JWT	JSON Web Token	Chuỗi mã thông báo định danh và phân quyền
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NDCG@K	Normalized Discounted Cumulative Gain	Độ đo tăng ích tích lũy giảm dần chuẩn hóa tại rank K
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ thực thể cơ sở dữ liệu sang đối tượng
OTP	One-Time Password	Mật khẩu dùng một lần
POS	Point of Sale	Điểm bán hàng / Thiết bị bán lẻ tại quầy

Từ viết tắt	Thuật ngữ tiếng Anh đầy đủ	Ý nghĩa / Khái niệm tiếng Việt
RAG	Retrieval-Augmented Generation	Kỹ thuật tạo câu trả lời tăng cường bằng truy xuất
RBAC	Role-Based Access Control	Mô hình kiểm soát truy cập dựa trên vai trò
RLS	Row-Level Security	Chính sách bảo mật cách ly dữ liệu ở mức từng dòng
RRF	Reciprocal Rank Fusion	Thuật toán hợp nhất thứ hạng tương hỗ
SAGA	Saga Pattern	Mẫu điều phối chuỗi giao dịch cục bộ phân tán
SKU	Stock Keeping Unit	Đơn vị phân loại và lưu kho hàng hóa riêng biệt
SPOF	Single Point of Failure	Điểm lỗi đơn lẻ gây sập toàn bộ hệ thống
TOTP	Time-Based One-Time Password	Mật khẩu dùng một lần dựa trên đồng hồ thời gian
UI / UX	User Interface / User Experience	Giao diện người dùng / Trải nghiệm người dùng

Chương 1

TỔNG QUAN ĐỀ TÀI

1.1 Động lực nghiên cứu và lý do chọn đề tài

1.1.1 Bối cảnh chuyển dịch mô hình bán lẻ và sự cấp thiết của đề tài

Trong bối cảnh nền kinh tế số phát triển mạnh mẽ, ngành bán lẻ truyền thống đang trải qua cuộc chuyển dịch sâu sắc sang mô hình bán lẻ đa kênh tích hợp (**Omnichannel Retailing**). Người tiêu dùng hiện đại không chỉ mua hàng trực tiếp tại các điểm bán lẻ vật lý (Point of Sale - POS) mà còn có nhu cầu tra cứu thông tin sản phẩm, kiểm tra lượng tồn kho thực tế, mua sắm qua các ứng dụng trực tuyến và nhận hàng tại nhà hoặc tại cửa hàng gần nhất (mô hình BOPIS - Buy Online, Pick Up In Store).

Để duy trì năng lực cạnh tranh, các chuỗi cửa hàng tiện lợi và siêu thị mini đòi hỏi một nền tảng công nghệ quản trị đủ mạnh để xử lý khối lượng dữ liệu giao dịch khổng lồ theo thời gian thực. Nền tảng này vừa phải đảm bảo thông lượng xử lý nhanh chóng cho các giao dịch bán buôn, bán lẻ trực tiếp tại quầy thanh toán, vừa phải đồng bộ tức thời số liệu tồn kho, biến động giá và các chương trình khuyến mãi xuyên suốt toàn bộ các chi nhánh.

1.1.2 Thách thức kỹ thuật cốt lõi trong kiến trúc hệ thống bán lẻ phân tán

Tuy nhiên, việc xây dựng và vận hành một hệ thống quản lý chuỗi bán lẻ đa chi nhánh đặt ra những bài toán kỹ thuật phức tạp:

- **Xung đột về đặc thù tải dữ liệu (Load Asymmetry):** Trong hệ thống bán lẻ, luồng tra cứu danh mục hàng hóa (Catalog) có tần suất đọc cực lớn (*Read-heavy*), trong khi luồng trừ tồn kho và thanh toán hóa đơn (Inventory & Order) lại có tần suất ghi liên tục (*Write-heavy*). Nếu xây dựng theo kiến trúc nguyên khối (Monolithic Architecture) dùng chung một cơ sở dữ liệu duy nhất, các tác vụ nặng (như truy vấn báo cáo doanh thu) sẽ gây khóa bảng, nghẽn I/O và làm tê liệt toàn bộ hoạt động thanh toán tại quầy POS của các chi nhánh.
- **Bài toán chi phí và cô lập dữ liệu (Multi-tenancy Trade-off):** Thiết lập cơ sở dữ liệu riêng biệt cho từng chi nhánh (Database-per-tenant) gây lãng phí nghiêm trọng tài nguyên máy chủ và chi phí bảo trì. Ngược lại, việc chia sẻ cơ sở dữ liệu (Shared Database, Shared Schema) đòi hỏi cơ chế cách ly dữ liệu ở mức dòng (**Row-Level Security**) tuyệt đối thông qua mã định danh `store_id`, ngăn ngừa rủi ro rò rỉ dữ liệu chéo giữa các cửa hàng.
- **Toàn vẹn giao dịch phân tán (Distributed Data Consistency):** Trong môi trường phân tán gồm nhiều dịch vụ độc lập, giao dịch tài chính (thanh toán VNPay) và trừ tồn kho

diễn ra trên các cơ sở dữ liệu logic khác nhau. Việc áp dụng giao thức 2-Phase Commit (2PC) truyền thống sẽ làm tăng độ trễ và giảm tính khả dụng. Do đó, việc áp dụng mô hình **Saga Pattern** kết hợp kiến trúc hướng sự kiện (**Event-Driven**) qua Message Broker là giải pháp sống còn để đảm bảo tính nhất quán cuối cùng (*Eventual Consistency*).

1.1.3 Động lực nâng cấp từ Đề án 2 lên Khóa luận tốt nghiệp

Trong giai đoạn Đề án 2 (SE122.P21), nhóm nghiên cứu đã xây dựng thành công bộ khung Backend gồm 8 Microservices cơ bản bằng Node.js (Express), thiết lập luồng SAGA thanh toán qua RabbitMQ và cổng VNPAY Sandbox.

Nhằm phát triển đề tài thành một công trình **Khóa luận tốt nghiệp Kỹ sư (SE505)** có giá trị khoa học và tính ứng dụng thực tiễn cao, đề tài đặt ra động lực nghiên cứu nâng cấp toàn diện:

1. **Hoàn thiện kiến trúc Backend chuẩn doanh nghiệp:** Bổ sung dịch vụ **Statistics Service (:3009)** ứng dụng bộ nhớ đệm phân tán Redis Caching để tổng hợp số liệu thời gian thực; chuẩn hóa cơ chế **Transactional Outbox** và **Idempotency Guard** (processed_events) nhằm triệt tiêu hoàn toàn lỗi trùng lặp tin nhắn hoặc mất mát sự kiện.
2. **Nâng cấp Trợ lý ảo tác vụ hai chiều (Action Assistant v2.0):** Khắc phục nhược điểm của các mô hình Chatbot chỉ đọc (Read-only RAG) thông thường bằng việc thiết kế giao thức **Action Response Protocol** và chốt chặn xác nhận (**Confirmation Gate**) bảo mật 7 lớp, cho phép AI thực thi các hành động cụ thể trên giao diện (thêm vào giỏ hàng, tra cứu tồn kho chi nhánh, giữ đơn tại quầy) mà vẫn giữ Backend ở trạng thái phi trạng thái (Stateless).
3. **Mũi nhọn nghiên cứu mới – Hệ gợi ý thông minh lai ('ai-service-v2'):** Thay thế các thuật toán heuristic cảm tính bằng một Recommender System độc lập được xây dựng bài bản. Mô hình kết hợp giữa ****Wide Tower**** (Luật kết hợp Apriori trên giỏ hàng mua sắm tự nhiên) và ****Deep Two-Tower**** (Mạng nơ-ron tháp đôi tích hợp chiều đặc trưng ngữ nghĩa sản phẩm, tối ưu qua hàm mất mát Bayesian Personalized Ranking - BPR). Đồng thời, thiết lập ****Quy trình Benchmark khoa học có thể tái lập**** với đánh giá xếp hạng trên toàn bộ danh mục sản phẩm (*Full-catalog ranking*), phục vụ mục tiêu công bố bài báo khoa học.

1.2 Khảo sát hiện trạng và Hệ thống đề xuất

1.2.1 Hiện trạng các nền tảng bán lẻ và thương mại điện tử

Thị trường công nghệ bán lẻ hiện nay phân bổ thành ba nhóm giải pháp chính:

1. **Nền tảng SaaS đóng gói nội địa (KiotViet, Sapo, Haravan):** Phổ biến rộng rãi tại thị trường Việt Nam cho các hộ kinh doanh cá thể nhờ chi phí thấp và giao diện trực quan. Tuy nhiên, các giải pháp này được xây dựng trên kiến trúc nguyên khối đóng kín, không cung cấp API mở để tích hợp sâu hệ thống phân tán, hoàn toàn thiếu vắng phân hệ trợ lý ảo đàm thoại thông minh (AI Chatbot) và hệ gợi ý sản phẩm học máy sâu.
2. **Nền tảng mã nguồn mở quốc tế (WooCommerce, Magento, Odoo POS):** Cho phép tùy biến mã nguồn cao nhưng đòi hỏi chi phí vận hành hạ tầng rất lớn. WooCommerce và Magento gặp hiện tượng nghẽn cổ chai nghiêm trọng tại cơ sở dữ liệu quan hệ (MySQL)

khi số lượng SKU và lượt truy cập tăng cao. Odoo POS có cấu trúc module hóa nhưng viết bằng Python đồng bộ, khó phân rã để co giãn độc lập từng dịch vụ khi có tải đột biến.

3. **Hệ thống đám mây quy mô lớn (Shopify Plus, Amazon):** Khả năng chịu tải và độ tin cậy vượt trội, song chi phí bản quyền và hoa hồng giao dịch định kỳ quá đắt đỏ đối với các doanh nghiệp vừa và nhỏ tại Việt Nam. Đồng thời, việc can thiệp để tích hợp sâu sơ đồ quản lý vị trí kho bãi chi tiết theo từng ô/kệ (Warehouse Location) và chính sách kiểm soát nội bộ chuỗi bị giới hạn nghiêm ngặt.

1.2.2 Phân tích các hạn chế kỹ thuật phổ biến

Qua khảo sát, các hệ thống bán lẻ hiện nay tồn tại ba điểm nghẽn kỹ thuật cốt lõi:

- **Nghẽn cổ chai tại tầng dữ liệu (Database Bottleneck):** Các tác vụ đọc thống kê doanh thu nặng nề tranh chấp tài nguyên trực tiếp với các tác vụ ghi nhật ký tại quầy POS, dễ gây treo hệ thống trong giờ cao điểm.
- **Hiện tượng bán vượt tồn kho (Overselling) và sai lệch trạng thái:** Thiếu vắng cơ chế điều phối giao dịch phân tán và hàng đợi tin nhắn bất đồng bộ tin cậy, dẫn đến tình trạng đơn hàng đã thanh toán nhưng kho không còn hàng để xuất.
- **Nợ kỹ thuật từ việc lạm dụng API AI thương mại:** Nhiều ứng dụng tích hợp AI bằng cách gọi thẳng API bên thứ ba (OpenAI GPT, Claude). Giải pháp ngắn hạn này làm phát sinh chi phí biến đổi tăng phi mã, độ trễ phản hồi quá lớn (> 2 giây) không phù hợp cho quầy bán lẻ, và tiềm ẩn nguy cơ rò rỉ toàn bộ bí mật kinh doanh và dữ liệu khách hàng.

1.2.3 Ma trận phân tích khoảng cách năng lực, điểm nghẽn kiến trúc và nợ kỹ thuật

Nhằm đối sánh một cách khoa học và làm rõ cơ sở cho các quyết định kiến trúc của đề tài, Bảng 1.1 tổng hợp ma trận phân tích 3 chiều theo chuẩn mực nghiên cứu phần mềm:

1.2.4 Hệ thống kiến trúc đề xuất (POSMART Enterprise Solution)

Để giải quyết triệt để các hạn chế trên, đề tài đề xuất kiến trúc hệ thống ****POSMART**** toàn diện:

1. **Hạ tầng 9 Microservices chuyên biệt:** Phân rã độc lập theo nguyên lý Domain-Driven Design (DDD): Auth (:3001), Catalog (:3002), Order (:3003), Settings (:3004), Supplier (:3005), Inventory (:3006), Payment (:3007), Chatbot (:3008), và Statistics (:3009).
2. **API Gateway Nginx (:8080) tập trung:** Xử lý xác thực sơ bộ, SSL Termination, nén GZIP, gắn mã vết X-Request-ID xuyên suốt và kiểm soát lưu lượng với Rate Limiting 3 tầng.
3. **Giao tiếp bất đồng bộ & Saga Pattern:** Sử dụng RabbitMQ Topic Exchange posmart.events kết hợp mẫu thiết kế Transactional Outbox và Idempotency Guard, đảm bảo các giao dịch thanh toán và trừ kho luôn toàn vẹn.

Tiêu chí phân tích	Biểu hiện thực tế & Nguyên nhân kỹ thuật	Hệ quả & Điểm nghẽn hệ thống
1. Khoảng cách năng lực và Ràng buộc hệ thống (Capability Gap)	- Các giải pháp POS SaaS đóng gói sẵn chỉ cung cấp các thao tác CRUD truyền thống, không có cơ chế xử lý ngôn ngữ tự nhiên. - Hoàn toàn thiếu hụt mô hình học máy gợi ý sản phẩm cá nhân hóa sâu theo hành vi thời gian thực. - Tìm kiếm sản phẩm phụ thuộc vào chuỗi từ khóa chính xác (Exact match), không xử lý được lỗi chính tả hay ngữ nghĩa tương đương.	- Bỏ lỡ cơ hội bán chéo sản phẩm (Cross-selling) để gia tăng giá trị giỏ hàng. - Nhân viên thu ngân mất nhiều thời gian tìm kiếm mặt hàng thay thế khi một mã hàng bị hết. - Trải nghiệm mua sắm của khách hàng trực tuyến bị gián đoạn, tỷ lệ chuyển đổi đơn hàng suy giảm.
2. Điểm nghẽn hệ thống và Nguồn gốc vấn đề (Architectural Bottlenecks)	- Bắt nguồn từ bản chất thiết kế nguyên khối (Monolithic Architecture) chia sẻ chung một cơ sở dữ liệu quan hệ vật lý. - Khi lưu lượng truy cập online tăng đột biến, tài nguyên CPU/RAM của database bị quá tải do các truy vấn JOIN phức tạp. - Chiến lược Multi-tenancy nếu triển khai theo dạng Database-per-tenant gây lãng phí tài nguyên và cạn kiệt Connection Pooling.	- Sự cố nghẽn cổ chai tại phân hệ báo cáo làm tê liệt toàn bộ luồng bán hàng tại máy POS của các chi nhánh (Single Point of Failure). - Chi phí thuê máy chủ tăng cao đột biến nhưng hiệu suất phục vụ không tăng tương xứng.
3. Nợ kỹ thuật và Sự đánh đổi kiến trúc (Technical Debt)	- Trong nỗ lực tích hợp AI, các hệ thống thường phụ thuộc hoàn toàn vào việc gọi API đám mây thương mại (OpenAI, Anthropic). - Bỏ qua tính tự chủ hạ tầng, độ trễ đường truyền mạng và sự kiểm soát mã nguồn. - Thiếu vắng hệ thống kiểm thử tự động phân tầng (Unit/Integration Test) và cơ chế đền bù giao dịch phân tán (Saga Compensation).	- Rủi ro chi phí vận hành biến đổi tăng mất kiểm soát theo số lượt request. - Độ trễ Chatbot lớn (> 2s), gây gián đoạn luồng tư vấn tại quầy. - Nguy cơ rò rỉ dữ liệu mật về giá vốn, nhà cung cấp và lịch sử mua sắm của khách hàng ra bên ngoài.

Bảng 1.1: Ma trận phân tích khoảng cách năng lực, điểm nghẽn kiến trúc và nợ kỹ thuật của các giải pháp hiện hành

4. **Cơ chế Row-Level Multi-Tenancy:** Sử dụng chung một hạ tầng PostgreSQL nhưng cô lập dữ liệu an toàn đến từng dòng thông qua trường `store_id` được trích xuất từ JWT Payload.
5. **Hệ gợi ý thông minh lai độc lập ('ai-service-v2'):** Kết hợp Wide Apriori và Deep Two-Tower (BPR Loss) với giao thức benchmark khoa học độc lập, kết hợp Trợ lý tác vụ hai chiều Action Assistant v2.0.

1.3 Đối tượng và Phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu chuyên sâu vào các mô hình kiến trúc phần mềm, mẫu thiết kế và thuật toán học máy:

1. **Kiến trúc hệ thống phân tán (Distributed Architecture):** Kiến trúc Microservices, API Gateway Pattern, Event-Driven Architecture với Message Broker (RabbitMQ AMQP 0-9-1), Mẫu Saga Choreography kết hợp Transactional Outbox và Idempotent Consumer.
2. **Quản trị dữ liệu đa người thuê (Multi-tenancy):** Chiến lược Row-Level Security (Shared Database, Shared Schema) trên PostgreSQL 16; Cơ chế lưu trữ phân tầng Polyglot Persistence (PostgreSQL cho ACID, Redis cho In-memory Caching, pgvector cho Vector Embeddings).
3. **Trợ lý tác vụ đàm thoại (Action Assistant):** Kỹ thuật RAG kết hợp Semantic Search (Vector nhúng 768 chiều, chỉ mục HNSW) và Full-Text Search (chỉ mục GIN) qua thuật toán hợp nhất thứ hạng tương hỗ (RRF); Giao thức Action Response Protocol và chốt chặn xác nhận Confirmation Gate bảo mật 7 lớp.
4. **Hệ gợi ý lai (Hybrid Recommender System):** Thuật toán khai phá luật kết hợp Apriori; Mạng nơ-ron Deep Two-Tower tối ưu hóa bằng hàm mất mát Bayesian Personalized Ranking (BPR Loss); Mô hình kết hợp cộng tính Additive Fusion với chuẩn hóa Z-score theo từng người dùng.

1.3.2 Phạm vi nghiên cứu

- **Quy mô ứng dụng:** Áp dụng cho chuỗi siêu thị mini và cửa hàng tiện lợi bán lẻ có quy mô vừa và nhỏ (dưới 100 chi nhánh).
- **Phân hệ hệ thống trong phạm vi thực hiện:**
 - Hoàn thiện trọn vẹn 9 Microservices Backend (Auth, Catalog, Order, Settings, Supplier, Inventory, Payment, Chatbot, Statistics).
 - Hoàn thiện phân hệ nghiên cứu thực nghiệm Recommender Runner (ai-service-v2) trên Python với quy trình benchmark khoa học độc lập.
 - Xây dựng hoàn chỉnh 3 phân hệ giao diện người dùng: Giao diện bán hàng POS tại quầy (POS Terminal), Giao diện quản trị chuỗi và cửa hàng (Admin & Manager Dashboard), và Giao diện khách hàng trực tuyến (Customer Storefront).
- **Ranh giới loại trừ (Out of Scope):**

- Đề tài không đi sâu vào bài toán logistics vận chuyển thực địa (quản lý lộ trình xe tải, tài xế giao hàng ngoài đường phố).
 - Phiên bản ai-service cũ (heuristic Node.js) được chuyển vào trạng thái lưu trữ (quarantined audit), toàn bộ phát triển khoa học mới được thực hiện trong ai-service-v2.
- **Môi trường tích hợp bên thứ ba:** Sử dụng môi trường thử nghiệm Sandbox chính thức của cổng thanh toán điện tử VNPay; sử dụng Hugging Face Inference API cho mô hình ngôn ngữ lớn (Qwen2.5-7B-Instruct / Mistral-7B).

1.4 Mục tiêu đề tài và Các yêu cầu hệ thống

1.4.1 Yêu cầu chức năng hệ thống (Functional Requirements)

Hệ thống đáp ứng đầy đủ các yêu cầu chức năng nghiệp vụ, phân rõ theo 4 nhóm tác nhân chính:

1. Phân hệ Khách hàng (Customer):

- Đăng ký tài khoản trực tuyến (gán vai trò Customer thuộc cấp toàn chuỗi), đăng nhập JWT, quản lý sổ địa chỉ giao hàng và thông tin cá nhân.
- Duyệt danh mục ngành hàng, tìm kiếm sản phẩm với cơ chế Debounce thời gian thực, lọc sản phẩm đa tiêu chí (danh mục, khoảng giá, trạng thái còn hàng).
- Xem chi tiết sản phẩm, bảng thông số kỹ thuật và các sản phẩm gợi ý mua kèm liên quan.
- Quản lý giỏ hàng, cập nhật số lượng, áp dụng mã giảm giá (Discount code) và chọn hình thức nhận hàng (giao hàng hoặc nhận tại cửa hàng - Pickup).
- Thanh toán an toàn trực tuyến qua cổng VNPay (quét mã QR) hoặc thanh toán khi nhận hàng (COD).
- Tra cứu lịch sử mua sắm và theo dõi tiến trình xử lý đơn hàng theo thời gian thực.
- Tương tác với Trợ lý ảo AI Chatbot bằng ngôn ngữ tự nhiên để tìm kiếm mặt hàng, nhận gợi ý sản phẩm và ra lệnh thêm hàng vào giỏ hàng tự động.

2. Phân hệ Thu ngân / Nhân viên bán hàng tại quầy (Cashier / Store Staff):

- Đăng nhập nhanh tại quầy POS bằng mã PIN 4-6 số được mã hóa một chiều Bcrypt, có cơ chế tự động khóa tài khoản khi nhập sai quá 5 lần.
- Màn hình bán hàng POS chuyên dụng: Quét mã vạch sản phẩm (Barcode scanner) hoặc tìm kiếm nhanh, hỗ trợ chọn đúng lô hàng còn hạn sử dụng.
- Quản lý đơn hàng chờ (Hold Orders): Cho phép lưu tạm trạng thái đơn hàng đang thanh toán dở để phục vụ khách hàng tiếp theo, và tải lại đơn chờ khi khách quay lại.
- Xử lý thanh toán đa phương thức: Thu tiền mặt (tự động tính tiền thối), hiển thị mã QR VNPay động trên màn hình POS để khách quét mã thanh toán tức thời.
- In hóa đơn bán lẻ (Thermal receipt) và tự động kích hoạt luồng trừ kho trên hệ thống.

3. Phân hệ Quản lý cửa hàng chi nhánh (Store Manager):

- Quản lý đơn đặt hàng nhà cung cấp (Purchase Order): Tạo đơn nhập hàng, duyệt đơn, và theo dõi tiến độ giao hàng từ nhà cung cấp.
- Nghiệm thu nhập kho (Receive Stock): Ghi nhận số lượng hàng thực tế nhận được, cập nhật ngày sản xuất, hạn sử dụng của từng lô (Batch), và gán vị trí lưu kho (Location) trên sơ đồ kho.
- Quản lý sơ đồ kho bãi (Warehouse Map): Thiết lập trực quan các khu vực kho (Block) và các ô/kệ hàng (Location) với sức chứa tối đa và cảnh báo tỷ lệ lấp đầy kho.
- Điều chuyển hàng hóa nội bộ (Stock Movement): Thực hiện chuyển hàng từ kho chung (On-Hand) lên kệ bán lẻ (On-Shelf) và ghi nhận Audit Log.
- Quản lý phiếu xuất kho hủy hàng (Stock Out): Lập phiếu xuất hủy đối với các lô hàng hết hạn, hàng hư hỏng hoặc xuất trả nhà cung cấp.
- Theo dõi công nợ nhà cung cấp (Supplier Debt) và xem báo cáo kinh doanh nội bộ của chi nhánh.

4. Phân hệ Quản trị viên cấp cao (Super Admin / Chain Owner):

- Quản lý danh sách chi nhánh cửa hàng: Thêm mới chi nhánh, sinh mã `store_id` tự động và gán nhân sự quản lý phụ trách.
- Quản trị danh mục hàng hóa tập trung (Centralized Catalog): Thêm/sửa/xóa sản phẩm dùng chung toàn chuỗi, thiết lập giá niêm yết chuẩn, tra cứu lịch sử biến động giá (Price History).
- Quản lý tài khoản và phân quyền (User & RBAC): Quản lý danh sách người dùng, kích hoạt/khóa tài khoản, cấu hình vai trò (Role) và gán nhóm quyền hạn (Permissions).
- Quản lý chính sách hệ thống (System Settings): Cấu hình tham số bảo mật (số lần nhập sai PIN tối đa, thời gian khóa tài khoản), thiết lập tỷ lệ chiết khấu theo phân hạng khách hàng (VIP/Wholesale/Retail) và kích hoạt chế độ tự động giảm giá ban đêm cho hàng cận hạn sử dụng.
- Dashboard giám sát toàn diện: Xem báo cáo thống kê doanh số toàn chuỗi, theo dõi hiệu năng hệ thống qua bộ nhớ đệm Redis Cache.

1.4.2 Yêu cầu dữ liệu (Data Requirements)

Hệ thống đáp ứng 5 trụ cột kiến trúc dữ liệu nghiêm ngặt:

1. **Tính nhất quán và toàn vẹn giao dịch (ACID & Eventual Consistency):** Mọi giao dịch cục bộ bên trong từng microservice đều tuân thủ ACID qua PostgreSQL Transaction. Các giao dịch xuyên dịch vụ (Cross-service) giữa Payment, Order và Inventory bắt buộc thực thi qua mô hình Saga Choreography trên RabbitMQ, tự động kích hoạt đền bù (Rollback) khi có lỗi.
2. **Cách ly dữ liệu đa người thuê (Multi-tenant Data Isolation):** Toàn bộ các bảng dữ liệu nghiệp vụ chi nhánh (`sale_order`, `product_batch`, `inventory_item`, `purchase_order`, `payment`) đều bắt buộc có trường khóa ngoại `store_id`. Middleware tự động trích xuất `storeId` từ JWT Payload đã được ký số để áp vào câu truy vấn, ngăn chặn hoàn toàn việc client tự can thiệp `store_id`.

3. **Linh hoạt bán cấu trúc và ghi nhật ký kiểm toán (Semi-structured & Auditability):** Sử dụng kiểu dữ liệu JSONB để lưu trữ các thuộc tính cấu hình động, danh sách mặt hàng trong snapshot đơn thanh toán, và siêu dữ liệu phiên hội thoại. Bảng `settings_history` và `inventory_movement` bắt buộc lưu vết thời gian, người thực hiện và lý do thay đổi.
4. **Lưu trữ Vector nhúng và Tìm kiếm Ngữ nghĩa (Vector Database):** Tích hợp mở rộng `pgvector` trên PostgreSQL tại dịch vụ Chatbot, lưu trữ vector nhúng 768 chiều (Vietnamese SBERT) với chỉ mục HNSW phục vụ tìm kiếm ngữ nghĩa, kết hợp cột `TSVECTOR` với chỉ mục GIN cho tìm kiếm từ khóa.
5. **Tính bất biến của Dữ liệu Thực nghiệm (ML Benchmark Lineage):** Bảng `sale_order` hỗ trợ các trường định danh nghiên cứu `benchmark_run_id`, `benchmark_kind`. Module `ai-service-v2` đóng gói dữ liệu thành các Dataset Snapshot bất biến, được kiểm chứng toàn vẹn bằng mã băm SHA-256, phân tách thời gian nghiêm ngặt giữa tập Train / Val / Test nhằm triệt tiêu hiện tượng rò rỉ dữ liệu.

1.4.3 Yêu cầu giao diện, phần cứng, phần mềm

- **Yêu cầu giao diện (UI/UX):** Thiết kế giao diện hiện đại, đáp ứng (Responsive) trên Desktop, Tablet và Mobile. Giao diện POS Terminal tối ưu cho màn hình cảm ứng, hỗ trợ phím tắt và bàn phím số lớn. Giao diện AI Chatbot tích hợp dạng bong bóng nổi (Floating Widget) với thẻ sản phẩm và nút xác nhận hành động trực quan. Tuân thủ tiêu chuẩn Core Web Vitals của Google (LCP < 2.5s).
- **Yêu cầu phần mềm phát triển:** Node.js v20 LTS, Python 3.11+, PostgreSQL 16 (`pgvector`), Redis 7 (Redis Cloud), RabbitMQ 3.13 (CloudAMQP), Nginx (Alpine), Docker Compose v2, Jest, Supertest, Pytest, Ruff và Mypy.
- **Yêu cầu phần cứng máy chủ:** Máy chủ phát triển RAM tối thiểu 16GB, CPU 6 nhân; Máy chủ vận hành Staging VPS tối thiểu 4 vCPU, 8GB - 16GB RAM kết hợp với các dịch vụ cơ sở dữ liệu và message broker được quản lý trên Cloud.

1.4.4 Yêu cầu phi chức năng (Quality Attributes)

- **Hiệu năng (Performance):** Thời gian phản hồi trung bình của API CRUD thông thường qua Gateway: P95 < 200ms. Thời gian xử lý toàn bộ pipeline AI Chatbot (Query Reformulation -> Embedding -> HNSW/GIN Search -> RRF Fusion -> LLM): P95 < 500ms. Tốc độ tra cứu luật kết hợp Apriori đạt độ phức tạp $O(1)$ (< 5ms) nhờ chỉ mục bán phần (Partial Index). Tỷ lệ trúng bộ nhớ đệm (Cache Hit Rate) tại Statistics Service đạt trên 85%.
- **Bảo mật (Security):** API Gateway áp dụng Rate Limiting 3 tầng (vùng strict: 10 req/phút; vùng standard: 60 req/phút; giới hạn kết nối `ws_conn`). Xác thực toàn bộ hệ thống bằng JWT Token. Mã hóa mật khẩu và mã PIN thu ngân bằng thuật toán Bcrypt. Kiểm tra chữ ký số HMAC-SHA512 đối với toàn bộ Webhook IPN từ cổng VNPay. Áp dụng thư viện Helmet chống các cuộc tấn công XSS, CSRF và Clickjacking.
- **Khả năng mở rộng (Scalability):** Kiến trúc Microservices cho phép mở rộng độc lập theo chiều ngang (Horizontal Scaling) các dịch vụ có tải cao (Catalog, Chatbot) mà không cần nhân bản toàn bộ hệ thống.

- **Độ tin cậy và Chịu lỗi (Reliability & Fault Tolerance):** Cô lập lỗi hoàn toàn giữa các dịch vụ. Một dịch vụ gặp sự cố không làm ảnh hưởng đến hoạt động bán hàng tại quầy POS. Mẫu thiết kế Transactional Outbox và Idempotency Guard đảm bảo tin nhắn không bị thất lạc và không xử lý lặp lại.
- **Khả năng bảo trì (Maintainability):** Toàn bộ mã nguồn backend tuân thủ cấu trúc 4 tầng phân tách trách nhiệm (Route -> Service -> Repository -> Database), tuân thủ tiêu chuẩn Clean Code và đạt tỷ lệ kiểm thử tự động hơn 50+ testcases bao phủ các luồng nghiệp vụ cốt lõi.

Chương 2

Cơ sở lý thuyết

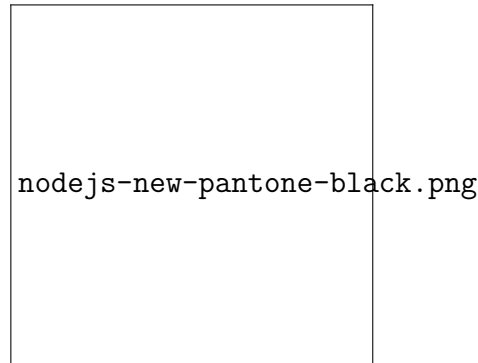
2.1 Những ứng dụng và công trình nghiên cứu liên quan

Trong quá trình phát triển các hệ thống thương mại điện tử và chuỗi bán lẻ, nhiều công trình nghiên cứu và kiến trúc phần mềm kinh điển đã đặt nền móng cho việc giải quyết các bài toán về quy mô, độ tin cậy và khả năng cá nhân hóa:

- **Mô hình Two-Tower trong hệ thống gợi ý quy mô lớn:** Nghiên cứu của Covington và cộng sự (2016) tại Google/YouTube [1] và Yi cùng cộng sự (2019) [2] đã chứng minh tính ưu việt của kiến trúc tháp đôi phi tuyến (Two-Tower Architecture) trong việc tách biệt hai giai đoạn: sinh ứng viên (Candidate Generation) và xếp hạng chi tiết (Ranking). Kiến trúc này cho phép tính toán trước các vector nhúng sản phẩm và thực hiện truy vấn tìm kiếm gần đúng (Approximate Nearest Neighbors) với độ trễ cực thấp.
- **Mô hình kết hợp Wide & Deep:** Công trình của Cheng và cộng sự (2016) tại Google [3] đã đề xuất sự kết hợp hoàn hảo giữa nhánh tuyến tính “Wide” (ghi nhớ các quy tắc kết hợp cụ thể) và nhánh học sâu “Deep” (khái quát hóa các mối tương quan ẩn). Kế thừa tư tưởng này, đề tài POSMART phát triển mô hình lai kết hợp luật kết hợp Apriori (Wide) và mạng nơ-ron tháp đôi (Deep) tối ưu hóa qua hàm mất mát Bayesian Personalized Ranking (BPR) [4].
- **Tính có thể tái lập và Quy trình Benchmark trong hệ gợi ý:** Các nghiên cứu gần đây của Gusak và cộng sự (2025) [5] và Petrov & Macdonald (2022) [6] đã chỉ ra rằng các kết quả đánh giá hệ gợi ý offline rất dễ bị sai lệch nếu không cố định quy trình phân tách thời gian (Temporal Split) và đánh giá trên tập mẫu ngẫu nhiên. Để khắc phục triệt để, đề tài POSMART thiết lập quy trình benchmark độc lập đánh giá trên toàn bộ danh mục sản phẩm ($C_u = I \setminus H_u^{\text{seen}}$) mà không dùng tập mẫu rút gọn.
- **Kiến trúc dữ liệu phân tán và Saga Pattern:** Công trình của Martin Kleppmann (2022) [15] đã phân tích sâu sắc các thách thức về tính toàn vẹn trong hệ thống phân tán. Đề tài áp dụng các mẫu thiết kế cốt lõi được khuyến nghị gồm Saga Choreography, Transactional Outbox và Idempotent Consumer nhằm giải quyết bài toán trừ tồn kho và thanh toán an toàn.

2.2 Công nghệ áp dụng

2.2.1 Node.js



Hình 2.1: Logo Node.js

Giới thiệu

Node.js là một môi trường thực thi mã JavaScript đa nền tảng, mã nguồn mở, được xây dựng trên trình biên dịch V8 của Google Chrome. Trong dự án POSMART, Node.js (phiên bản 20 LTS) đóng vai trò là Runtime nền tảng để xây dựng toàn bộ 8 Microservices, giúp xử lý các yêu cầu I/O (Input/Output) không đồng bộ một cách hiệu quả.

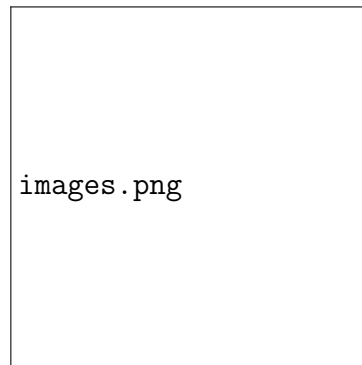
Ưu điểm

- **Hiệu suất cao:** Cơ chế Non-blocking I/O và Event-driven giúp xử lý hàng ngàn kết nối đồng thời với mức tiêu thụ tài nguyên thấp.
- **Hệ sinh thái phong phú:** NPM (Node Package Manager) cung cấp thư viện khổng lồ, hỗ trợ tích hợp nhanh chóng các công cụ như PostgreSQL, RabbitMQ, JWT.
- **Đồng nhất ngôn ngữ:** Sử dụng JavaScript cho cả Frontend lẫn Backend giúp tối ưu nguồn lực phát triển.

Nhược điểm

- **Xử lý tính toán nặng:** Do bản chất đơn luồng (single-threaded), Node.js có thể bị nghẽn nếu phải xử lý các tác vụ tính toán quá phức tạp (CPU-intensive).

2.2.2 Express.js



Hình 2.2: Logo Express.js

Giới thiệu

Express.js là một Web Framework tối giản và linh hoạt dành cho Node.js. Nó cung cấp tập hợp các tính năng mạnh mẽ để phát triển các ứng dụng web và di động, đặc biệt là trong việc xây dựng các RESTful API. Hệ thống sử dụng Express.js (v4.21) làm bộ khung chính để định tuyến (routing) và quản lý middleware cho mọi service.

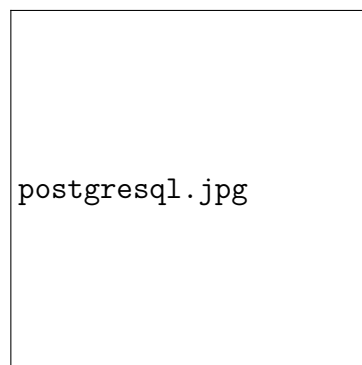
Ưu điểm

- **Nhẹ và linh hoạt:** Không gò bó kiến trúc, cho phép lập trình viên tự do thiết kế cấu trúc thư mục (Layered Architecture) theo ý muốn.
- **Hỗ trợ Middleware mạnh mẽ:** Dễ dàng tích hợp các lớp bảo mật như Helmet, CORS, và Express-rate-limit để chống tấn công Brute-force.

Nhược điểm

- **Đòi hỏi kỷ luật thiết kế:** Sự tự do của Express đồng nghĩa với việc nếu tổ chức code không tốt, dự án sẽ rất khó bảo trì khi mở rộng.

2.2.3 PostgreSQL



Hình 2.3: Logo PostgreSQL

Giới thiệu

PostgreSQL (phiên bản 16) là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở tiên tiến nhất hiện nay. Trong đồ án, PostgreSQL đóng vai trò lưu trữ toàn bộ dữ liệu nghiệp vụ theo mô hình 1 instance - 8 logical databases, đảm bảo việc thiết lập Multi-tenancy (qua cột store_id) và thực thi các Transaction Zones nghiêm ngặt.

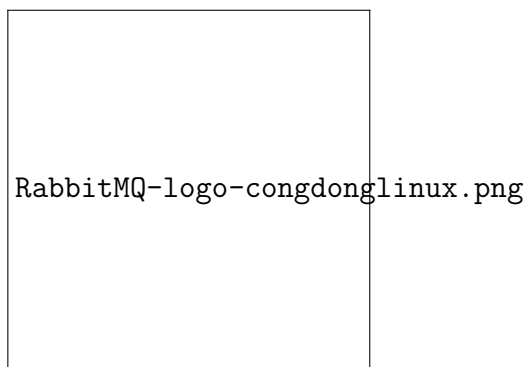
Ưu điểm

- **Mạnh mẽ và tuân thủ ACID:** Đảm bảo tính toàn vẹn dữ liệu tuyệt đối trong các tác vụ nhạy cảm như thanh toán đơn hàng và trừ tồn kho.
- **Hỗ trợ JSONB:** Cho phép lưu trữ dữ liệu dạng tài liệu linh hoạt (như lưu lịch sử cấu hình) mà vẫn giữ được khả năng truy vấn mạnh mẽ.

Nhược điểm

- **Quản lý kết nối phức tạp:** Yêu cầu cấu hình Connection Pooling chặt chẽ để không làm cạn kiệt tài nguyên kết nối khi chia nhỏ thành nhiều Microservices.

2.2.4 RabbitMQ



Hình 2.4: Logo RabbitMQ

Giới thiệu

RabbitMQ là hệ thống trung gian truyền thông (Message Broker) phổ biến sử dụng giao thức AMQP. Trong hệ thống POSMART, nó chịu trách nhiệm điều phối luồng giao tiếp bất đồng bộ thông qua cơ chế Publish/Subscribe (Topic Exchange), điển hình như phát sự kiện thanh toán hoàn tất để cập nhật đơn hàng và trừ tồn kho.

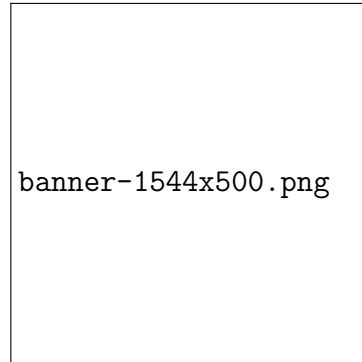
Ưu điểm

- **Đảm bảo giao nhận tin nhắn:** Hỗ trợ cơ chế xác nhận (ACK/NACK) giúp tin nhắn không bị thất lạc ngay cả khi một service đang tạm ngưng hoạt động.
- **Giảm độ kết dính (Decoupling):** Các service không cần gọi trực tiếp lẫn nhau qua HTTP, làm tăng khả năng chịu lỗi của toàn hệ thống.

Nhược điểm

- **Phức tạp hóa hạ tầng:** Cần giám sát độc lập và xử lý các trường hợp tin nhắn bị lỗi (Dead Letter Queue) để tránh vòng lặp vô tận.

2.2.5 Redis



Hình 2.5: Logo Redis

Giới thiệu

Redis là hệ thống lưu trữ cấu trúc dữ liệu trên bộ nhớ (In-memory data structure store), được tối ưu hóa cho tốc độ đọc/ghi cực nhanh. Hệ thống ứng dụng Redis (thông qua Redis Cloud) chủ yếu để làm bộ đệm (Caching) và lưu trữ trạng thái đếm cho tính năng giới hạn lưu lượng (Rate Limiting) bảo vệ các API quan trọng.

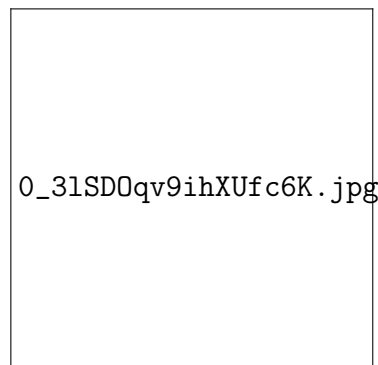
Ưu điểm

- **Hiệu suất vượt trội:** Phản hồi dữ liệu trong thời gian mili-giây do dữ liệu hoàn toàn nằm trên RAM.
- **Đa cấu trúc dữ liệu:** Hỗ trợ đa dạng cấu trúc như Strings, Hashes, Lists, Sets thuận tiện cho nhiều kịch bản sử dụng.

Nhược điểm

- **Giới hạn bộ nhớ:** Kích thước dữ liệu lưu trữ bị giới hạn bởi dung lượng RAM của máy chủ, cần có chiến lược giải phóng bộ nhớ hợp lý.

2.2.6 Hugging Face Inference API



Hình 2.6: Logo Hugging Face

Giới thiệu

Hugging Face là nền tảng hàng đầu cung cấp các mô hình học máy mã nguồn mở. Dự án sử dụng Hugging Face Inference API để gọi các mô hình Ngôn ngữ lớn (LLMs) như Phi-3-mini hoặc Mistral-7B, nhằm vận hành AI Chatbot Service tích hợp cơ chế Function Calling mà không cần tài nguyên GPU cục bộ.

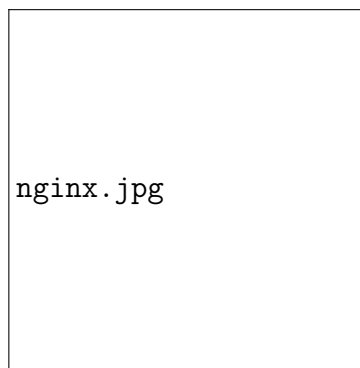
Ưu điểm

- **Tiết kiệm chi phí phần cứng:** Không cần thiết lập hạ tầng GPU đắt đỏ cục bộ, dễ dàng tích hợp qua REST API với thư viện chính thức.
- **Triển khai linh hoạt:** Nhanh chóng thử nghiệm và chuyển đổi giữa các mô hình AI tiên tiến nhất hiện nay mà không cần cấu hình lại hệ thống lõi.

Nhược điểm

- **Phụ thuộc đường truyền và Cloud:** Độ trễ của Chatbot phụ thuộc hoàn toàn vào tốc độ kết nối internet và thời gian phản hồi từ máy chủ Hugging Face.

2.2.7 Nginx



Hình 2.7: Logo Nginx

Giới thiệu

Nginx là một máy chủ web đa năng, nổi bật với khả năng xử lý truy cập đồng thời cao. Trong kiến trúc phân tán này, Nginx được cấu hình làm API Gateway (Reverse Proxy) duy nhất ở cổng 8080, đóng vai trò định tuyến các truy vấn từ client vào đúng các port cục bộ của 8 Microservices nội bộ.

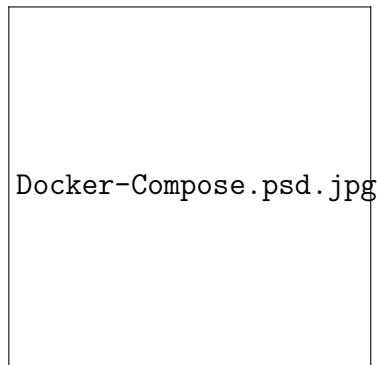
Ưu điểm

- **Hiệu suất và ổn định:** Khả năng chịu tải xuất sắc, tiêu thụ cực ít tài nguyên RAM ngay cả với hàng chục ngàn kết nối.
- **Kiểm soát luồng truy cập:** Che giấu được cấu trúc nội bộ của hệ thống Microservices, dễ dàng gắn thêm cấu hình Load Balancing hoặc bảo mật SSL.

Nhược điểm

- **Quản lý cấu hình:** Cú pháp cấu hình file conf đòi hỏi kiến thức chuyên môn, dễ xảy ra sai sót định tuyến nếu hệ thống mở rộng quá nhanh.

2.2.8 Docker & Docker Compose



Hình 2.8: Logo Docker

Giới thiệu

Docker là nền tảng ảo hóa cấp hệ điều hành sử dụng container, cho phép đóng gói ứng dụng cùng toàn bộ môi trường và thư viện phụ thuộc. Docker Compose được sử dụng để định nghĩa và chạy nhiều container cùng lúc, bao gồm 9 Node.js services, 1 Nginx, 1 PostgreSQL, 1 RabbitMQ và 1 Redis để giả lập môi trường Production ngay trên máy lập trình viên.

Ưu điểm

- **Đồng nhất môi trường:** Triệt tiêu hoàn toàn rủi ro xung đột môi trường giữa máy phát triển và máy chủ vận hành, đảm bảo tính nhất quán cao.
- **Đóng gói dễ dàng:** Quá trình dựng (build) lại một Microservice để kiểm thử diễn ra tự động và hoàn toàn độc lập với các services khác.

Nhược điểm

- **Tiêu tốn tài nguyên cục bộ:** Chạy đồng thời toàn bộ hệ thống Microservices qua Docker Compose yêu cầu máy tính lập trình viên phải có lượng RAM và CPU tương đối lớn.

2.2.9 Các thuật toán và kiến trúc gợi ý sản phẩm

Trong kiến trúc của hệ thống POSMART (cụ thể hỗ trợ cho intent RECOMMENDATION của Chatbot), việc lựa chọn thuật toán gợi ý sản phẩm đóng vai trò quyết định đến trải nghiệm người dùng và chi phí vận hành. Thay vì sử dụng các mô hình Học sâu nguyên thuần (Pure Deep Learning) vốn đang là xu hướng, POSMART được thiết kế dựa trên kiến trúc Gợi ý Lai (Hybrid Ensemble). Quyết định này xuất phát từ đặc thù của ngành bán lẻ: hệ thống cần sự minh bạch trong tư vấn, tốc độ phản hồi thời gian thực và khả năng chạy trên hạ tầng có giới hạn.

Bài toán đánh đổi: Độ phức tạp tính toán vs. Độ chính xác thương mại

Trong lĩnh vực hệ thống gợi ý (Recommendation Systems), luôn tồn tại một sự đánh đổi (trade-off) gay gắt giữa độ phức tạp của thuật toán và giá trị thương mại mang lại:

- **Độ phức tạp tính toán (Computational Complexity):** Các mô hình Deep Learning (như Neural Collaborative Filtering hay Transformer-based recommenders) có khả năng trích xuất các đặc trưng ẩn (latent features) cực kỳ tốt, mang lại độ chính xác toán học cao. Tuy

nhien, chúng đòi hỏi tài nguyên huấn luyện khổng lồ (GPU), thời gian inference lâu và liên tục cần dữ liệu lớn để hội tụ. Điều này tạo ra "độ trễ" (latency) không thể chấp nhận được trong một phiên chat thời gian thực của POSMART.

- **Độ chính xác thương mại (Commercial Accuracy):** Trong môi trường POS bán lẻ, "gợi ý đúng" không chỉ là đoán trúng sản phẩm khách muốn mua (dựa trên toán học), mà còn là gợi ý những sản phẩm mang lại lợi nhuận cao, đang có sẵn tồn kho (Stock check), hoặc đang chạy chương trình khuyến mãi.

Kiến trúc Hybrid Ensemble giải quyết bài toán này bằng cách kết hợp nhiều thuật toán nhẹ (Lightweight algorithms) – chẳng hạn như Lọc cộng tác (Collaborative Filtering) để tìm khách hàng tương đồng, Lọc theo nội dung (Content-Based) dựa trên mô tả sản phẩm, và Tập luật kinh doanh (Rule-based: ưu tiên hàng tồn kho cao). Phương pháp này hy sinh một phần nhỏ độ chính xác toán học ở các "đuôi dài" (long-tail) của Deep Learning để đổi lấy tốc độ truy xuất cực nhanh và khả năng kiểm soát hoàn toàn kết quả đầu ra, đáp ứng đúng "độ chính xác thương mại" mà một hệ thống bán lẻ cần.

Bảng so sánh Trade-off: Hybrid Ensemble vs. Deep Learning

Để làm rõ lợi thế của việc áp dụng Hybrid Ensemble cho hệ thống POSMART, dưới đây là bảng phân tích so sánh chi tiết trên bốn khía cạnh cốt lõi: Vấn đề Cold-start, Tính minh bạch, Tối ưu tài nguyên, và Kiểm soát linh hoạt.

Tiêu chí	Kiến trúc Gợi ý Lai (Hybrid Ensemble) - Lựa chọn cho POSMART	Học sâu nguyên thuần (Pure Deep Learning)
Giải quyết Cold-start	Xuất sắc. Khi có sản phẩm mới (chưa có lịch sử mua) hoặc khách hàng mới, hệ thống tự động chuyển trọng số sang thuật toán Content-Based hoặc Rule-Based.	Kém. Phụ thuộc nặng nề vào khối lượng lớn dữ liệu tương tác. Rất khó gợi ý chính xác khi dữ liệu đầu vào trống.
Tính minh bạch	Cao (White-box). Dễ dàng giải thích lý do gợi ý. VD: “Gợi ý Bò Kobe vì bạn vừa thêm Rượu Vang và sản phẩm đang giảm 10%”.	Thấp (Black-box). Các lớp ẩn (Hidden layers) khiến việc giải thích kết quả trở nên bất khả thi.
Tối ưu tài nguyên	Tối ưu. Tính toán nhẹ, chạy mượt mà trên CPU tiêu chuẩn với độ trễ thấp dưới 100ms.	Nặng nề. Yêu cầu GPU/TPU để huấn luyện và Inference. Tồn kém chi phí duy trì hạ tầng.
Kiểm soát linh hoạt	Dễ dàng can thiệp bằng tay (Hard-code rules) để đẩy tồn kho cận date hoặc sản phẩm chiến lược.	Rất khó can thiệp trực tiếp mà không làm hỏng trọng số của toàn bộ mô hình.

Bảng 2.1: So sánh giữa kiến trúc Hybrid Ensemble và Pure Deep Learning

Kết luận: Đối với một hệ thống tích hợp Trợ lý thao tác (Action Assistant) như POSMART, nơi Chatbot không chỉ cần gợi ý mà còn phải trực tiếp giải thích và thêm sản phẩm vào giỏ, kiến trúc Hybrid Ensemble mang lại tính thực chiến cao nhất. Nó cân bằng hoàn hảo giữa hiệu năng hệ thống, chi phí vận hành và khả năng tối ưu hóa doanh thu thương mại.

Thuật toán 1: Content-Based Filtering (RAG + RRF)

Trụ cột đầu tiên của hệ thống gợi ý là kỹ thuật **Retrieval-Augmented Generation (RAG)** kết hợp tìm kiếm ngữ nghĩa (Semantic Search) và tìm kiếm từ khóa (Keyword Search), hợp nhất kết quả thông qua **Reciprocal Rank Fusion (RRF)**.

Yêu cầu về tính đồng nhất không gian Vector: Để Semantic Search hoạt động chính xác, hệ thống bắt buộc sử dụng cùng một mô hình Embedding (Vietnamese SBERT, 768 chiều) cho cả hai giai đoạn nạp dữ liệu (Indexing) và truy vấn (Query). Nếu dùng hai mô hình khác nhau, hệ tọa độ sẽ bị lệch, khiến phép đo khoảng cách Cosine Similarity trở nên vô nghĩa.

Công thức toán học:

Semantic Search – Cosine Similarity giữa vector câu hỏi $\vec{q}$ và vector sản phẩm $\vec{d}$ (cùng 768 chiều):

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \cdot \|\vec{d}\|}$$

Keyword Search – PostgreSQL Full-Text Search với hàm xếp hạng `ts_rank` trên chỉ mục GIN:

$$\text{Score}_{\text{Keyword}}(q, d) = \text{FTS}_{\text{Rank}}(d.\text{fts}, q)$$

Reciprocal Rank Fusion – Hợp nhất 2 danh sách kết quả, với $\text{rank}_i(d)$ là thứ hạng của sản phẩm d trong danh sách thứ i , hằng số $k = 60$:

$$\text{RRF}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

Tại sao chọn RRF? Điểm số từ Semantic Search (cosine similarity, thang $[0, 1]$) và Keyword Search (`ts_rank`, thang không cố định) không cùng đơn vị đo. RRF giải quyết vấn đề này bằng cách chỉ sử dụng *thứ hạng* thay vì điểm số thô, loại bỏ hoàn toàn nhu cầu normalize. Giá trị $k = 60$ là hằng số chuẩn được đề xuất trong bài báo gốc về RRF (Cormack et al., 2009), giúp giảm sự chênh lệch điểm số giữa rank 1 và rank 10, tránh tình trạng kết quả rank cao ở một danh sách hoàn toàn áp đảo.

Tại sao chọn Top 5 thay vì Top 10? Phân tích RRF Score Decay cho thấy từ rank 6 trở đi, điểm số giảm đột ngột trên 50% do hầu hết sản phẩm chỉ xuất hiện ở một luồng tìm kiếm. Ngoài ra, giới hạn Top 5 giúp rút ngắn độ dài prompt gửi đến LLM, giảm thiểu hiện tượng hallucination và phù hợp với hành vi người dùng trên giao diện chat (khách hàng thường bỏ qua sau 3-5 sản phẩm).

Độ phức tạp thuật toán: HNSW Semantic Search đạt $O(\log n)$, GIN Full-Text Search đạt $O(k)$ với k là số token khớp, và RRF Fusion đạt $O(m \log m)$ với m là tổng kết quả hợp nhất. Toàn bộ pipeline Content-Based có độ phức tạp chi phối $O(\log n)$, vượt trội so với Brute-force vector scan $O(n)$.

Thuật toán 2: Apriori (Association Rules)

Trụ cột thứ hai khai thác lịch sử đơn hàng để phát hiện các sản phẩm thường được mua cùng nhau, phục vụ mục đích bán chéo (Cross-selling). Hệ thống sử dụng ba chỉ số đánh giá: Support, Confidence và Lift.

Công thức toán học: Cho hai sản phẩm A và B trong tập giao dịch T :

$$\text{support}(A, B) = \frac{|A \cap B|}{|T|}, \quad \text{confidence}(A \Rightarrow B) = \frac{|A \cap B|}{|A|}, \quad \text{lift}(A, B) = \frac{|A \cap B| \times |T|}{|A| \times |B|}$$

Tại sao Lift là tiêu chí quyết định? Support cao không đảm bảo tương quan thật sự (nước suối có support cao với mọi sản phẩm vì ai cũng mua, nhưng $\text{lift} \approx 1$). Confidence cao có thể gây hiểu lầm (100% người mua xúc xích cũng mua bánh mì, nhưng nếu ai cũng mua bánh mì thì luật này vô nghĩa). Lift xác nhận đây là tương quan thật hay ngẫu nhiên: $\text{lift} > 1$ nghĩa là A và B thật sự liên quan, $\text{lift} = 1$ nghĩa là độc lập thống kê.

Tối ưu hóa tốc độ tra cứu: Kết quả Apriori được tính toán định kỳ (Nightly Batch lúc 2AM) và lưu vào bảng frequent_itemsets với Partial Index theo product_a_id. Điều này cho phép tra cứu gợi ý “mua kèm” đạt tốc độ $O(1)$ tại runtime, giảm tải hoàn toàn cho CPU khi khách hàng đang ở màn hình thanh toán.

Thuật toán 3: Item-based Collaborative Filtering

Trụ cột thứ ba tính **Cosine Similarity** giữa các sản phẩm dựa trên vector hành vi mua của toàn bộ người dùng, theo nguyên lý: “*Những sản phẩm được mua bởi cùng nhóm khách hàng sẽ có hành vi tương tự.*”

Công thức toán học: Cosine Similarity giữa item i và j qua ma trận tương tác $R_{u,i}$:

$$\text{sim}(i, j) = \frac{\sum_u R_{u,i} \cdot R_{u,j}}{\|\vec{R}_{\cdot,i}\| \cdot \|\vec{R}_{\cdot,j}\|}$$

Trong đó $R_{u,i}$ là điểm tương tác của người dùng u với sản phẩm i , được tổng hợp từ cả dữ liệu mua hàng lẫn implicit feedback (lượt click, thêm giỏ hàng, thời gian dừng xem).

Điểm dự đoán (Prediction Score) cho người dùng u với sản phẩm ứng viên i , chỉ xét các sản phẩm j mà người dùng đã mua và có $\text{sim}(i, j) \geq 0.1$:

$$\hat{r}_{u,i} = \frac{\sum_{j \in S_u} \text{sim}(i, j) \cdot R_{u,j}}{\sum_{j \in S_u} |\text{sim}(i, j)|}$$

Tại sao Plain Cosine thay vì Adjusted Cosine? Adjusted Cosine trừ đi giá trị trung bình $\bar{R}_u$ của người dùng để loại bỏ “thang đo cá nhân” – phù hợp khi dữ liệu là Explicit Feedback (rating 1-5 sao). Tuy nhiên, dữ liệu siêu thị là Implicit Feedback (số lần mua, số lượng). Khi khách hàng cùng cluster mua đều đặn các sản phẩm thiết yếu thì $R_{u,i} - \bar{R}_u \approx 0$, khiến similarity bằng 0 – kết quả sai hoàn toàn. Plain Cosine giữ nguyên magnitude của dữ liệu gốc, cho kết quả chính xác hơn với dữ liệu hành vi mua hàng.

Độ phức tạp Batch Job: Xây dựng ma trận tương tác $O(|U| \cdot |I|)$; tính toán độ tương đồng tất cả các cặp $O(m^2 \cdot \bar{c})$ với $\bar{c}$ là số người dùng chung trung bình; lọc Top-K tương đồng $O(m \cdot K \log K)$. Với 200 sản phẩm, hệ thống tạo ra 19.900 cặp, sau Pruning (loại bỏ cặp có ít hơn 2 người dùng chung) còn khoảng 4.000 cặp, hoàn thành trong dưới 5 giây.

Thuật toán 4: Hybrid Ensemble và Adaptive Weight Learning

Công thức tổng hợp điểm:

Điểm cuối cùng của mỗi sản phẩm được tổng hợp từ bốn thành phần, tất cả đã được chuẩn hóa về $[0, 1]$:

$$\text{Score}(p) = \alpha \cdot S_{\text{Content}}(p) + \beta \cdot S_{\text{CF}}(p) + \gamma \cdot S_{\text{Apriori}}(p) + \delta \cdot S_{\text{Persona}}(u)$$

với ràng buộc $\alpha + \beta + \gamma + \delta = 1$. Bộ trọng số mặc định là $\alpha = 0.40$, $\beta = 0.25$, $\gamma = 0.25$, $\delta = 0.10$. Content-Based (α) chiếm tỷ trọng lớn nhất vì hoạt động ngay từ ngày đầu mà không cần dữ liệu lịch sử (giải quyết Cold-start), trong khi CF (β) và Apriori (γ) tăng hiệu quả dần theo thời gian sử dụng.

Xử lý Cold-start: Khi CF không có dữ liệu (người dùng mới chưa có lịch sử mua), trọng số β được tái phân bổ sang Content-Based: $\alpha' = \alpha + \beta$, $\beta' = 0$, đảm bảo hệ thống vẫn gợi ý chính xác.

Cơ chế Adaptive Weight Learning: Hệ thống không cố định bộ trọng số mà tự động học qua vòng lặp phản hồi (Feedback Loop) chạy định kỳ hàng đêm (Nightly Batch). Điểm chuyển đổi của mỗi nguồn gợi ý s được tính dựa trên hành vi thực tế của người dùng (mua hàng, thêm giỏ, click, hover), sau đó cập nhật trọng số bằng công thức Exponential Smoothing với hệ số làm mượt $\lambda = 0.8$:

$$w_{t+1}(s) = \lambda \cdot w_t(s) + (1 - \lambda) \cdot w_{\text{raw}}(s)$$

Công thức này đảm bảo hệ thống *học từ dữ liệu mới nhưng không quên lịch sử*: 80% giữ lại trọng số cũ đã được kiểm chứng, 20% cập nhật từ tín hiệu thực tế gần đây. Để tránh mất cân bằng, trọng số được giới hạn trong khoảng $[0.05, 0.60]$ và cần tối thiểu 20 feedbacks mới thực hiện cập nhật.

Trong thực nghiệm, sau một chu kỳ học, trọng số CF (β) tăng từ 0.25 lên 0.27 do đạt tỷ lệ chuyển đổi cao nhất trong nhóm – minh chứng rõ ràng cho khả năng tự tối ưu hóa của hệ thống.

Thuật toán 5: Mạng nơ-ron Deep Two-Tower và Tối ưu hóa BPR Loss ('ai-service-v2')

Nhằm nâng tầm hệ thống gợi ý từ các heuristic cơ bản lên mô hình học máy hiện đại phục vụ nghiên cứu khoa học chuyên sâu, đề tài xây dựng phân hệ ai-service-v2 với mô hình mạng nơ-ron tháp đôi (Two-Tower Architecture).

Bài toán xếp hạng danh mục đầy đủ (Full-catalog Ranking Estimand): Đối với mỗi người dùng u , tập hợp các sản phẩm ứng viên C_u là toàn bộ danh mục sản phẩm của hệ thống sau khi đã loại bỏ các sản phẩm người dùng đã mua trong lịch sử huấn luyện:

$$C_u = I \setminus H_u^{\text{seen}} \quad (2.1)$$

Quy trình đánh giá không sử dụng tập mẫu rút gọn (Sampled candidate sets) để tránh tạo ra sai lệch đo lường, đảm bảo các chỉ số NDCG@10 và HR@10 phản ánh chính xác năng lực thực tế của mô hình.

Cấu trúc tháp đôi phi tuyến (Decoupled User and Item Towers):

- **Tháp Người dùng (User Tower):** Tiếp nhận định danh người dùng u và lịch sử các sản phẩm đã tương tác H_u , tạo ra vector biểu diễn người dùng $\vec{e}_u \in \mathbb{R}^d$:

$$\vec{h}_u = \tanh\left((\vec{u} + \vec{h}_{\text{hist}})\mathbf{W}_1^{(u)} + \vec{b}_1^{(u)}\right), \quad \vec{e}_u = \vec{h}_u\mathbf{W}_2^{(u)} + \vec{b}_2^{(u)} \quad (2.2)$$

- **Tháp Sản phẩm (Item Tower):** Tích hợp vector nhúng của sản phẩm cùng tầng chiều đặc trưng ngữ nghĩa và giá bán (Content Projection):

$$\vec{h}_i = \tanh\left((\vec{v}_i + \vec{f}_i\mathbf{W}_{\text{proj}})\mathbf{W}_1^{(i)} + \vec{b}_1^{(i)}\right), \quad \vec{e}_i = \vec{h}_i\mathbf{W}_2^{(i)} + \vec{b}_2^{(i)} \quad (2.3)$$

Điểm số dự đoán tương thích giữa người dùng u và sản phẩm i được tính bằng tích vô hướng giữa hai vector biểu diễn:

$$S_{\text{deep}}(u, i) = \vec{e}_u \cdot \vec{e}_i \quad (2.4)$$

Hàm mất mát Bayesian Personalized Ranking (BPR Loss): Mô hình được tối ưu hóa thông qua hàm mất mát BPR cho dữ liệu phản hồi ngầm (Implicit Feedback):

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j) \in \mathcal{D}} \ln \sigma(S_{\text{deep}}(u,i) - S_{\text{deep}}(u,j)) + \frac{\lambda_{\Theta}}{2} \|\Theta\|^2 \quad (2.5)$$

trong đó i là sản phẩm người dùng đã mua (Positive Item), j là sản phẩm người dùng chưa quan sát (Unseen Negative Item), và $\sigma(z) = \frac{1}{1+e^{-z}}$ là hàm sigmoid.

Thuật toán 6: Mô hình Kết hợp Lai Cộng tính với Chuẩn hóa Z-Score per-user

Để dung hòa giữa năng lực khái quát hóa của nhánh Deep Two-Tower (S_{deep}) và khả năng bắt trúng các mối tương quan bán chéo cụ thể của nhánh Wide Apriori (S_{wide}), hệ thống áp dụng cơ chế chuẩn hóa Z-score độc lập cho từng người dùng:

$$\text{Norm}(S(u, \cdot)) = \frac{S(u, i) - \mu_u}{\sigma_u} \quad (2.6)$$

trong đó μ_u và σ_u là giá trị trung bình và độ lệch chuẩn của các điểm số trong danh mục ứng viên của người dùng u .

Điểm số lai tổng hợp cuối cùng (S_{hybrid}) được tính theo công thức:

$$S_{\text{hybrid}}(u, i) = \text{Norm}(S_{\text{deep}}(u, i)) + w_{\text{wide}} \cdot \text{Norm}(S_{\text{wide}}(u, i)) \quad (2.7)$$

với w_{wide} là trọng số điều phối sự đóng góp của luật kết hợp Apriori.

Thuật toán 7: Giao thức Trợ lý Tác vụ Hai chiều (Action Assistant v2.0)

Vượt lên trên các hệ thống Chatbot hỏi đáp thuần túy, POSMART thiết kế giao thức ****Action Response Protocol**** cho phép Trợ lý ảo thực thi các hành động điều khiển trạng thái Frontend một cách có kiểm soát:

- **Cấu trúc thông điệp tác vụ (Action Payload):** Mỗi phản hồi hành động chứa các trường dữ liệu có cấu trúc: `action_type` (ví dụ: `ADD_TO_CART`, `CHECK_STOCK`, `HOLD_ORDER`), `parameters` (ID sản phẩm, số lượng, `store_id`) và cờ `require_confirmation`.
- **Chốt chặn xác nhận an toàn (Confirmation Gate):** Đối với các tác vụ làm thay đổi trạng thái giỏ hàng hoặc dữ liệu tài chính, Chatbot bắt buộc kích hoạt Confirmation Gate hiển thị hộp thoại xác nhận trên giao diện người dùng, ngăn chặn tuyệt đối các hành vi tự ý thực thi sai lệch.

Chương 3

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu hệ thống

3.1.1 Yêu cầu chức năng

Dựa trên các phân tích nghiệp vụ, hệ thống cần đáp ứng các yêu cầu chức năng cơ bản được phân chia theo từng nhóm đối tượng người dùng như sau:

Đối với Khách hàng (Customer):

- Hệ thống hiển thị trang chủ với Hero section, các banner khuyến mãi và sản phẩm nổi bật.
- Hệ thống phải cho phép người dùng đăng ký tài khoản mới với lựa chọn customer hoặc vendor.
- Hệ thống phải cho phép người dùng đăng nhập vào tài khoản đã có.
- Hệ thống hiển thị danh mục sản phẩm với hình ảnh và tên danh mục.
- Hệ thống hiển thị sản phẩm với thông tin đầy đủ bao gồm hình ảnh, tên, giá, đánh giá và các nhãn (badges).
- Người dùng có thể tìm kiếm sản phẩm thông qua thanh tìm kiếm (search bar).
- Hệ thống hiển thị giỏ hàng với số lượng sản phẩm chi tiết.
- Giao diện phải được thiết kế responsive, tương thích hiển thị trên các thiết bị desktop, tablet và mobile.

Đối với Quản trị viên (Admin/Manager/Sales):

- Hệ thống cung cấp giao diện quản trị (admin) riêng biệt với bố cục LayoutAdmin.
- Cung cấp Dashboard tổng quan với thanh điều hướng (sidebar navigation) và hồ sơ người dùng (user profile).
- Trang Overview cần hiển thị biểu đồ doanh thu (SalesChart component).
- Header của trang quản trị cung cấp thanh tìm kiếm, thông báo (notifications) và biểu tượng giỏ hàng.
- Cung cấp chức năng "Add Product" để thêm sản phẩm mới vào hệ thống.

- Cung cấp menu quản lý với các module chức năng: Dashboard, Orders, Products, Customers, Inventory, Suppliers, v.v.
- Hỗ trợ các thao tác CRUD (Thêm, Đọc, Sửa, Xóa) cho dữ liệu sản phẩm, đơn hàng, người dùng, lô hàng và tồn kho.

3.1.2 Yêu cầu phi chức năng

Hệ thống cần đảm bảo các tiêu chuẩn về chất lượng phần mềm (Quality Attributes) bao gồm:

Hiệu suất (Performance):

- Thời gian hiển thị nội dung đầu tiên (First Contentful Paint) < 1.5s, thời gian hiển thị phần tử lớn nhất (Largest Contentful Paint) < 2.5s.
- Tối ưu hóa hình ảnh với cơ chế tải lười (lazy loading) và lưu trữ qua mạng phân phối nội dung (CDN).
- Tối ưu hóa mã nguồn với cơ chế phân tách mã (code splitting) và loại bỏ mã thừa (tree shaking) bằng công cụ build Vite.
- Sử dụng hình ảnh phản hồi (responsive images) phù hợp cho từng điểm dừng (breakpoint) của màn hình.

Bảo mật (Security):

- Thực hiện xác thực biểu mẫu (Form validation) cho các tính năng đăng ký, đăng nhập.
- Làm sạch dữ liệu đầu vào (Input sanitization) để phòng chống các cuộc tấn công XSS.
- Bắt buộc sử dụng kết nối HTTPS cho môi trường production.
- Mã hóa mật khẩu (Password hashing) sử dụng thư viện bcrypt.
- Quản lý phiên làm việc an toàn (Secure session management) thông qua JWT và bảo vệ chống tấn công CSRF.

Khả năng bảo trì (Maintainability):

- Kiến trúc hệ thống dựa trên các thành phần (Component-based architecture) tuân thủ các nguyên tắc phát triển phần mềm.
- Sử dụng các quy tắc ESLint để đảm bảo chất lượng mã nguồn.
- Thống nhất các quy ước đặt tên (naming conventions) và cấu trúc thư mục.
- Quản lý phiên bản mã nguồn bằng Git kèm theo các thông điệp commit rõ ràng.

Tính dễ sử dụng (Usability):

- Điều hướng trực quan với breadcrumb và cấu trúc menu rõ ràng.
- Cung cấp phản hồi trực quan (visual feedback) cho các tương tác của người dùng như hiệu ứng hover, trạng thái nút bấm.
- Xử lý lỗi trơn tru và hiển thị các thông báo lỗi thân thiện với người dùng.

- Chức năng tìm kiếm hỗ trợ văn bản gợi ý (placeholder) và đề xuất từ khóa (search suggestions).

Khả năng mở rộng (Scalability):

- Kiến trúc thành phần theo mô-đun (Modular component architecture) cho phép dễ dàng bổ sung các tính năng mới.
- Các hàm tiện ích (utility functions) và custom hooks có khả năng tái sử dụng cao.
- Kiến trúc sẵn sàng tích hợp API (API-ready) để giao tiếp với các dịch vụ backend phân tán (Microservices).
- Cấu hình môi trường riêng biệt cho các quá trình development, staging và production.

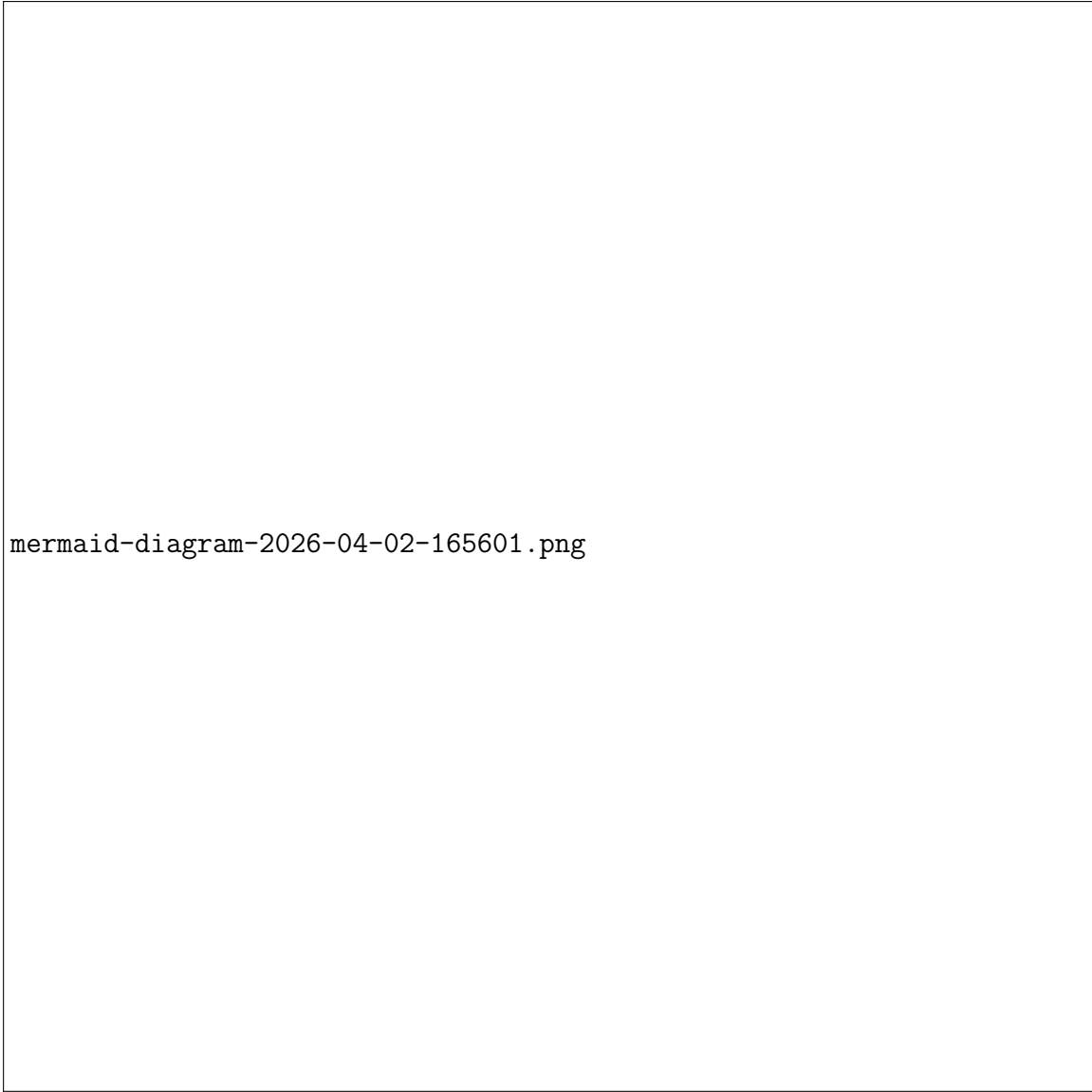
3.2 Kiến trúc hệ thống

3.2.1 Sơ đồ kiến trúc tổng thể

Hệ thống POSMART được thiết kế dựa trên mô hình phân tán tiên tiến, kết hợp giữa kiến trúc Microservices và chiến lược Multi-tenancy (Đa người thuê). Mục tiêu của kiến trúc này là đảm bảo khả năng mở rộng linh hoạt, chịu tải cao và cô lập dữ liệu an toàn cho từng chi nhánh cửa hàng.

Luồng dữ liệu tổng thể của hệ thống đi qua các lớp (layers) chính như sau:

- **Client Layer:** Bao gồm các ứng dụng Web (dành cho người quản trị, nhân viên tại quầy POS, và khách hàng) giao tiếp với hệ thống backend thông qua giao thức HTTP/HTTPS.
- **API Gateway Layer:** Sử dụng Nginx làm điểm truy cập duy nhất (Single Entry Point) ở cổng 8080. Nginx chịu trách nhiệm phân giải request, cân bằng tải và định tuyến (routing) yêu cầu từ Client tới đúng Microservice xử lý nghiệp vụ bên dưới.
- **Microservices Layer:** Lớp lõi chứa 9 dịch vụ độc lập biệt lập, mỗi dịch vụ xử lý một vùng nghiệp vụ (Domain) cụ thể (Auth, Catalog, Order, Inventory,...).
- **Infrastructure & Data Layer:** Các dịch vụ lưu trữ và trung gian bao gồm cơ sở dữ liệu PostgreSQL (chia thành 8 logical databases), hệ thống Message Broker (RabbitMQ) để giao tiếp bất đồng bộ, và Redis để xử lý bộ đệm (Caching).
- **External Services:** Tích hợp với các API bên thứ ba như cổng thanh toán VNPay và Hugging Face Inference API phục vụ cho AI Chatbot.



mermaid-diagram-2026-04-02-165601.png

Hình 3.1: Sơ đồ kiến trúc tổng thể hệ thống POSMART

3.2.2 Chi tiết Backend Architecture

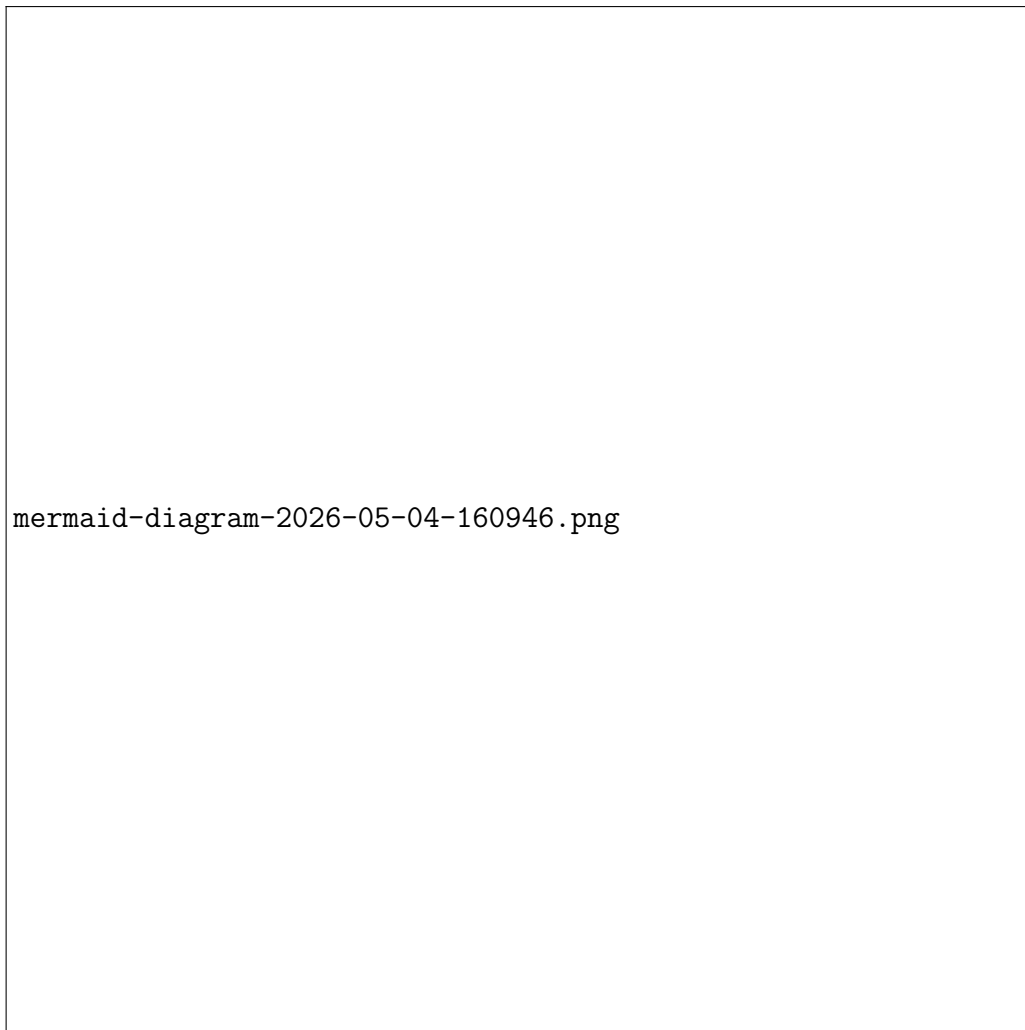
Backend của POSMART được chia thành **9 Microservices** độc lập chạy trên nền tảng Node.js (Express), liên kết chặt chẽ với nhau thông qua mạng lưới hướng sự kiện (Event-Driven Architecture) với RabbitMQ và API Gateway Nginx. Các thành phần chính bao gồm:

1. **Auth & Identity Service (Port 3001):** Quản lý định danh người dùng (Khách hàng, Nhân viên, Quản trị viên), phân quyền RBAC, xác thực nhanh tại quầy POS bằng mã PIN 4-6 số được mã hóa Bcrypt và cấp phát mã định danh `store_id` cho các truy vấn Multi-tenancy.
2. **Catalog Service (Port 3002):** Quản lý cây danh mục sản phẩm và dữ liệu hàng hóa tập trung (Centralized Catalog) dùng chung toàn chuỗi, tra cứu mã vạch EAN-13 và lưu vết lịch sử biến động giá (`product_price_history`).
3. **Order Service (Port 3003):** Xử lý máy trạng thái vòng đời đơn hàng bán ra (Draft → Paid → Shipping → Delivered → Cancelled), quản lý danh sách đơn hàng chờ tại quầy

(Held Orders), tham gia điều phối giao dịch Saga và hỗ trợ các trường định danh nghiên cứu học máy (benchmark_run_id).

4. **Settings Service (Port 3004):** Quản lý các thiết lập hệ thống dùng chung (Singleton Security Settings, Sales Settings, tự động kích hoạt khuyến mãi giảm giá ban đêm cho hàng cận date lúc 18h) và ghi log kiểm toán (settings_history).
5. **Supplier Service (Port 3005):** Quản lý danh bạ nhà cung cấp, kiểm soát công nợ chi nhánh (current_debt, credit_limit) và quy trình đặt hàng nhà cung cấp (Purchase Order).
6. **Inventory Service (Port 3006):** Quản lý tồn kho chi tiết theo lô (product_batch), quản lý hạn sử dụng, sơ đồ kho bãi theo Block và kệ hàng (Location), thực hiện trừ kho phân tán và điều chuyển hàng lên kệ (inventory_movement).
7. **Payment Service (Port 3007):** Đóng vai trò **Saga Orchestrator**, điều phối thanh toán tiền mặt, thẻ và cổng điện tử VNPay Sandbox. Xác thực chữ ký số bảo mật Webhook IPN (vnp_SecureHash) và phát sự kiện payment.completed qua RabbitMQ.
8. **AI Chatbot Service (Port 3008):** Cung cấp dịch vụ đàm thoại thời gian thực qua Socket.IO và REST. Tích hợp RAG Pipeline với pgvector HNSW (Vietnamese SBERT 768 chiều) kết hợp Full-Text Search GIN và RRF Fusion; hiện thực hóa giao thức Trợ lý tác vụ hai chiều Action Assistant v2.0.
9. **Statistics & Analytics Service (Port 3009):** Dịch vụ phi trạng thái (Stateless), đóng vai trò tổng hợp dữ liệu (Aggregator) từ các service Order, Inventory, Supplier và Catalog; tích hợp bộ nhớ đệm phân tán Redis Caching để phục vụ Dashboard và báo cáo doanh thu, lợi nhuận gộp với độ trễ thấp.

Cơ chế **Multi-tenancy** được áp dụng triệt để ở lớp Backend: Các dịch vụ như Order, Inventory, Supplier đều dùng chung một cơ sở dữ liệu vật lý (Shared DB, Shared Schema) nhưng bắt buộc phải có khóa ngoại store_id để thực thi chính sách cách ly dữ liệu ở mức dòng (Row-Level Security).



Hình 3.2: Chi tiết phân rã 8 Microservices trong Backend

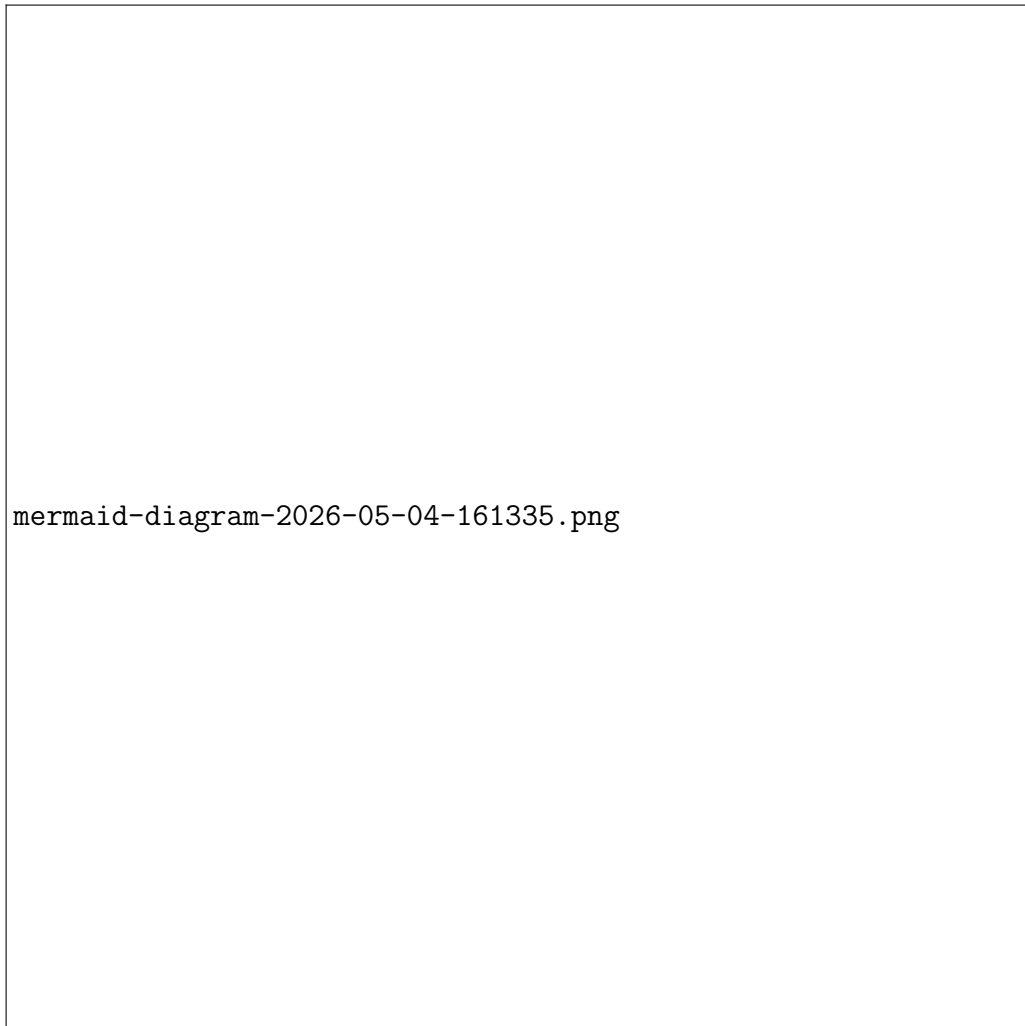
3.2.3 Chi tiết Frontend Architecture

Kiến trúc Frontend của POSMART được xây dựng theo hướng Component-Based (Dựa trên thành phần), nhằm mục tiêu tái sử dụng mã nguồn và tối ưu hóa hiệu suất hiển thị (Rendering). Mặc dù Backend được phân rã thành nhiều dịch vụ, Frontend được tổ chức thành một Single Page Application (SPA) thông qua các framework hiện đại (như React/Next.js).

Kiến trúc Frontend được chia thành các tầng logic rõ rệt:

- **Tầng UI Components (Giao diện người dùng):** Sử dụng các thư viện UI (như Tailwind CSS, Shaden UI) để xây dựng các thành phần giao diện nhất quán cho cả trang bán hàng (Storefront) và trang quản trị (POS/Admin Dashboard).
- **Tầng State Management (Quản lý trạng thái):** Quản lý dữ liệu toàn cục (Global State) như thông tin người dùng đang đăng nhập (JWT Token), giỏ hàng hiện tại, và trạng thái cấu hình cửa hàng.
- **Tầng API Integration (Tích hợp API):** Chứa các module giao tiếp HTTP (thông qua Axios hoặc Fetch API) được cấu hình tập trung. Tầng này sẽ đính kèm tự động chuỗi JWT Bearer Token vào các Header để thực thi bảo mật trước khi gửi Request lên API Gateway của Backend.

- **Tầng Routing (Định tuyến Client-side):** Quản lý việc chuyển trang mượt mà không cần tải lại trình duyệt, đồng thời bảo vệ các trang yêu cầu quyền truy cập (Private Routes) dựa trên Role và Permission của người dùng.



Hình 3.3: Sơ đồ kiến trúc cấu trúc các tầng logic trên Frontend

Kiến trúc Hệ thống Gợi ý Sản phẩm AI (Hybrid Ensemble)

Hệ thống gợi ý của POSMART không dựa trên một thuật toán đơn lẻ mà là một hệ thống phối hợp (Ensemble) gồm bốn trụ cột chiến lược, mục tiêu tối ưu hóa điểm số tổng hợp để đưa ra những sản phẩm có xác suất chuyển đổi cao nhất dựa trên ngữ nghĩa, hành vi người dùng và quy luật thương mại.

RAG Pipeline – Luồng xử lý tổng thể Hình 3.4 mô tả luồng xử lý đầy đủ từ câu hỏi của người dùng đến phản hồi cuối cùng, qua bảy giai đoạn nối tiếp nhau.

1. **Query Reformulation:** Câu hỏi thô được cải thiện để tăng khả năng truy xuất. Ví dụ: “có thịt bò không?” → “thịt bò nguyên liệu nấu ăn”.
2. **Embedding:** Câu hỏi đã cải thiện được chuyển thành vector 768 chiều bằng mô hình Vietnamese SBERT.

mermaid-diagram-2026-05-28-183055.png

Hình 3.4: Sơ đồ luồng xử lý RAG

3. **Hybrid Search:** Thực hiện song song Semantic Search (HNSW + Cosine Similarity) và Keyword Search (GIN + Full-Text Search).
4. **RRF Fusion:** Hợp nhất hai danh sách kết quả thành Top 5 ứng viên bằng Reciprocal Rank Fusion.
5. **Hybrid Ensemble Scoring:** Tính điểm tổng hợp $\alpha S_{\text{Content}} + \beta S_{\text{CF}} + \gamma S_{\text{Apriori}} + \delta S_{\text{Persona}}$.
6. **Session Context Boost:** Nhận diện ngữ cảnh mua sắm của phiên hiện tại và cộng thêm điểm boost cho sản phẩm phù hợp.
7. **LLM Generation:** Mô hình Qwen2.5-7B-Instruct tạo phản hồi ngôn ngữ tự nhiên dựa trên dữ liệu sản phẩm thực tế.

Trụ cột 1: Content-Based Filtering (RAG + RRF) – Hệ số α Trụ cột này tập trung vào việc hiểu sâu nội dung sản phẩm thông qua kỹ thuật RAG, thực hiện tìm kiếm song song để đảm bảo tính bao quát:

- **Semantic Search:** Sử dụng mô hình Vector Embedding và chỉ mục HNSW (Hierarchical Navigable Small World) trên dịch vụ Catalog, đạt độ phức tạp $O(\log n)$ – so sánh với Brute-force vector scan $O(n)$, HNSW chỉ cần ~ 13 phép so sánh thay vì 10.000 với 10.000 sản phẩm.
- **Full-Text Search:** Sử dụng chỉ mục GIN (Generalized Inverted Index) trên PostgreSQL để xử lý các truy vấn từ khóa chính xác, đặc biệt hiệu quả với tên thương hiệu và thuật ngữ chuyên ngành.

Kết quả từ hai luồng được hợp nhất bằng Reciprocal Rank Fusion:

$$\text{RRF}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}, \quad k = 60$$

Lý do chọn Top 5 thay vì Top 10 xuất phát từ phân tích Score Decay: từ rank 6 trở đi, điểm RRF giảm đột ngột trên 50% do hầu hết sản phẩm chỉ xuất hiện ở một luồng. Top 5 còn giúp rút ngắn Context Window gửi đến LLM, giảm hallucination và phù hợp với hành vi người dùng trên giao diện chat.

Trụ cột 2: Luật kết hợp Apriori – Hệ số γ Trụ cột này phục vụ mục đích bán chéo (Cross-selling) bằng cách khai thác các mẫu hành vi mua hàng đồng thời. Thay vì chạy Apriori theo thời gian thực, hệ thống tính toán sẵn các cặp sản phẩm có mối tương quan cao thông qua chỉ số Lift:

$$\text{lift}(A, B) = \frac{|A \cap B| \times |T|}{|A| \times |B|}$$

Lift được chọn là tiêu chí quyết định thay vì Support hay Confidence vì: Support cao không đảm bảo tương quan thật (nước suối có support cao với mọi sản phẩm nhưng $\text{lift} \approx 1$), còn Confidence cao có thể gây hiểu lầm khi một trong hai sản phẩm đã có tần suất mua rất lớn.

Tối ưu hóa: Kết quả Apriori được lưu vào bảng `frequent_itemsets` với Partial Index theo `product_a_id`, đạt tốc độ tra cứu $O(1)$ tại runtime. Toàn bộ tính toán nặng được thực hiện trong Nightly Batch (2AM), không ảnh hưởng đến trải nghiệm người dùng.

Trụ cột 3: Item-based Collaborative Filtering – Hệ số β Trụ cột này dựa trên ma trận tương tác User-Item để tìm ra sự tương đồng giữa các sản phẩm dựa trên hành vi của số đông khách hàng. POSMART sử dụng dữ liệu Implicit Feedback (lượt mua, click, thêm giỏ hàng, thời gian dừng xem) thay vì Explicit Feedback (điểm đánh giá sao).

Cosine Similarity giữa item i và j qua ma trận tương tác $R_{u,i}$:

$$\text{sim}(i, j) = \frac{\sum_u R_{u,i} \cdot R_{u,j}}{\|\vec{R}_{\cdot,i}\| \cdot \|\vec{R}_{\cdot,j}\|}$$

Điểm dự đoán cho người dùng u với sản phẩm ứng viên i (chỉ xét các sản phẩm j mà người dùng đã mua và có $\text{sim}(i, j) \geq 0.1$):

$$\hat{r}_{u,i} = \frac{\sum_{j \in S_u} \text{sim}(i, j) \cdot R_{u,j}}{\sum_{j \in S_u} |\text{sim}(i, j)|}$$

Plain Cosine được chọn thay vì Adjusted Cosine vì dữ liệu Implicit Feedback không bị ảnh hưởng bởi “thang đo cá nhân” như điểm rating 1-5, nên phép trừ mean của Adjusted Cosine gây ra $R_{u,i} - \bar{R}_u \approx 0$ và làm similarity sai về 0.

Phân tích độ phức tạp Batch Job:

Giai đoạn	Thuật toán	Độ phức tạp
Xây dựng ma trận tương tác	Sparse Matrix Construction	$O(U \cdot I)$
Tính toán độ tương đồng	All-pairs Cosine	$O(m^2 \cdot \bar{c})$
Lọc Top-K tương đồng	Heap Sort	$O(m \cdot K \log K)$

Bảng 3.1: Phân tích độ phức tạp Batch Job cho Collaborative Filtering

Với $m = 200$ sản phẩm, hệ thống tạo ra 19.900 cặp; sau Pruning (loại các cặp có ít hơn 2 người dùng chung) còn khoảng 4.000 cặp, hoàn thành trong dưới 5 giây.

Trụ cột 4: Hybrid Ensemble và Adaptive Weight Learning Công thức tổng hợp điểm cuối cùng:

$$\text{Score}(p) = \alpha \cdot S_{\text{Content}}(p) + \beta \cdot S_{\text{CF}}(p) + \gamma \cdot S_{\text{Apriori}}(p) + \delta \cdot S_{\text{Persona}}(u)$$

với $\alpha + \beta + \gamma + \delta = 1$. Tất cả thành phần được chuẩn hóa về $[0, 1]$ trước khi tổng hợp. Bộ trọng số mặc định là $\alpha = 0.40$, $\beta = 0.25$, $\gamma = 0.25$, $\delta = 0.10$.

Thành phần $\delta \cdot S_{\text{Persona}}(u)$ cá nhân hóa kết quả theo loại khách hàng: hệ số Persona bằng 1.0 cho khách VIP (tần suất mua cao, gợi ý có giá trị nhất), 0.8 cho khách sỉ (Wholesale), và 0.3 cho khách lẻ vãng lai (Retail) để tránh làm nhiễu kết quả khi chưa có đủ dữ liệu hành vi.

Xử lý Cold-start: Khi người dùng mới chưa có lịch sử mua, trọng số CF được tái phân bổ sang Content-Based: $\alpha' = \alpha + \beta$, $\beta' = 0$, đảm bảo hệ thống vẫn gợi ý chính xác ngay từ lần đầu sử dụng.

Session Context Boost – Nhận diện ngữ cảnh mua sắm Sau khi tính điểm Ensemble, hệ thống nhận diện mục đích mua sắm của phiên hiện tại thông qua Rule-based Clustering và cộng thêm điểm boost cho các sản phẩm phù hợp ngữ cảnh đó. Điểm cluster được tính theo:

$$\text{totalScore} = \text{productHits} \times 2 + \text{keywordHits} \times 1, \quad \text{confidence} = \frac{\text{topScore}}{\sum \text{allScores}}$$

Hệ thống chỉ kích hoạt boost khi $\text{confidence} \geq 0.4$ và tỷ lệ điểm giữa cluster đứng đầu và cluster thứ hai vượt 1.5 lần – tránh boost sai khi người dùng đang duyệt tổng quát. Bảng 3.2 liệt kê các cluster được định nghĩa sẵn.

Adaptive Weight Learning – Tự động học trọng số Hệ thống không cố định bộ trọng số (α, β, γ) mà tự động học qua Feedback Loop chạy định kỳ hàng đêm (Nightly Batch lúc 2AM). Vòng lặp phản hồi thu thập năm loại hành vi theo chuỗi chuyển đổi: *recommended* $\rightarrow$ *hovered* (rê chuột $\geq 1.5s$) $\rightarrow$ *clicked* $\rightarrow$ *added_to_cart* $\rightarrow$ *purchased* (trong cửa sổ 24 giờ).

Điểm chuyển đổi của mỗi nguồn gợi ý s được tính theo:

$$\text{score}(s) = n_{\text{purchased}} \times 1.0 + n_{\text{cart}} \times 0.5 + n_{\text{clicked}} \times 0.2 + n_{\text{hovered}} \times 0.1$$

Cluster	Tên gọi	Sản phẩm nhận diện	Boost
lau_bo	Lẩu Bò / Nấu ăn	bò, nấm, rau, gia vị, bún	+0.15
bua_sang	Bữa Sáng	bánh mì, sữa, trứng, xúc xích	+0.12
an_vat	Ăn vặt / Sinh viên	mì gói, snack, nước ngọt	+0.12
nhau	Nhậu / Giải khát	bia, khô, đậu phộng	+0.15
gia_vi	Gia vị	nước mắm, muối, đường, dầu ăn	+0.10

Bảng 3.2: Các cluster ngữ cảnh mua sắm trong Session Context Boost

$$\text{rate}(s) = \frac{\text{score}(s)}{n_{\text{recommended}}(s)}, \quad w_{\text{raw}}(s) = \frac{\text{rate}(s)}{\sum_s \text{rate}(s)} \times (1 - \delta)$$

Trọng số mới được cập nhật bằng Exponential Smoothing với $\lambda = 0.8$:

$$w_{t+1}(s) = \lambda \cdot w_t(s) + (1 - \lambda) \cdot w_{\text{raw}}(s)$$

Giá trị $\lambda = 0.8$ đảm bảo hệ thống *học từ dữ liệu mới nhưng không quên lịch sử cốt lõi*: 80% giữ lại trọng số đã được kiểm chứng, 20% cập nhật từ tín hiệu gần đây. Để tránh mất cân bằng, trọng số được giới hạn trong khoảng $[0.05, 0.60]$ và chỉ cập nhật khi đạt tối thiểu 20 feedbacks. Thành phần Personalization (δ) không tham gia vào quá trình học tự động.

Hệ thống theo dõi kép (Dual-Tracking): Để giải quyết bài toán Ma trận Thưa (Data Sparsity), hệ thống thu thập dữ liệu qua hai luồng độc lập: luồng Chatbot (source tag: content/cf/apriori) phục vụ Weight Learner đánh giá hiệu quả AI; và luồng Organic (source tag: organic) từ toàn bộ trang khách hàng phục vụ làm dày ma trận CF. Hai luồng được cách ly dữ liệu nghiêm ngặt – Weight Learner chỉ sử dụng dữ liệu Chatbot để tránh Data Poisoning do organic traffic không có bước recommended.

Nightly Batch Pipeline Toàn bộ quá trình tính toán nặng được thực hiện định kỳ qua bốn bước tuần tự với cơ chế Fault Isolation: mỗi bước có try/catch độc lập, nếu một bước thất bại các bước còn lại vẫn tiếp tục và hệ thống fallback sang dữ liệu cũ thay vì crash toàn bộ pipeline.

1. **Apriori Batch:** Tính toán lại toàn bộ co-purchase pairs, support, confidence, lift từ lịch sử đơn hàng. Cập nhật bảng co_purchase_stats.
2. **CF Similarity:** Xây dựng lại ma trận tương tác User-Item từ dữ liệu kép, tính Cosine Similarity cho tất cả cặp sản phẩm sau Pruning. Cập nhật bảng item_similarity.
3. **Weight Learner:** Đọc recommendation_feedback (chỉ source Chatbot), tính tỷ lệ chuyển đổi, áp dụng Exponential Smoothing. Cập nhật bảng ensemble_weights và lưu snapshot lịch sử vào ensemble_weights_history.
4. **Cache Warmup:** Nạp dữ liệu mới tính toán vào bộ nhớ In-memory Cache để các request runtime tiếp theo được phục vụ với độ trễ thấp nhất.

Tổng hợp độ phức tạp và hiệu năng **Kết luận:** Đối với quy mô siêu thị mini đến chuỗi dưới 100 cửa hàng, kiến trúc Hybrid Ensemble đạt được sự cân bằng tối ưu giữa bốn yếu tố: hoạt động ngay từ ngày đầu không cần dữ liệu lịch sử (Cold-start qua RAG), tự cải thiện theo thời gian sử dụng (Adaptive Weights), giải thích được lý do gợi ý cho người quản trị (White-box), và vận hành toàn bộ trên hạ tầng CPU tiêu chuẩn với độ trễ dưới 500ms – không phụ thuộc vào GPU hay tài nguyên đám mây đắt đỏ.

Thành phần	Offline (Nightly Batch)	Online (Runtime)	Bộ nhớ
RAG + RRF	$O(n)$ embedding sync	$O(\log n)$ HNSW + $O(k)$ FTS	~50MB
Apriori	$O(\sum C(k, 2))$	$O(1)$ index lookup	~1MB
CF Similarity	$O(m^2)$ pairs (pruned)	$O(k)$ prediction	~5MB
Weight Learning	$O(f)$ feedback scan	N/A	Negligible
Session Context	N/A	$O(p)$ với p = sản phẩm	~1KB
Tổng Runtime	–	< 500ms P95	~56MB

Bảng 3.3: Tổng hợp độ phức tạp và hiệu năng của hệ thống gợi ý

3.3 Thiết kế use case

3.3.1 Biểu đồ use case tổng quan



Hình 3.5: Biểu đồ use case tổng quan

3.3.2 Biểu đồ use case phân rã

Phân rã use case "Sales"

3.3.3 Đặc tả use case

Bảng 3.4: Đặc tả Usecase: Đăng nhập

Tên Use case	Đăng nhập
Mã Use case	UC_01
Tác nhân	Admin, Nhân viên
Mô tả	Use case này cho phép người dùng đăng nhập vào hệ thống để bắt đầu phiên làm việc.
Điều kiện tiên quyết	Không có.
Luồng sự kiện chính (Basic Flow)	- Người dùng chọn chức năng "Đăng nhập". - Hệ thống hiển thị màn hình đăng nhập. - Người dùng điền thông tin đăng nhập (Tài khoản, Mật khẩu) và chọn "Đăng nhập". - Hệ thống kiểm tra thông tin đăng nhập hợp lệ. - Hệ thống thông báo đăng nhập thành công và chuyển đến trang chủ.
Luồng sự kiện thay thế (Alternative Flow)	4a. Thông tin đăng nhập không hợp lệ: Hệ thống thông báo lỗi và yêu cầu nhập lại.
Điều kiện sau (Post-condition)	Người dùng đăng nhập thành công vào hệ thống.

Bảng 3.5: Đặc tả Usecase: Tạo hóa đơn (Create order)

Tên Use case	Tạo hóa đơn (Create order)
Mã Use case	UC_02
Tác nhân	Nhân viên
Mô tả	Use case này cho phép nhân viên tạo một hóa đơn mới cho khách mua hàng.
Điều kiện tiên quyết	Nhân viên đã đăng nhập vào hệ thống.
Luồng sự kiện chính (Basic Flow)	- Nhân viên chọn chức năng "Tạo hóa đơn". - Hệ thống hiển thị màn hình tạo hóa đơn. - Nhân viên chọn sản phẩm và số lượng. - Hệ thống tính tổng tiền. - Nhân viên chọn "Thanh toán". - Hệ thống hiển thị màn hình thanh toán. - Nhân viên chọn phương thức thanh toán và nhập số tiền khách đưa. - Hệ thống tính tiền thừa và cập nhật trạng thái hóa đơn. - Hệ thống thông báo thanh toán thành công và in hóa đơn.
Luồng sự kiện thay thế (Alternative Flow)	4a. Sản phẩm không đủ số lượng: Hệ thống thông báo lỗi và yêu cầu điều chỉnh lại số lượng.
Điều kiện sau (Post-condition)	Hóa đơn được tạo và thanh toán thành công, tồn kho được hệ thống cập nhật.

Bảng 3.6: Đặc tả Usecase: Lưu hóa đơn chờ (Hold order)

Tên Use case	Lưu hóa đơn chờ (Hold order)
Mã Use case	UC_03
Tác nhân	Nhân viên
Mô tả	Use case này cho phép nhân viên lưu một hóa đơn đang tạo vào danh sách chờ để xử lý sau.
Điều kiện tiên quyết	Nhân viên đang ở màn hình tạo hóa đơn và đã có sản phẩm được chọn.
Luồng sự kiện chính (Basic Flow)	- Nhân viên chọn chức năng "Lưu chờ". - Hệ thống lưu trạng thái hiện tại của hóa đơn vào danh sách chờ. - Hệ thống thông báo lưu thành công và làm mới màn hình bán hàng để đón khách tiếp theo.
Luồng sự kiện thay thế (Alternative Flow)	Không có.
Điều kiện sau (Post-condition)	Hóa đơn được lưu vào danh sách chờ.

Bảng 3.7: Đặc tả Usecase: Quản lý đơn hàng chờ (Management hold order)

Tên Use case	Quản lý đơn hàng chờ (Management hold order)
Mã Use case	UC_04
Tác nhân	Nhân viên
Mô tả	Use case này cho phép nhân viên quản lý danh sách các hóa đơn đang chờ thanh toán.
Điều kiện tiên quyết	Nhân viên đã đăng nhập vào hệ thống.
Luồng sự kiện chính (Basic Flow)	- Nhân viên chọn chức năng "Đơn hàng chờ". - Hệ thống hiển thị danh sách các hóa đơn đang chờ xử lý. - Nhân viên chọn một hóa đơn để tiếp tục thanh toán hoặc xóa bỏ. - Hệ thống thực hiện yêu cầu (tải lại dữ liệu lên màn hình bán hàng hoặc hủy đơn) và cập nhật lại danh sách.
Luồng sự kiện thay thế (Alternative Flow)	Không có.
Điều kiện sau (Post-condition)	Danh sách hóa đơn chờ được cập nhật thành công.

Bảng 3.8: Đặc tả Usecase: Truy vấn hóa đơn (Query order)

Tên Use case	Truy vấn hóa đơn (Query order)
Mã Use case	UC_05
Tác nhân	Nhân viên
Mô tả	Use case này cho phép nhân viên tra cứu thông tin chi tiết của một hóa đơn đã tạo trong quá khứ.
Điều kiện tiên quyết	Nhân viên đã đăng nhập vào hệ thống.
Luồng sự kiện chính (Basic Flow)	- Nhân viên chọn chức năng "Truy vấn hóa đơn". - Hệ thống hiển thị màn hình tra cứu. - Nhân viên nhập mã hóa đơn hoặc thông tin liên quan để tìm kiếm. - Hệ thống tìm kiếm và hiển thị danh sách kết quả. - Nhân viên chọn xem chi tiết một hóa đơn cụ thể. - Hệ thống hiển thị toàn bộ thông tin chi tiết của hóa đơn đó.
Luồng sự kiện thay thế (Alternative Flow)	4a. Không tìm thấy hóa đơn: Hệ thống thông báo không có kết quả phù hợp với thông tin tìm kiếm.
Điều kiện sau (Post-condition)	Không có.

Bảng 3.9: Đặc tả Usecase: Return order

Tên Use case	Return order		
Mã Use case	UC_06		
Tác nhân	Nhân viên		
Mô tả	Usecase này cho phép nhân viên quản lý đơn hàng bị trả về.		
Điều kiện tiên quyết	Nhân viên đã đăng nhập vào hệ thống.		
Luồng sự kiện chính (Basic Flow)	- Nhân viên chọn chức năng "Đơn hàng" → "Trả hàng". - Hệ thống hiển thị danh sách các đơn hàng đã bị khách hàng trả về. - Nhân viên bấm chọn một đơn hàng. - Hệ thống hiển thị thông tin chi tiết của đơn hàng đó. - Nhân viên xem thông tin đơn hàng và cập nhật trạng thái nếu cần.		
Luồng sự kiện thay thế (Alternative Flow)	Không có		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Trạng thái	String	Bắt buộc chọn
Điều kiện sau (Post-condition)	Hệ thống lưu lại trạng thái mới của đơn hàng đã được cập nhật.		

Bảng 3.10: Đặc tả Usecase: Đăng nhập

Tên Use case	Đăng nhập		
Mã Use case	UC_07		
Tác nhân	Khách hàng		
Mô tả	Khách hàng đăng nhập vào hệ thống để mua hàng.		
Điều kiện tiên quyết	Khách hàng truy cập vào hệ thống.		
Luồng sự kiện chính (Basic Flow)	- Khách hàng chọn chức năng đăng nhập. - Khách hàng nhập email, password. - Khách hàng nhấn nút đăng nhập. - Hệ thống xác nhận thông tin đăng nhập thành công. - Hệ thống chuyển khách hàng sang trang chủ dành cho người dùng đã đăng nhập.		
Luồng sự kiện thay thế (Alternative Flow)	Tại bước 4: Nếu khách hàng nhập sai email hoặc password, hệ thống hiển thị thông báo lỗi.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Email	String	Bắt buộc nhập, đúng định dạng email
	Password	String	Bắt buộc nhập
Điều kiện sau (Post-condition)	Đăng nhập thành công.		

Bảng 3.11: Đặc tả Usecase: Manage Product Category

Tên Use case	Manage Product Category
Mã Use case	UC_08
Tác nhân	Admin
Mô tả	Admin có thể thêm mới, sửa, xóa, tìm kiếm các danh mục sản phẩm (Product Category).
Điều kiện tiên quyết	Admin đã đăng nhập vào hệ thống.
Luồng sự kiện chính (Basic Flow)	- Admin chọn chức năng "Manage Product Category". - Hệ thống hiển thị danh sách các danh mục sản phẩm hiện có. - Admin chọn một trong các thao tác: - Thêm mới danh mục: Nhập thông tin danh mục mới và nhấn "Lưu". - Sửa danh mục: Chọn danh mục cần sửa, thay đổi thông tin và nhấn "Lưu". - Xóa danh mục: Chọn danh mục cần xóa và xác nhận xóa. - Hệ thống cập nhật dữ liệu danh mục sản phẩm. - Hệ thống hiển thị lại danh sách danh mục sản phẩm đã được cập nhật.

Luồng sự kiện thay thế (Alternative Flow)	Không có		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Tên danh mục	String	Bắt buộc nhập
	Mô tả	String	Không bắt buộc
Điều kiện sau (Post-condition)	Danh mục sản phẩm được cập nhật thành công.		

Bảng 3.12: Đặc tả Usecase: Tìm kiếm Category

Tên Use case	Tìm kiếm Category		
Mã Use case	UC_09		
Tác nhân	Admin		
Mô tả	Admin tìm kiếm một danh mục sản phẩm theo tên.		
Điều kiện tiên quyết	Admin đã đăng nhập vào hệ thống và đang ở trang quản lý danh mục sản phẩm.		
Luồng sự kiện chính (Basic Flow)	- Admin nhập tên danh mục vào ô tìm kiếm. - Hệ thống lọc và hiển thị kết quả tìm kiếm tương ứng.		
Luồng sự kiện thay thế (Alternative Flow)	Tại bước 2: Nếu không tìm thấy kết quả, hệ thống hiển thị thông báo "Không tìm thấy danh mục".		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Tên danh mục	String	Bắt buộc nhập
Điều kiện sau (Post-condition)	Kết quả tìm kiếm được hiển thị.		

Bảng 3.13: Đặc tả Usecase: Manage Product

Tên Use case	Manage Product
Mã Use case	UC_10
Tác nhân	Admin
Mô tả	Admin có thể thêm mới, sửa, xóa, tìm kiếm các sản phẩm.
Điều kiện tiên quyết	Admin đã đăng nhập vào hệ thống.

Luồng sự kiện chính (Basic Flow)	1. Admin chọn chức năng "Manage Product". 2. Hệ thống hiển thị danh sách các sản phẩm hiện có. 3. Admin chọn một trong các thao tác: • Thêm mới sản phẩm: Nhập thông tin sản phẩm mới và nhấn "Lưu". • Sửa sản phẩm: Chọn sản phẩm cần sửa, thay đổi thông tin và nhấn "Lưu". • Xóa sản phẩm: Chọn sản phẩm cần xóa và xác nhận xóa. 4. Hệ thống cập nhật dữ liệu sản phẩm. 5. Hệ thống hiển thị lại danh sách sản phẩm đã được cập nhật.																				
Luồng sự kiện thay thế (Alternative Flow)	Không có																				
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><th>Tên trường</th><th>Kiểu dữ liệu</th><th>Ràng buộc</th></tr><tr><td>Tên sản phẩm</td><td>String</td><td>Bắt buộc nhập</td></tr><tr><td>Danh mục</td><td>String</td><td>Bắt buộc chọn</td></tr><tr><td>Giá bán</td><td>Number</td><td>Bắt buộc nhập, > 0</td></tr><tr><td>Mô tả</td><td>String</td><td>Không bắt buộc</td></tr><tr><td>Hình ảnh</td><td>File</td><td>Không bắt buộc</td></tr></table>			Tên trường	Kiểu dữ liệu	Ràng buộc	Tên sản phẩm	String	Bắt buộc nhập	Danh mục	String	Bắt buộc chọn	Giá bán	Number	Bắt buộc nhập, > 0	Mô tả	String	Không bắt buộc	Hình ảnh	File	Không bắt buộc
Tên trường	Kiểu dữ liệu	Ràng buộc																			
Tên sản phẩm	String	Bắt buộc nhập																			
Danh mục	String	Bắt buộc chọn																			
Giá bán	Number	Bắt buộc nhập, > 0																			
Mô tả	String	Không bắt buộc																			
Hình ảnh	File	Không bắt buộc																			
Điều kiện sau (Post-condition)	• Dữ liệu sản phẩm được cập nhật thành công.																				

Bảng 3.14: Đặc tả Usecase: Tìm kiếm Product

Tên Use case	Tìm kiếm Product								
Mã Use case	UC_11								
Tác nhân	Admin								
Mô tả	Admin tìm kiếm một sản phẩm theo tên.								
Điều kiện tiên quyết	Admin đã đăng nhập vào hệ thống và đang ở trang quản lý sản phẩm.								
Luồng sự kiện chính (Basic Flow)	- Admin nhập tên sản phẩm vào ô tìm kiếm. - Hệ thống lọc và hiển thị kết quả tìm kiếm tương ứng.								
Luồng sự kiện thay thế (Alternative Flow)	Tại bước 2: Nếu không tìm thấy kết quả, hệ thống hiển thị thông báo "Không tìm thấy sản phẩm".								
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><td>Tên trường</td><td>Kiểu dữ liệu</td><td>Ràng buộc</td></tr><tr><td>Tên sản phẩm</td><td>String</td><td>Bắt buộc nhập</td></tr></table>			Tên trường	Kiểu dữ liệu	Ràng buộc	Tên sản phẩm	String	Bắt buộc nhập
Tên trường	Kiểu dữ liệu	Ràng buộc							
Tên sản phẩm	String	Bắt buộc nhập							
Điều kiện sau (Post-condition)	Kết quả tìm kiếm được hiển thị.								

Bảng 3.15: Đặc tả Usecase: Manage supplier

Tên Use case	Manage supplier																	
Mã Use case	UC_12																	
Tác nhân	Admin																	
Mô tả	Admin có thể thêm mới, sửa, xóa các nhà cung cấp.																	
Điều kiện tiên quyết	Admin đã đăng nhập vào hệ thống.																	
Luồng sự kiện chính (Basic Flow)	- Admin chọn chức năng "Manage supplier". - Hệ thống hiển thị danh sách các nhà cung cấp hiện có. - Admin chọn một trong các thao tác: - Thêm nhà cung cấp mới: Chọn "Add new supplier", nhập thông tin và nhấn "Save". - Sửa thông tin nhà cung cấp: Chọn biểu tượng chỉnh sửa (Edit) tại nhà cung cấp muốn sửa, cập nhật thông tin và nhấn "Save". - Xóa nhà cung cấp: Chọn biểu tượng xóa (Delete) tại nhà cung cấp muốn xóa và xác nhận "Xóa". - Hệ thống cập nhật dữ liệu. - Hệ thống hiển thị lại danh sách nhà cung cấp đã được cập nhật.																	
Luồng sự kiện thay thế (Alternative Flow)	Không có																	
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><th>Tên trường</th><th>Kiểu dữ liệu</th><th>Ràng buộc</th></tr><tr><td>Company name</td><td>String</td><td>Bắt buộc nhập</td></tr><tr><td>Phone</td><td>String</td><td>Bắt buộc nhập</td></tr><tr><td>Address</td><td>String</td><td>Bắt buộc nhập</td></tr><tr><td>Account number</td><td>String</td><td>Bắt buộc nhập</td></tr></table>			Tên trường	Kiểu dữ liệu	Ràng buộc	Company name	String	Bắt buộc nhập	Phone	String	Bắt buộc nhập	Address	String	Bắt buộc nhập	Account number	String	Bắt buộc nhập
Tên trường	Kiểu dữ liệu	Ràng buộc																
Company name	String	Bắt buộc nhập																
Phone	String	Bắt buộc nhập																
Address	String	Bắt buộc nhập																
Account number	String	Bắt buộc nhập																
Điều kiện sau (Post-condition)	Dữ liệu của nhà cung cấp được cập nhật.																	

Bảng 3.16: Đặc tả Usecase: Tìm kiếm supplier

Tên Use case	Tìm kiếm supplier
Mã Use case	UC_13
Tác nhân	Admin
Mô tả	Admin tìm kiếm một nhà cung cấp.
Điều kiện tiên quyết	Admin đã đăng nhập vào hệ thống.
Luồng sự kiện chính (Basic Flow)	- Admin chọn chức năng "Manage supplier". - Hệ thống hiển thị danh sách các nhà cung cấp hiện có. - Admin nhập tên nhà cung cấp cần tìm vào ô tìm kiếm. - Hệ thống lọc và hiển thị kết quả.
Luồng sự kiện thay thế (Alternative Flow)	Không có

Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Company name	String	Bắt buộc nhập
Điều kiện sau (Post-condition)	Kết quả tìm kiếm được hiển thị.		

Bảng 3.17: Đặc tả Usecase: View Inventory

Tên Use case	Xem tồn kho
Mã Use case	UC014
Tác nhân	Manager
Mô tả	Xem tồn kho có trên hệ thống
Điều kiện tiên quyết	Đăng nhập thành công vào hệ thống
Luồng sự kiện chính (Basic Flow)	1. Manager: Yêu cầu xem danh sách Inventories 2. Hệ thống: Hiển thị danh sách Inventories
Luồng sự kiện thay thế (Alternative Flow)	2a. Hệ thống: Thông báo lỗi nếu không có ít nhất một Inventories nào
Dữ liệu đầu vào	Không có
Điều kiện sau (Post-condition)	Hiển thị những Inventories tương ứng với thông tin cần tìm kiếm

Bảng 3.18: Đặc tả Usecase: Manage Movement

Tên Use case	Quản lý chuyển hàng
Mã Use case	UC015
Tác nhân	Manager
Mô tả	Quản lý chuyển hàng trên hệ thống
Điều kiện tiên quyết	Hiển thị thành công List Inventories

Luồng sự kiện chính (Basic Flow)	Xem (R - Read): 1. Manager: Yêu cầu xem danh sách Movement History của Inventories 2. Hệ thống: Hiển thị danh sách lịch sử chuyển hàng của Product tương ứng Thêm (C - Create): 1. Manager: Click button Bulk Transfer Stock trên giao diện Inventories 2. Hệ thống: Hiển thị giao diện tạo Movement 3. Manager: Nhập các thông tin và yêu cầu thêm mới (*) 4. Hệ thống: Kiểm tra trường nhập liệu 5. Hệ thống: Thay đổi số lượng hàng hóa on hand, on shelf 6. Hệ thống: Thêm mới thông tin Movement																								
Luồng sự kiện thay thế (Alternative Flow)	Xem: 2a. Hệ thống: Thông báo lỗi nếu không có ít nhất một Movement nào Thêm: 4a. Hệ thống: Thông báo lỗi: Trường nhập liệu không hợp lệ 6a. Hệ thống: Thông báo lỗi: Thêm mới không thành công																								
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><th>Tên trường</th><th>Kiểu dữ liệu</th><th>Ràng buộc</th></tr><tr><td>Transfer Direction</td><td>Radio Button</td><td>Bắt buộc: Có, 1 trong 2 button</td></tr><tr><td>Tên</td><td>Dropdown</td><td>Bắt buộc: Có, Tối đa 255 kí tự</td></tr><tr><td>Batch</td><td>Lô hàng</td><td>Bắt buộc: Có, Đúng định dạng</td></tr><tr><td>Quantity</td><td>Số lượng</td><td>Bắt buộc: Có, Number</td></tr><tr><td>Date</td><td>Ngày hết hạn</td><td>Bắt buộc: Có, Định dạng Date</td></tr><tr><td>Reason</td><td>Lý do Move</td><td>Bắt buộc: Không, Text</td></tr><tr><td>Note</td><td>Mô tả</td><td>Bắt buộc: Không, Text</td></tr></table>	Tên trường	Kiểu dữ liệu	Ràng buộc	Transfer Direction	Radio Button	Bắt buộc: Có, 1 trong 2 button	Tên	Dropdown	Bắt buộc: Có, Tối đa 255 kí tự	Batch	Lô hàng	Bắt buộc: Có, Đúng định dạng	Quantity	Số lượng	Bắt buộc: Có, Number	Date	Ngày hết hạn	Bắt buộc: Có, Định dạng Date	Reason	Lý do Move	Bắt buộc: Không, Text	Note	Mô tả	Bắt buộc: Không, Text
Tên trường	Kiểu dữ liệu	Ràng buộc																							
Transfer Direction	Radio Button	Bắt buộc: Có, 1 trong 2 button																							
Tên	Dropdown	Bắt buộc: Có, Tối đa 255 kí tự																							
Batch	Lô hàng	Bắt buộc: Có, Đúng định dạng																							
Quantity	Số lượng	Bắt buộc: Có, Number																							
Date	Ngày hết hạn	Bắt buộc: Có, Định dạng Date																							
Reason	Lý do Move	Bắt buộc: Không, Text																							
Note	Mô tả	Bắt buộc: Không, Text																							
Điều kiện sau (Post-condition)	Hiển thị những lịch sử chuyển hàng tương ứng với thông tin cần tìm kiếm, thêm đơn nhập thành công																								

Bảng 3.19: Đặc tả Usecase: Manage Batch

Tên Use case	Quản lý lô hàng
Mã Use case	UC016
Tác nhân	Manager
Mô tả	Quản lý lô hàng trên hệ thống
Điều kiện tiên quyết	Hiển thị thành công List Inventories

Luồng sự kiện chính (Basic Flow)	Xem (R - Read): 1. Manager: Yêu cầu xem danh sách Batch của Inventories 2. Hệ thống: Hiển thị danh sách Batch Product tương ứng Cập nhật (U - Update): 1. Manager: Click button Adjust Batch trên giao diện mỗi Batch 2. Hệ thống: Hiển thị giao diện chỉnh sửa Batch 3. Manager: Nhập các thông tin và yêu cầu chỉnh sửa (*) 4. Hệ thống: Kiểm tra trường nhập liệu 5. Hệ thống: Cập nhật dữ liệu của Batch														
Luồng sự kiện thay thế (Alternative Flow)	Xem: 2a. Hệ thống: Thông báo lỗi nếu không có ít nhất một Batch nào Cập nhật: 4a. Hệ thống: Thông báo lỗi: Trường nhập liệu không hợp lệ														
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><th>Tên trường</th><th>Kiểu dữ liệu</th><th>Ràng buộc</th></tr><tr><td>Location</td><td>Dropdown</td><td>Bắt buộc: Có</td></tr><tr><td>Quantity</td><td>Số lượng</td><td>Bắt buộc: Có, Number</td></tr><tr><td>Reason</td><td>Lý do</td><td>Bắt buộc: Optional, Text</td></tr></table>			Tên trường	Kiểu dữ liệu	Ràng buộc	Location	Dropdown	Bắt buộc: Có	Quantity	Số lượng	Bắt buộc: Có, Number	Reason	Lý do	Bắt buộc: Optional, Text
Tên trường	Kiểu dữ liệu	Ràng buộc													
Location	Dropdown	Bắt buộc: Có													
Quantity	Số lượng	Bắt buộc: Có, Number													
Reason	Lý do	Bắt buộc: Optional, Text													
Điều kiện sau (Post-condition)	Xem, cập nhật thành công trạng thái lô hàng														

Bảng 3.20: Đặc tả Usecase: Tạo chỉnh sửa tồn kho

Tên Use case	Chỉnh sửa tồn kho
Mã Use case	UC017
Tác nhân	Manager
Mô tả	Chỉnh sửa tồn kho của lô hàng
Điều kiện tiên quyết	Hiển thị thành công List Batches
Luồng sự kiện chính (Basic Flow)	1. Manager: Click Action "Adjust Stock" 2. Manager: Nhập số lượng, lý do chỉnh sửa 3. Hệ thống: Kiểm tra các trường nhập liệu (*) 4. Hệ thống: Cập nhật trạng thái mới của Batch
Luồng sự kiện thay thế (Alternative Flow)	3a. Hệ thống: Thông báo lỗi: Trường nhập liệu không hợp lệ
Dữ liệu đầu vào	Không có
Điều kiện sau (Post-condition)	Tạo, chỉnh sửa thành công tồn kho của lô hàng

Bảng 3.21: Đặc tả Usecase: Quản lý hủy lô hàng

Tên Use case	Hủy lô hàng hết hạn
---------------------	---------------------

Mã Use case	UC018
Tác nhân	Manager
Mô tả	Đánh dấu hủy lô hàng đã hết hạn
Điều kiện tiên quyết	Hiển thị thành công List Batches
Luồng sự kiện chính (Basic Flow)	1. Manager: Click Action "Disable" 2. Manager: Nhập lý do 3. Hệ thống: Kiểm tra điều kiện hủy tương ứng với lý do 4. Hệ thống: Cập nhật trạng thái mới của Batch
Luồng sự kiện thay thế (Alternative Flow)	3a. Hệ thống: Thông báo lỗi: Không thỏa mãn điều kiện hủy
Dữ liệu đầu vào	Không có
Điều kiện sau (Post-condition)	Hủy thành công lô hàng được chọn

Bảng 3.22: Đặc tả Usecase: View Disposal History

Tên Use case	View Disposal History
Mã Use case	UC019
Tác nhân	Manager
Mô tả	Xem lịch sử hủy lô hàng
Điều kiện tiên quyết	Hiển thị thành công List Batches
Luồng sự kiện chính (Basic Flow)	1. Manager: Click button "View Disposal History" 2. Hệ thống: Hiển thị danh sách lịch sử hủy hàng của Product tương ứng
Luồng sự kiện thay thế (Alternative Flow)	2a. Hệ thống: Thông báo lỗi nếu không có ít nhất một History nào
Dữ liệu đầu vào	Không có
Điều kiện sau (Post-condition)	Hiển thị những History tương ứng với thông tin cần tìm kiếm

Bảng 3.23: Đặc tả Usecase: Quản lý Role

Tên Use case	Quản lý Role
Mã Use case	UC020
Tác nhân	Admin
Mô tả	Thực hiện các tác vụ như thêm, sửa, xóa, tìm kiếm Role
Điều kiện tiên quyết	Đăng nhập thành công

Luồng sự kiện chính (Basic Flow)	Xem (R - Read): 1. Admin: Yêu cầu xem danh sách Role 2. Hệ thống: Hiển thị danh sách Role Sửa (U - Update): 1. Admin: Chọn yêu cầu sửa Role trên danh sách 2. Hệ thống: Lấy thông tin chi tiết Role và hiển thị thông tin lên giao diện của chức năng sửa 3. Admin: Chỉnh sửa các thông tin Role (mô tả phía dưới **) và yêu cầu sửa 4. Hệ thống: Kiểm tra các trường nhập liệu 5. Hệ thống: Cập nhật các thông tin cần chỉnh sửa và thông báo chỉnh sửa thành công. Xóa (D - Delete): 1. Admin: Chọn Role cần xóa và yêu cầu xóa 2. Hệ thống: Hiển thị thông báo yêu cầu xác nhận việc xóa 3. Admin: Xác nhận xóa 4. Hệ thống: Xóa và thông báo xóa thành công Thêm (C-Create): 1. Admin: Yêu cầu chức năng thêm mới Role 2. Hệ thống: Hiển thị chức năng thêm mới Role 3. Admin: Nhập các thông tin Role (mô tả phía dưới **) và yêu cầu thêm mới 4. Hệ thống: Kiểm tra các trường nhập liệu 5. Hệ thống: Thêm mới thông tin Role												
Luồng sự kiện thay thế (Alternative Flow)	Xem: 2a. Hệ thống: Thông báo lỗi nếu không có ít nhất một Role nào Sửa: 4a. Hệ thống: Thông báo lỗi nếu dữ liệu nhập vào không đúng định dạng 5a. Hệ thống: Thông báo lỗi nếu cập nhật không thành công Xóa: 4a. Hệ thống: Thông báo lỗi nếu xóa không thành công Thêm: 4a. Hệ thống: Thông báo lỗi nếu dữ liệu nhập vào không đúng định dạng 5a. Hệ thống: Thông báo lỗi nếu thêm mới không thành công												
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><th>Tên trường</th><th>Kiểu dữ liệu</th><th>Ràng buộc</th></tr><tr><td>Tên</td><td>Text field</td><td>Bắt buộc: Có, Tối đa 255 kí tự</td></tr><tr><td>Description</td><td>Giảng viên</td><td>Bắt buộc: Không, Text</td></tr><tr><td>Permissions</td><td>Dropdown</td><td>Bắt buộc: Có</td></tr></table>	Tên trường	Kiểu dữ liệu	Ràng buộc	Tên	Text field	Bắt buộc: Có, Tối đa 255 kí tự	Description	Giảng viên	Bắt buộc: Không, Text	Permissions	Dropdown	Bắt buộc: Có
Tên trường	Kiểu dữ liệu	Ràng buộc											
Tên	Text field	Bắt buộc: Có, Tối đa 255 kí tự											
Description	Giảng viên	Bắt buộc: Không, Text											
Permissions	Dropdown	Bắt buộc: Có											
Điều kiện sau (Post-condition)	Hiển thị danh sách tương ứng với thông tin cần tìm kiếm; Cập nhật thành công, thông tin mới sẽ được lưu trữ vào hệ thống; Xóa thành công Role; Role khi tạo mới được lưu trữ trong cơ sở dữ liệu của hệ thống.												

Bảng 3.24: Đặc tả Usecase: Tìm kiếm Role

Tên Use case	Tìm kiếm Role		
Mã Use case	UC021		
Tác nhân	Admin		
Mô tả	Tìm kiếm Role có trên hệ thống		
Điều kiện tiên quyết	• Đăng nhập thành công vào hệ thống		
Luồng sự kiện chính (Basic Flow)	1. Admin: Click Search Box tìm kiếm 2. Admin: Nhập tên, tên Role muốn tìm kiếm (mô tả phía dưới *) 3. Admin: Yêu cầu tìm kiếm 5. Hệ thống: Tìm và lấy về thông tin những Role thoả mãn các tiêu chí tìm kiếm 6. Hệ thống: Hiển thị danh sách những Role thoả mãn điều kiện tìm kiếm nếu có ít nhất một Role được tìm thấy		
Luồng sự kiện thay thế (Alternative Flow)	6a. Hệ thống: Thông báo: Không tìm thấy thông tin Role nào thoả mãn tiêu chí tìm kiếm nếu trả về danh sách rỗng		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Tên	Input text field	Bắt buộc: Không, Chuỗi kí tự
Điều kiện sau (Post-condition)	• Hiển thị những Role tương ứng với thông tin cần tìm kiếm		

Bảng 3.25: Đặc tả Usecase: Quản lý Tài khoản

Tên Use case	Quản lý Tài khoản
Mã Use case	UC022
Tác nhân	Admin
Mô tả	Thực hiện các tác vụ như thêm, sửa, xóa, tìm kiếm Tài khoản
Điều kiện tiên quyết	• Đăng nhập thành công

Luồng sự kiện chính (Basic Flow)	Xem (R - Read): 1. Admin: Yêu cầu xem danh sách Account 2. Hệ thống: Hiển thị danh sách Account Sửa (U - Update): 1. Admin: Chọn yêu cầu sửa Account trên danh sách 2. Hệ thống: Lấy thông tin chi tiết Account và hiển thị thông tin lên giao diện của chức năng sửa 3. Admin: Chỉnh sửa các thông tin Account (mô tả phía dưới **) và yêu cầu sửa 4. Hệ thống: Kiểm tra các trường nhập liệu 5. Hệ thống: Cập nhật các thông tin cần chỉnh sửa và thông báo chỉnh sửa thành công. Xóa (D- Delete): 1. Admin: Chọn Account cần xóa và yêu cầu xóa 2. Hệ thống: Hiển thị thông báo yêu cầu xác nhận việc xóa 3. Admin: Xác nhận xóa 4. Hệ thống: Xóa và thông báo xóa thành công Thêm (C-Create): 1. Admin: Yêu cầu chức năng thêm mới Account 2. Hệ thống: Hiển thị chức năng thêm mới Account 3. Admin: Nhập các thông tin Account (mô tả phía dưới **) và yêu cầu thêm mới 4. Hệ thống: Kiểm tra các trường nhập liệu 5. Hệ thống: Thêm mới thông tin Account
Luồng sự kiện thay thế (Alternative Flow)	Xem: 2a. Hệ thống: Thông báo lỗi nếu không có ít nhất một Account nào Sửa: 4a. Hệ thống: Thông báo lỗi nếu dữ liệu nhập vào không đúng định dạng 5a. Hệ thống: Thông báo lỗi nếu cập nhật không thành công Xóa: 4a. Hệ thống: Thông báo lỗi nếu xóa không thành công Thêm: 4a. Hệ thống: Thông báo lỗi nếu dữ liệu nhập vào không đúng định dạng 5a. Hệ thống: Thông báo lỗi nếu thêm mới không thành công

Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Username	Tên đăng nhập	Bắt buộc: Có, Tối đa 255 ký tự
	Email	Email	Bắt buộc: Có, Đúng định dạng email
	Password	Mật khẩu	Bắt buộc: Có, Tối thiểu 6 ký tự
	Role	Role	Bắt buộc: Có
	Full Name	Tên	Bắt buộc: Có, 3-100 ký tự
	Phone Number	Số điện thoại	Bắt buộc: Không, Đúng định dạng
	DOB	Ngày sinh	Bắt buộc: Không, Đúng định dạng
	Address	Địa chỉ	Bắt buộc: Không, Đúng định dạng
	Status	Trạng thái	Bắt buộc: Có, Active/Inactive
Điều kiện sau (Post-condition)	Hiển thị danh sách tương ứng với thông tin cần tìm kiếm; Cập nhật thành công, thông tin mới sẽ được lưu trữ vào hệ thống; Xóa thành công Account; Account khi tạo mới được lưu trữ trong cơ sở dữ liệu của hệ thống.		

Bảng 3.26: Đặc tả Usecase: Tìm kiếm Account

Tên Use case	Tìm kiếm Account
Mã Use case	UC023
Tác nhân	Admin
Mô tả	Tìm kiếm Account có trên hệ thống
Điều kiện tiên quyết	Đăng nhập thành công vào hệ thống
Luồng sự kiện chính (Basic Flow)	- Admin: Click Search Box tìm kiếm - Admin: Nhập tên, tên Account muốn tìm kiếm (mô tả phía dưới *) - Admin: Yêu cầu tìm kiếm - Hệ thống: Tìm và lấy về thông tin những Account thoả mãn các tiêu chí tìm kiếm - Hệ thống: Hiển thị danh sách những Account thoả mãn điều kiện tìm kiếm nếu có ít nhất một Role được tìm thấy
Luồng sự kiện thay thế (Alternative Flow)	6a. Hệ thống: Thông báo: Không tìm thấy thông tin Account nào thoả mãn tiêu chí tìm kiếm nếu trả về danh sách rỗng

Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Kiểu dữ liệu	Ràng buộc
	Tên	Input text field	Bắt buộc: Không, Chuỗi kí tự
Điều kiện sau (Post-condition)	Hiển thị những Account tương ứng với thông tin cần tìm kiếm		

Bảng 3.27: Đặc tả Usecase: Quản lý Đơn nhập hàng (Purchase Order)

Mã Use case	UC025		
Tên Use case	Quản lý đơn nhập hàng		
Tác nhân	Manager		
Mô tả	Thực hiện các tác vụ tạo mới, xem chi tiết, và duyệt đơn đặt hàng từ nhà cung cấp.		
Sự kiện kích hoạt	Click nút "Add Purchase Order" hoặc đổi trạng thái đơn hàng.		
Tiền điều kiện	Manager đã đăng nhập thành công vào hệ thống.		
Luồng sự kiện chính (Thành công)	- Manager yêu cầu chức năng tạo đơn nhập hàng mới. - Hệ thống hiển thị giao diện thêm Purchase Order. - Manager chọn nhà cung cấp, thêm các sản phẩm, nhập số lượng, giá nhập và nhấn "Lưu". - Hệ thống kiểm tra dữ liệu và lưu đơn hàng với trạng thái nháp ("Draft"). - Manager xem lại đơn và nhấn duyệt ("Approve"). - Hệ thống cập nhật trạng thái đơn thành "Approved" và cộng thêm công nợ cho nhà cung cấp tương ứng.		
Luồng sự kiện thay thế	4a. Dữ liệu không hợp lệ: Hệ thống thông báo lỗi thiếu sản phẩm hoặc nhà cung cấp đang bị khóa.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Nhà cung cấp	Có	Đang Active
	Sản phẩm	Có	Nằm trong Catalog
	Số lượng	Có	> 0
	Đơn giá nhập	Có	≥ 0
Hậu điều kiện	Đơn nhập hàng được lưu vào cơ sở dữ liệu, công nợ nhà cung cấp được tính toán lại.		

Bảng 3.28: Đặc tả Usecase: Nhập kho hàng hóa (Receive Stock)

Mã Use case	UC026
--------------------	-------

Tên Use case	Nhập kho hàng hóa		
Tác nhân	Manager		
Mô tả	Ghi nhận số lượng thực tế nhận được từ đơn đặt hàng và cộng dồn vào tồn kho hệ thống.		
Sự kiện kích hoạt	Click nút "Receive" tại đơn nhập hàng đã được duyệt.		
Tiền điều kiện	Đơn nhập hàng (Purchase Order) đang ở trạng thái "Approved".		
Luồng sự kiện chính (Thành công)	- Manager mở đơn nhập hàng và yêu cầu nhập kho. - Hệ thống hiển thị danh sách sản phẩm cần nhập. - Manager nhập ngày hết hạn (Expiry Date) và chọn vị trí lưu kho (Location) cho từng sản phẩm. - Hệ thống kiểm tra sức chứa của vị trí kho và tạo các Lô hàng mới (Product Batch). - Hệ thống cộng dồn số lượng vào tồn kho thực tế (Inventory Item). - Hệ thống tự động tạo phiếu lịch sử biến động (Inventory Movement) và cập nhật đơn hàng thành "Received".		
Luồng sự kiện thay thế	4a. Vị trí kho không hợp lệ: Hệ thống báo lỗi sức chứa không đủ và rollback giao dịch.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Location	Có	Thuộc kho hiện tại
	Expiry Date	Có	Ngày hợp lệ (> hiện tại)
Hậu điều kiện	Tồn kho thực tế tăng lên, lô hàng mới được quản lý trên hệ thống.		

Bảng 3.29: Đặc tả Usecase: Quản lý Phiếu Xuất Kho (Stock Out)

Mã Use case	UC027
Tên Use case	Quản lý phiếu xuất kho
Tác nhân	Manager
Mô tả	Quản lý việc xuất kho cho các mục đích ngoài bán lẻ (như hủy hàng hết hạn, xuất trả nhà cung cấp, hoặc hao hụt).
Sự kiện kích hoạt	Click "Create Stock Out Order" trên màn hình Inventory.
Tiền điều kiện	Đang có sẵn lô hàng còn tồn kho (> 0).

Luồng sự kiện chính (Thành công)	1. Manager yêu cầu tạo phiếu xuất kho mới. 2. Hệ thống hiển thị giao diện tạo phiếu. 3. Manager chọn lý do xuất (Hư hỏng/Trả hàng), chọn lô hàng (Batch) cần xuất và nhập số lượng. 4. Hệ thống kiểm tra số lượng tồn kho hiện tại của lô đó. 5. Manager xác nhận "Hoàn thành" phiếu xuất. 6. Hệ thống trừ trực tiếp số lượng khỏi kho, tính toán lại lượng tồn và tạo lịch sử xuất kho.														
Luồng sự kiện thay thế	• 4a. Số lượng xuất lớn hơn tồn kho: Hệ thống báo lỗi "Tồn kho không đủ" và chặn thao tác.														
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><th>Tên trường</th><th>Bắt buộc</th><th>Ràng buộc</th></tr><tr><td>Lý do xuất</td><td>Có</td><td>Chọn từ danh sách</td></tr><tr><td>Lô hàng (Batch)</td><td>Có</td><td>Đang có tồn kho</td></tr><tr><td>Số lượng</td><td>Có</td><td>> 0 và ≤ tồn kho</td></tr></table>			Tên trường	Bắt buộc	Ràng buộc	Lý do xuất	Có	Chọn từ danh sách	Lô hàng (Batch)	Có	Đang có tồn kho	Số lượng	Có	> 0 và ≤ tồn kho
Tên trường	Bắt buộc	Ràng buộc													
Lý do xuất	Có	Chọn từ danh sách													
Lô hàng (Batch)	Có	Đang có tồn kho													
Số lượng	Có	> 0 và ≤ tồn kho													
Hậu điều kiện	• Hàng hóa được trừ khỏi kho hợp lệ, tổng giá trị tồn kho giảm.														

Bảng 3.30: Đặc tả Usecase: Thanh toán qua cổng VNPay

Mã Use case	UC028
Tên Use case	Thanh toán qua VNPay
Tác nhân	Sales, Khách hàng
Mô tả	Khách hàng thanh toán đơn hàng bằng cách quét mã QR qua cổng VNPay. Hệ thống xử lý tự động qua Webhook.
Sự kiện kích hoạt	Chọn phương thức "VNPay" khi đang thanh toán đơn hàng.
Tiền điều kiện	• Đơn hàng bán ra đang ở trạng thái chờ thanh toán.
Luồng sự kiện chính (Thành công)	1. Sales chọn thanh toán bằng chuyển khoản VNPay. 2. Hệ thống mã hóa thông tin đơn hàng và gọi API VNPay để tạo URL thanh toán. 3. Hệ thống hiển thị mã QR Code lên màn hình POS. 4. Khách hàng quét mã và thanh toán thành công trên App ngân hàng. 5. Cổng VNPay tự động gọi Webhook (IPN) chứa dữ liệu giao dịch về server POSMART. 6. Hệ thống xác thực chữ ký bảo mật (Secure Hash), cập nhật thanh toán thành "Completed" và chốt đơn hàng.
Luồng sự kiện thay thế	• 6a. Chữ ký IPN không hợp lệ: Hệ thống từ chối cập nhật để phòng chống gian lận. • 5a. Khách hàng hủy giao dịch trên App: IPN trả về mã lỗi, hệ thống cập nhật Payment thành "Failed".
Dữ liệu đầu vào	Không yêu cầu nhập tay, dữ liệu tự động đồng bộ từ VNPay.

Hậu điều kiện	Đơn hàng được ghi nhận thanh toán thành công, trừ kho sản phẩm.
----------------------	---

Bảng 3.31: Đặc tả Usecase: Tương tác với Trợ lý AI (Chatbot)

Mã Use case	UC029		
Tên Use case	Tra cứu dữ liệu qua AI Chatbot		
Tác nhân	Manager, Sales, Khách hàng		
Mô tả	Tương tác với trợ lý AI bằng ngôn ngữ tự nhiên để tra cứu thông tin sản phẩm, tồn kho và báo cáo doanh thu thông qua API của Hugging Face.		
Sự kiện kích hoạt	Nhập tin nhắn vào khung chat AI trên hệ thống.		
Tiền điều kiện	Hệ thống kết nối API với Hugging Face thành công.		
Luồng sự kiện chính (Thành công)	- Người dùng gửi câu hỏi (Ví dụ: "Cửa hàng còn Coca Cola không?"). - Chatbot Service phân tích và nhận diện ý định (Intent Classifier) của người dùng. - Hệ thống tự động kích hoạt Function Calling, gọi API nội bộ (Inventory/Catalog) để lấy dữ liệu thực tế. - Hệ thống đóng gói dữ liệu thu thập được và gửi cho mô hình LLM. - Mô hình sinh ra câu trả lời bằng ngôn ngữ tự nhiên (Ví dụ: "Sản phẩm Coca Cola hiện còn 57 lon trên kệ"). - Hệ thống hiển thị câu trả lời và lưu trữ lịch sử chat.		
Luồng sự kiện thay thế	3a. Vi phạm phân quyền: Nếu Khách hàng hỏi về doanh thu (chỉ dành cho Manager), hệ thống từ chối truy xuất dữ liệu do ràng buộc RBAC. 2a. Câu hỏi giao tiếp ngoài lề: AI phản hồi theo kiến thức thông thường mà không cần gọi DB nội bộ.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Đoạn chat (Prompt)	Có	Text tự do
Hậu điều kiện	Người dùng nhận được thông tin chính xác theo thời gian thực.		

Bảng 3.32: Đặc tả Usecase: Quản lý Cấu hình hệ thống (System Settings)

Mã Use case	UC030
Tên Use case	Quản lý cấu hình hệ thống

Tác nhân	Admin		
Mô tả	Quản lý và thiết lập các cấu hình dùng chung cho toàn bộ chuỗi cửa hàng, bao gồm cấu hình bảo mật (Security Settings) và cấu hình bán hàng/chiết khấu (Sales Settings).		
Sự kiện kích hoạt	Click vào mục "Settings" trên thanh điều hướng.		
Tiền điều kiện	Admin đã đăng nhập thành công vào hệ thống.		
Luồng sự kiện chính (Thành công)	- Admin chọn chức năng Cấu hình hệ thống. - Hệ thống truy xuất và hiển thị các thông số hiện tại (số lần sai mã PIN tối đa, chiết khấu mặc định cho VIP/Wholesale, v.v.). - Admin thay đổi các thông số cần thiết, nhập "Lý do thay đổi" (Change Reason) và nhấn "Save Changes". - Hệ thống kiểm tra tính hợp lệ của các giá trị. - Hệ thống lưu cấu hình mới và ghi nhận lại lịch sử thay đổi (Audit Trail) vào bảng settings_history. - Hệ thống thông báo cập nhật thành công.		
Luồng sự kiện thay thế	4a. Thiếu lý do thay đổi: Hệ thống báo lỗi "Change reason is required for audit trails" và yêu cầu nhập bổ sung.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Các thông số cấu hình	Có	Số nguyên dương
	Change Reason	Có	Text tự do
Hậu điều kiện	Cấu hình mới được áp dụng ngay lập tức trên toàn hệ thống; lịch sử được ghi log.		

Bảng 3.33: Đặc tả Usecase: Xem Báo cáo thống kê (View Reports)

Mã Use case	UC031
Tên Use case	Xem báo cáo thống kê
Tác nhân	Admin, Manager
Mô tả	Xem các báo cáo phân tích về doanh thu, lợi nhuận, hàng tồn kho và hiệu suất bán hàng của nhân viên.
Sự kiện kích hoạt	Click vào menu "Reports" và chọn loại báo cáo tương ứng.
Tiền điều kiện	Người dùng đã đăng nhập thành công.

Luồng sự kiện chính (Thành công)	1. Người dùng chọn chức năng Xem báo cáo. 2. Người dùng chọn bộ lọc thời gian (Hôm nay, Tuần này, Tháng này, Năm nay) và loại báo cáo (Sales, Profit, Inventory). 3. Hệ thống tổng hợp dữ liệu từ các dịch vụ (Order, Inventory, Payment) theo thời gian đã chọn. 4. Hệ thống tính toán các chỉ số (Tổng doanh thu, Tổng chi phí, Lợi nhuận gộp, Số lượng bán). 5. Hệ thống hiển thị dữ liệu dưới dạng bảng biểu và biểu đồ trực quan.								
Luồng sự kiện thay thế	• 3a. Không có dữ liệu giao dịch: Hệ thống hiển thị biểu đồ trống và thông báo "Không có dữ liệu trong khoảng thời gian này".								
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><td>Tên trường</td><td>Bắt buộc</td><td>Ràng buộc</td></tr><tr><td>Khoảng thời gian</td><td>Có</td><td>Từ ngày ≤ Đến ngày</td></tr></table>			Tên trường	Bắt buộc	Ràng buộc	Khoảng thời gian	Có	Từ ngày ≤ Đến ngày
Tên trường	Bắt buộc	Ràng buộc							
Khoảng thời gian	Có	Từ ngày ≤ Đến ngày							
Hậu điều kiện	• Hệ thống hiển thị chính xác các số liệu thống kê.								

Bảng 3.34: Đặc tả Usecase: Quản lý Khách hàng (Manage Customer)

Mã Use case	UC032
Tên Use case	Quản lý khách hàng
Tác nhân	Manager, Sales
Mô tả	Quản lý hồ sơ khách hàng, tra cứu thông tin và theo dõi tổng chi tiêu (Total Spent) của khách hàng.
Sự kiện kích hoạt	Click vào nút "Customers" hoặc "Add Customer".
Tiền điều kiện	Người dùng đã đăng nhập thành công vào hệ thống.
Luồng sự kiện chính (Thành công)	1. Người dùng truy cập danh sách khách hàng. 2. Hệ thống hiển thị danh sách kèm theo tổng chi tiêu và phân loại khách hàng (Retail, Wholesale, VIP). 3. Người dùng chọn "Add Customer", nhập thông tin cá nhân (Tên, SĐT, Email). 4. Hệ thống kiểm tra trùng lặp dữ liệu (Email/SĐT). 5. Hệ thống lưu hồ sơ khách hàng mới vào cơ sở dữ liệu. 6. Hệ thống hiển thị thông báo thành công và cập nhật danh sách.
Luồng sự kiện thay thế	4a. Email đã tồn tại: Hệ thống chặn lưu và báo lỗi "Email already exists".

Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Full name	Có	Text
	Email	Có	Đúng định dạng Email
	Phone number	Không	Số điện thoại hợp lệ
	Customer Type	Có	Dropdown (Re-tail/VIP...)
Hậu điều kiện	Thông tin khách hàng được hệ thống lưu trữ/cập nhật.		

Bảng 3.35: Đặc tả Usecase: Quản lý Vị trí Kho bãi (Warehouse Locations)

Mã Use case	UC033		
Tên Use case	Quản lý sơ đồ vị trí kho		
Tác nhân	Manager		
Mô tả	Thiết lập và quản lý các khu vực kho (Block) và các ô/kệ hàng (Location) để tối ưu hóa việc lưu trữ và lấy hàng.		
Sự kiện kích hoạt	Click vào menu "Inventory" → "Locations".		
Tiền điều kiện	Manager đã đăng nhập thành công.		
Luồng sự kiện chính (Thành công)	- Manager yêu cầu xem sơ đồ kho. - Hệ thống hiển thị các Block hiện có và sức chứa của từng ô kệ (Location) kèm mã màu trực quan (Trống, Sắp đầy, Đầy). - Manager chọn "Build Warehouse Map" để thêm một Block mới. - Manager nhập tên Block, số Hàng (Rows), số Cột (Cols). - Hệ thống kiểm tra và tự động sinh ra ma trận các vị trí (Locations) tương ứng trong DB. - Hệ thống tải lại sơ đồ kho với Block mới được thêm.		
Luồng sự kiện thay thế	4a. Tên Block bị trùng trong cùng một cửa hàng: Hệ thống báo lỗi "Name already exists" và chặn khởi tạo.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Tên Block	Có	Text (Tối đa 50 kí tự)
	Số hàng (Rows)	Có	$1 \leq Rows \leq 20$
	Số cột (Cols)	Có	$1 \leq Cols \leq 20$
Hậu điều kiện	Cấu trúc vật lý của kho được lưu trữ, sẵn sàng để gán lô hàng vào.		

Bảng 3.36: Đặc tả Usecase: Cấp quyền và Bảo mật POS (Manage POS Auth)

Mã Use case	UC034		
Tên Use case	Thiết lập bảo mật POS		
Tác nhân	Manager, Admin		
Mô tả	Quản lý quyền truy cập vào máy POS của nhân viên thu ngân, khởi tạo hoặc cấp lại mã PIN 4 số.		
Sự kiện kích hoạt	Click "POS Access Management" trong mục Dashboard.		
Tiền điều kiện	Quản lý đã đăng nhập và nhân viên cần cấp quyền đã có tài khoản (Employee Profile) hợp lệ.		
Luồng sự kiện chính (Thành công)	- Quản lý xem danh sách trạng thái truy cập POS của nhân viên. - Quản lý chọn nhân viên cần cấp quyền hoặc reset PIN. - Quản lý nhập mã PIN mới gồm 4-6 chữ số và xác nhận. - Hệ thống băm (hash) mã PIN bằng Bcrypt. - Hệ thống cập nhật bảng pos_auth, reset số lần đăng nhập sai (failed_attempts = 0) và mở khóa tài khoản (nếu đang bị khóa). - Hệ thống hiển thị thông báo thiết lập mã PIN thành công.		
Luồng sự kiện thay thế	3a. Định dạng PIN không hợp lệ (có chứa chữ cái hoặc quá ngắn/dài): Hệ thống cảnh báo "PIN chỉ chứa 4 đến 6 chữ số".		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Mã PIN mới	Có	4 – 6 chữ số
Hậu điều kiện	Nhân viên có thể sử dụng mã PIN mới để đăng nhập nhanh vào thiết bị POS.		

Bảng 3.37: Đặc tả Usecase: Quản lý chi nhánh (Manage Store)

Mã Use case	UC035
Tên Use case	Quản lý chi nhánh cửa hàng
Tác nhân	Super Admin (Chain Owner)
Mô tả	Quản lý các cửa hàng vật lý trong chuỗi. Đây là chức năng gốc để khởi tạo store_id phục vụ cho kiến trúc Multi-tenancy của toàn hệ thống.
Sự kiện kích hoạt	Chọn menu "Stores Management" từ trang quản trị tổng.
Tiền điều kiện	Admin đã đăng nhập với vai trò Super Admin.

Luồng sự kiện chính (Thành công)	
---	--

Bảng 3.38: Đặc tả Usecase: Khách hàng tự đăng ký (Customer Self-Registration)

Mã Use case	UC036
Tên Use case	Khách hàng tự đăng ký trực tuyến
Tác nhân	Khách vãng lai (Guest)
Mô tả	Khách hàng tự tạo tài khoản qua giao diện website (Web store) để mua sắm trực tuyến. Khách hàng thuộc về chuỗi (Chain-level) chứ không cố định ở một store_id cụ thể.
Sự kiện kích hoạt	Chọn "Đăng ký" trên giao diện Web khách hàng.
Tiền điều kiện	• Khách truy cập vào hệ thống website cửa hàng.
Luồng sự kiện chính (Thành công)	1. Khách hàng nhập thông tin: Họ tên, Email, Mật khẩu, Số điện thoại. 2. Hệ thống (Auth Service) mở Transaction cơ sở dữ liệu. 3. Hệ thống kiểm tra tính duy nhất của Email. 4. Hệ thống mã hóa (Hash) mật khẩu bằng Bcrypt. 5. Hệ thống tạo User Account (Role = Customer) và Customer Profile tương ứng. 6. Hệ thống tự động cấp phát JWT Token (Auto-login) và trả về cho trình duyệt. 7. Khách hàng được đăng nhập tự động và chuyển đến trang mua sắm.

Luồng sự kiện thay thế	• 3a. Email đã tồn tại: Hệ thống ROLLBACK transaction, báo lỗi "Email already exists". • 1a. Mật khẩu dưới 6 ký tự: Hệ thống báo lỗi Validation.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Full Name	Có	Họ và tên
	Email	Có	Không được trùng lặp
	Password	Có	Tối thiểu 6 ký tự
Hậu điều kiện	• Tài khoản Customer được tạo thành công trên hệ thống.		

Bảng 3.39: Đặc tả Usecase: Điều chuyển hàng lên kệ (Move Stock to Shelf)

Mã Use case	UC037											
Tên Use case	Điều chuyển hàng lên kệ											
Tác nhân	Manager, Nhân viên kho											
Mô tả	Chuyển số lượng sản phẩm từ trạng thái tồn kho chung (On-Hand) sang trạng thái sẵn sàng bán trên kệ (On-Shelf).											
Sự kiện kích hoạt	Chọn lô hàng và nhấn "Move to Shelf" tại giao diện Inventory.											
Tiền điều kiện	- Nhân viên đã đăng nhập và thuộc đúng store_id. - Lô hàng (Batch) đang có lượng On-Hand > 0.											
Luồng sự kiện chính (Thành công)	- Người dùng chọn lô hàng và vị trí, nhập số lượng cần chuyển (Move Quantity). - Inventory Service bắt đầu Transaction và khóa dòng dữ liệu (Lock FOR UPDATE) của item tồn kho. - Hệ thống kiểm tra số lượng On-Hand có lớn hơn hoặc bằng số lượng yêu cầu chuyển hay không. - Hệ thống thực hiện phép tính: trừ đi quantity_on_hand và cộng vào quantity_on_shelf. - Hệ thống ghi một bản ghi Audit Log (inventory_movement) với movement_type = 'transfer' và lý do 'moved_to_shelf'. - Hệ thống COMMIT giao dịch và báo thành công.											
Luồng sự kiện thay thế	3a. Số lượng chuyển lớn hơn On-Hand: Hệ thống ROLLBACK và báo lỗi "Not enough stock on-hand".											
Dữ liệu đầu vào	Gồm các trường dữ liệu sau: <table><tr><td>Tên trường</td><td>Bắt buộc</td><td>Ràng buộc</td></tr><tr><td>Batch ID</td><td>Có</td><td>Thuộc cửa hàng hiện tại</td></tr><tr><td>Move Quantity</td><td>Có</td><td>Số nguyên dương (> 0)</td></tr></table>			Tên trường	Bắt buộc	Ràng buộc	Batch ID	Có	Thuộc cửa hàng hiện tại	Move Quantity	Có	Số nguyên dương (> 0)
Tên trường	Bắt buộc	Ràng buộc										
Batch ID	Có	Thuộc cửa hàng hiện tại										
Move Quantity	Có	Số nguyên dương (> 0)										
Hậu điều kiện	Hàng hóa được phân bổ lên kệ, sẵn sàng cho POS quét mã bán.											

Bảng 3.40: Đặc tả Usecase: Xem lịch sử biến động giá (View Price History)

Mã Use case	UC038
Tên Use case	Xem lịch sử biến động giá
Tác nhân	Admin, Manager
Mô tả	Tra cứu lịch sử các lần thay đổi giá niêm yết của một sản phẩm trong Catalog (Ai đổi, đổi lúc nào, lý do đổi).
Sự kiện kích hoạt	Chọn "Price History" tại trang chi tiết Sản phẩm.
Tiền điều kiện	Người dùng đã đăng nhập và sản phẩm tồn tại trong Catalog.
Luồng sự kiện chính (Thành công)	- Người dùng chọn xem lịch sử giá của một sản phẩm. - Catalog Service truy vấn bảng product_price_history dựa trên Product ID. - Hệ thống trả về danh sách lịch sử sắp xếp theo thời gian mới nhất (DESC). - Giao diện hiển thị bảng lịch sử gồm: Giá cũ, Giá mới, Lý do đổi, Người đổi, và Thời gian đổi.
Luồng sự kiện thay thế	2a. Sản phẩm chưa từng đổi giá: Hệ thống trả về danh sách rỗng và hiển thị "Chưa có biến động giá".
Dữ liệu đầu vào	Không có (Chỉ cần ID Sản phẩm từ URL).
Hậu điều kiện	Quản trị viên nắm được lộ trình biến động giá của sản phẩm.

Bảng 3.41: Đặc tả Usecase: Hoàn tiền giao dịch (Refund Payment)

Mã Use case	UC039
Tên Use case	Hoàn tiền giao dịch
Tác nhân	Manager
Mô tả	Thực hiện hoàn lại tiền cho một giao dịch đã hoàn tất (Ví dụ: khách hàng trả hàng hoặc hoàn tiền cho đơn nhập hàng bị hủy).
Sự kiện kích hoạt	Chọn "Refund" trên một Payment đang có trạng thái "Completed".
Tiền điều kiện	- Tài khoản quản lý đăng nhập hợp lệ. - Giao dịch thanh toán (Payment) ở trạng thái "Completed".
Luồng sự kiện chính (Thành công)	- Manager chọn yêu cầu Hoàn tiền cho một mã thanh toán. - Hệ thống (Payment Service) kiểm tra trạng thái hiện tại của Payment. - Hệ thống mở Transaction, cập nhật trạng thái Payment hiện tại thành "Refunded". - Hệ thống truy xuất mã tham chiếu (reference_type, reference_id) để cập nhật ngược lại trạng thái thanh toán của Đơn hàng gốc (Sale Order hoặc Purchase Order). - Hệ thống COMMIT giao dịch. - Hệ thống hiển thị thông báo hoàn tiền thành công.

Luồng sự kiện thay thế	2a. Payment không ở trạng thái 'Completed': Hệ thống báo lỗi "Only completed payments can be refunded" và từ chối xử lý.		
Dữ liệu đầu vào	Gồm các trường dữ liệu sau:		
	Tên trường	Bắt buộc	Ràng buộc
	Payment ID	Có	Trạng thái Completed
Hậu điều kiện	Lý do hoàn tiền	Không	Text tự do
	Trạng thái giao dịch chuyển sang hoàn tiền, đối soát tài chính được cập nhật.		

3.4 Thiết kế Cơ sở dữ liệu

3.4.1 Sơ đồ dữ liệu

Hệ thống OCOP được thiết kế theo kiến trúc Microservices kết hợp Multi-tenancy, lưu trữ trên cơ sở dữ liệu PostgreSQL. Hệ thống phân chia thành 8 dịch vụ độc lập nhằm đảm bảo khả năng quản lý và mở rộng.

mermaid-diagram-2026-04-02-165452.png

Hình 3.6: Sơ đồ cơ sở dữ liệu phân tán của hệ thống POSMART

3.4.2 Bảng CSDL

Dưới đây là thiết kế chi tiết cho 10 bảng đầu tiên thuộc các dịch vụ Auth & Identity và Catalog:

1. Bảng store

Bảng 3.42: Bảng store

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
name	TEXT		Không	
address	TEXT		Có	
phone	TEXT		Có	

manager_id	BIGINT	FK	Có	Tham chiếu bảng user_account
is_active	BOOLEAN		Không	Mặc định: true
created_at	TIMESTAMPZ		Có	Mặc định: now()

2. Bảng permission

Bảng 3.43: Bảng permission

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
code	TEXT	Unique	Không	
description	TEXT		Có	

3. Bảng role

Bảng 3.44: Bảng role

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
name	TEXT	Unique	Không	
description	TEXT		Có	

4. Bảng role_permission

Bảng 3.45: Bảng role_permission

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
role_id	BIGINT	PK, FK	Có	Tham chiếu bảng role
permission_id	BIGINT	PK, FK	Có	Tham chiếu bảng permission

5. Bảng user_account

Bảng 3.46: Bảng user_account

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
username	TEXT	Unique	Không	
email	TEXT	Unique	Không	
password_hash	TEXT		Không	
role_id	BIGINT	FK	Có	Tham chiếu bảng role
is_active	BOOLEAN		Không	Mặc định: true
last_login	TIMESTAMPZ		Có	

6. Bảng employee

Bảng 3.47: Bảng employee

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
user_id	BIGINT	PK, FK	Không	Tham chiếu bảng user_account
store_id	BIGINT	FK	Có	Tham chiếu bảng store
full_name	TEXT		Không	
address	TEXT		Có	
phone	TEXT		Có	
gender	TEXT		Có	
dob	DATE		Có	

7. Bảng customer

Bảng 3.48: Bảng customer

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
user_id	BIGINT	PK, FK	Không	Tham chiếu bảng user_account
full_name	TEXT		Không	
phone	TEXT		Có	
gender	TEXT		Có	
dob	DATE		Có	
total_spent	NUMERIC		Có	Mặc định: 0
customer_type	TEXT		Có	

8. Bảng auth_tokens

Bảng 3.49: Bảng auth_tokens

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
user_id	BIGINT	FK	Có	Tham chiếu bảng user_account
token_hash	TEXT		Không	
type	TEXT		Có	REFRESH, PASS-WORD_RESET
expires_at	TIMESTAMP		Không	

9. Bảng pos_auth

Bảng 3.50: Bảng pos_auth

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
user_id	BIGINT	PK, FK	Không	Tham chiếu bảng user_account

pin_hash	TEXT		Không	
failed_attempts	INT		Có	Mặc định: 0
locked_until	TIMESTAMPZ		Có	
is_enabled	BOOLEAN		Có	Mặc định: true
last_login	TIMESTAMPZ		Có	

10. Bảng category

Bảng 3.51: Bảng category

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
name	TEXT		Không	
image_url	TEXT		Có	
description	TEXT		Có	

11. Bảng product

Bảng 3.52: Bảng product

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
category_id	BIGINT	FK	Có	Tham chiếu bảng category
name	TEXT		Không	
image_url	TEXT		Có	
unit_price	NUMERIC		Không	Mặc định: 0
is_active	BOOLEAN		Không	Mặc định: true
vendor	TEXT		Có	

12. Bảng product_price_history

Bảng 3.53: Bảng product_price_history

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
product_id	BIGINT	FK	Có	Tham chiếu bảng product
old_price	NUMERIC		Không	
new_price	NUMERIC		Không	
reason	TEXT		Có	
changed_by	BIGINT		Có	Tham chiếu logic bảng user_account
changed_at	TIMESTAMPZ		Có	Mặc định: now()

13. Bảng sale_order

Bảng 3.54: Bảng sale_order

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
store_id	BIGINT		Không	Tham chiếu logic bảng store
customer_id	BIGINT		Không	Tham chiếu logic bảng customer
created_by	BIGINT		Không	Tham chiếu logic bảng employee
order_date	TIMESTAMPZ		Không	Mặc định: now()
delivery_type	TEXT		Không	Mặc định: 'pickup'
address	TEXT		Có	
shipping_fee	NUMERIC		Không	Mặc định: 0
discount_percentage	NUMERIC		Không	Mặc định: 0
total_amount	NUMERIC		Không	Mặc định: 0
payment_status	TEXT		Không	Mặc định: 'pending'
status	TEXT		Không	Mặc định: 'draft'

14. Bảng sale_order_detail

Bảng 3.55: Bảng sale_order_detail

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
order_id	BIGINT	FK	Có	Tham chiếu bảng sale_order
product_name	TEXT		Không	
batch_id	BIGINT		Không	Tham chiếu logic bảng product_batch
quantity	INT		Không	Mặc định: 1
unit_price	NUMERIC		Không	
total_price	NUMERIC		Không	

15. Bảng security_settings

Bảng 3.56: Bảng security_settings

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	INT	PK	Không	Mặc định: 1
max_failed_attempts	INT		Không	Mặc định: 5
lock_duration_minutes	INT		Không	Mặc định: 30
updated_by	BIGINT		Có	
updated_at	TIMESTAMPZ		Có	Mặc định: now()

16. Bảng sales_settings

Bảng 3.57: Bảng sales_settings

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	INT	PK	Không	Mặc định: 1
auto_promotion_enabled	BOOLEAN		Không	Mặc định: false
promotion_start_time	TIME		Có	
promotion_discount_percent	NUMERIC		Không	Mặc định: 0
discount_retail	NUMERIC		Không	Mặc định: 5
discount_wholesale	NUMERIC		Không	Mặc định: 10
discount_vip	NUMERIC		Không	Mặc định: 15
updated_by	BIGINT		Có	
updated_at	TIMESTAMP TZ		Có	Mặc định: now()

17. Bảng settings_history

Bảng 3.58: Bảng settings_history

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
setting_type	TEXT		Không	
old_value	JSONB		Không	
new_value	JSONB		Không	
changed_by	BIGINT		Có	
change_reason	TEXT		Có	
changed_at	TIMESTAMP TZ		Có	Mặc định: now()

18. Bảng supplier

Bảng 3.59: Bảng supplier

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
company_name	TEXT		Không	
phone	TEXT		Có	
address	TEXT		Có	
account_number	TEXT		Có	
payment_terms	TEXT		Không	Mặc định: 'cod'
credit_limit	NUMERIC		Không	Mặc định: 0
current_debt	NUMERIC		Không	Mặc định: 0
is_active	BOOLEAN		Không	Mặc định: true

19. Bảng purchase_order

Bảng 3.60: Bảng purchase_order

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
------------	--------------	------	------	-------

id	BIGINT	PK	Không	Tự tăng
store_id	BIGINT		Không	Tham chiếu logic bảng store
supplier_id	BIGINT	FK	Không	Tham chiếu bảng supplier
order_date	TIMESTAMPZ		Không	Mặc định: now()
received_date	TIMESTAMPZ		Có	
shipping_fee	NUMERIC		Không	Mặc định: 0
discount_percentage	NUMERIC		Không	Mặc định: 0
total_price	NUMERIC		Không	Mặc định: 0
status	TEXT		Không	Mặc định: 'draft'
payment_status	TEXT		Không	Mặc định: 'unpaid'
created_by	BIGINT		Có	
notes	TEXT		Có	

20. Bảng purchase_order_detail

Bảng 3.61: Bảng purchase_order_detail

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
po_id	BIGINT	FK	Không	Tham chiếu bảng purchase_order
product_id	BIGINT		Không	
product_name	TEXT		Không	
batch_id	BIGINT		Có	
quantity	INT		Không	Mặc định: 1
cost_price	NUMERIC		Không	
total_price	NUMERIC		Không	

21. Bảng product_batch

Bảng 3.62: Bảng product_batch

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
store_id	BIGINT		Không	
product_id	BIGINT		Không	
cost_price	NUMERIC		Không	
unit_price	NUMERIC		Không	
discount_percentage	NUMERIC		Có	Mặc định: 0
quantity	INT		Không	
mfg_date	DATE		Có	
expiry_date	DATE		Có	
status	TEXT		Không	Mặc định: 'active'
notes	TEXT		Có	

22. Bảng warehouse_block

Bảng 3.63: Bảng warehouse_block

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
store_id	BIGINT		Không	
name	TEXT		Không	
rows	INT		Không	
cols	INT		Không	
column_gaps	INT[]		Có	Mặc định: '{}'

23. Bảng location

Bảng 3.64: Bảng location

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
block_id	BIGINT	FK	Không	Tham chiếu bảng warehouse_block
name	TEXT		Không	
position	INT		Không	
max_capacity	INT		Không	Mặc định: 100
is_active	BOOLEAN		Không	Mặc định: true

24. Bảng inventory_item

Bảng 3.65: Bảng inventory_item

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
product_batch_id	BIGINT	FK	Không	Tham chiếu bảng product_batch
location_id	BIGINT	FK	Không	Tham chiếu bảng location
quantity_on_hand	INT		Không	Mặc định: 0
quantity_on_shelf	INT		Không	Mặc định: 0
quantity_reserved	INT		Không	Mặc định: 0

25. Bảng inventory_movement

Bảng 3.66: Bảng inventory_movement

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
inventory_item_id	BIGINT	FK	Không	Tham chiếu bảng inventory_item
movement_type	TEXT		Không	
quantity	INT		Không	

reason	TEXT		Có	
moved_at	TIMESTAMPTZ		Không	Mặc định: now()
performed_by	BIGINT		Có	

26. Bảng stock_out_order

Bảng 3.67: Bảng stock_out_order

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
store_id	BIGINT		Không	
order_date	TIMESTAMPTZ		Không	Mặc định: now()
completed_date	TIMESTAMPTZ		Có	
reason	TEXT		Không	Mặc định: 'sales'
destination	TEXT		Có	
status	TEXT		Không	Mặc định: 'draft'
total_price	NUMERIC		Không	Mặc định: 0
created_by	BIGINT		Không	

27. Bảng stock_out_detail

Bảng 3.68: Bảng stock_out_detail

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
so_id	BIGINT	FK	Không	Tham chiếu bảng stock_out_order
batch_id	BIGINT	FK	Không	Tham chiếu bảng product_batch
quantity	INT		Không	
unit_price	NUMERIC		Không	Mặc định: 0
total_price	NUMERIC		Không	Mặc định: 0

28. Bảng payment

Bảng 3.69: Bảng payment

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
store_id	BIGINT		Không	
amount	NUMERIC		Không	
method	TEXT		Không	
status	TEXT		Không	Mặc định: 'pending'
reference_type	TEXT		Không	
reference_id	BIGINT		Không	

payment_date	TIMESTAMPZ		Không	Mặc định: now()
created_by	BIGINT		Có	
notes	TEXT		Có	

29. Bảng vnipay_transaction

Bảng 3.70: Bảng vnipay_transaction

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
payment_id	BIGINT	FK	Có	Tham chiếu bảng payment
reference_id	BIGINT		Không	
vnp_txn_ref	TEXT	Unique	Không	
vnp_transaction_no	TEXT		Có	
vnp_amount	BIGINT		Không	
vnp_response_code	TEXT		Có	
vnp_transaction_status	TEXT		Có	
vnp_bank_code	TEXT		Có	
vnp_bank_tran_no	TEXT		Có	
vnp_card_type	TEXT		Có	
vnp_pay_date	TEXT		Có	
vnp_order_info	TEXT		Có	
vnp_ip_addr	TEXT		Có	
vnp_locale	TEXT		Có	
vnp_secure_hash	TEXT		Có	
status	TEXT		Không	Mặc định: 'pending'
payment_url	TEXT		Có	
ipn_verified	BOOLEAN		Không	Mặc định: false
return_url_accessed	BOOLEAN		Không	Mặc định: false

30. Bảng chat_session

Bảng 3.71: Bảng chat_session

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
user_id	BIGINT		Không	Tham chiếu logic bảng user_account
user_type	TEXT		Không	Mặc định: 'customer' (customer, employee)
store_id	BIGINT		Có	Tham chiếu logic bảng store
started_at	TIMESTAMPZ		Không	Mặc định: now()
ended_at	TIMESTAMPZ		Có	
is_active	BOOLEAN		Không	Mặc định: true

31. Bảng chat_message

Bảng 3.72: Bảng chat_message

Tên trường	Kiểu dữ liệu	Khóa	Null	Mô tả
id	BIGINT	PK	Không	Tự tăng
session_id	BIGINT	FK	Không	Tham chiếu bảng chat_session
role	TEXT		Không	user, assistant, system
content	TEXT		Không	
intent	TEXT		Có	Phân loại ý định: tra cứu giá, kiểm tra đơn hàng...
metadata	JSONB		Có	Lưu ngữ cảnh AI
created_at	TIMESTAMPTZ		Không	Mặc định: now()

3.5 Thiết kế Giao diện

3.5.1 Danh sách các giao diện

Giao diện Khách hàng (Customer Storefront)

Vai trò sử dụng: Khách hàng (Customer)

Mô tả chức năng: Giao diện mua sắm trực tuyến dành cho khách hàng, bao gồm duyệt sản phẩm, tìm kiếm, đặt hàng, thanh toán và theo dõi đơn hàng cá nhân.

Giao diện Đăng nhập / Đăng ký

Vai trò sử dụng: Tất cả người dùng (Admin, Manager, Employee, Customer)

Mô tả chức năng: Xác thực danh tính người dùng. Admin và Manager đăng nhập qua email/mật khẩu. Employee đăng nhập POS nhanh bằng mã PIN 4-6 số.

Giao diện Quản trị hệ thống (Super Admin)

Vai trò sử dụng: Quản trị viên cấp cao (Super Admin / Chain Owner)

Mô tả chức năng: Quản lý toàn bộ chuỗi cửa hàng, cấu hình hệ thống, phân quyền tài khoản và kiểm soát cấu hình bảo mật/bán hàng.

Giao diện Quản lý cửa hàng (Manager Dashboard)

Vai trò sử dụng: Quản lý cửa hàng (Manager)

Mô tả chức năng: Quản lý toàn bộ hoạt động kinh doanh của chi nhánh: tồn kho, nhập hàng, đơn hàng, khách hàng, nhà cung cấp và báo cáo doanh thu.

Giao diện Bán hàng POS (Sales Terminal)

Vai trò sử dụng: Nhân viên bán hàng / Thu ngân (Employee / Sales)

Mô tả chức năng: Giao diện điểm bán hàng tại quầy, hỗ trợ tạo đơn hàng nhanh, quản lý đơn chờ, thanh toán đa phương thức và xem lịch sử hóa đơn.

Giao diện AI Chatbot

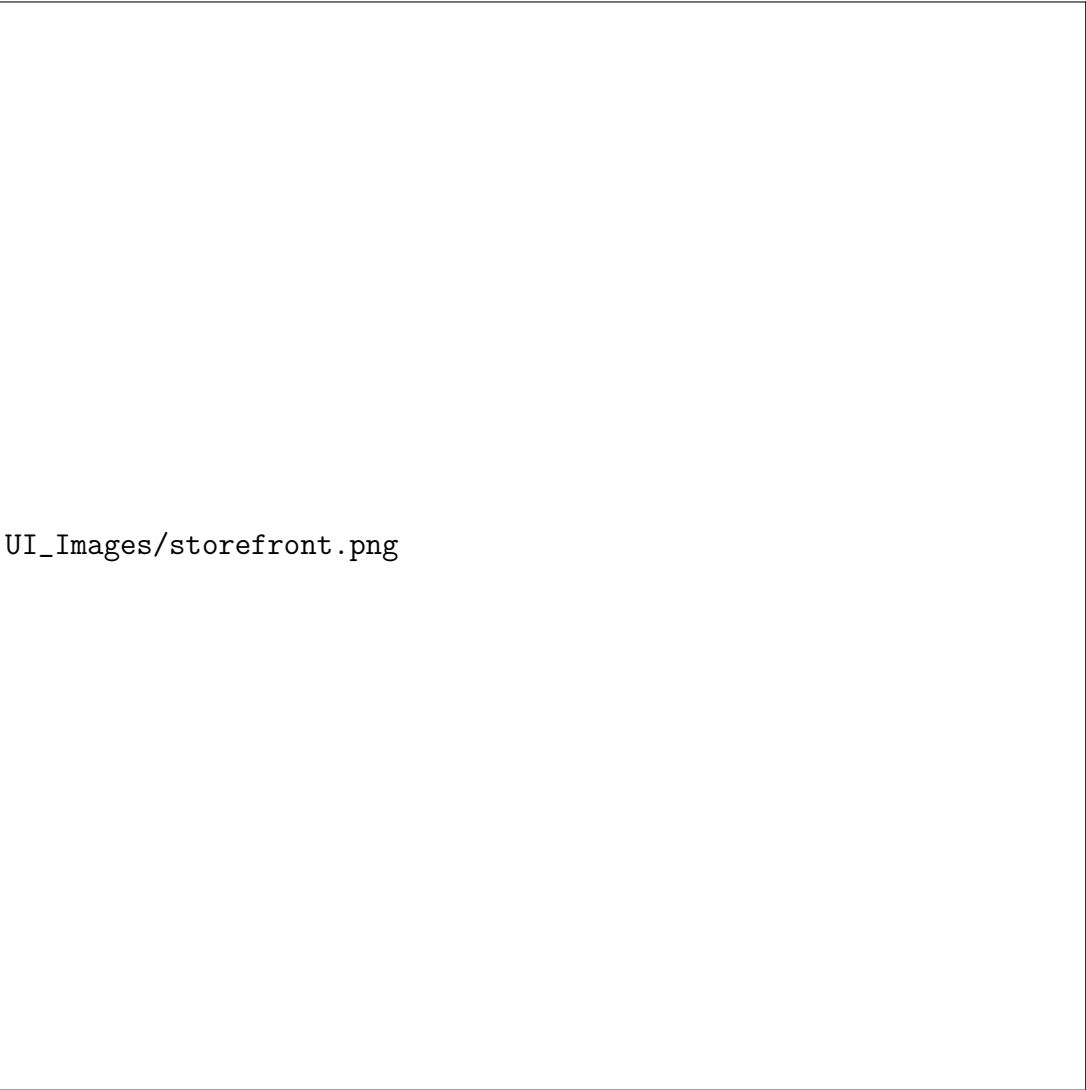
Vai trò sử dụng: Khách hàng, Nhân viên, Quản lý

Mô tả chức năng: Trợ lý ảo tích hợp trên hệ thống, hỗ trợ tra cứu thông tin sản phẩm, tồn kho, giá cả và lịch sử đơn hàng qua ngôn ngữ tự nhiên.

3.5.2 Chi tiết các giao diện

Giao diện Khách hàng (Customer Storefront)

UI-01: Trang chủ Storefront Trang chủ là điểm tiếp cận đầu tiên của khách hàng, được thiết kế theo bố cục hiện đại với ba khu vực chính: Hero Banner giới thiệu khuyến mãi nổi bật, lưới danh mục sản phẩm (Category Grid) hiển thị toàn bộ ngành hàng với hình ảnh trực quan, và khu vực sản phẩm nổi bật (Featured Products) hiển thị các mặt hàng bán chạy.



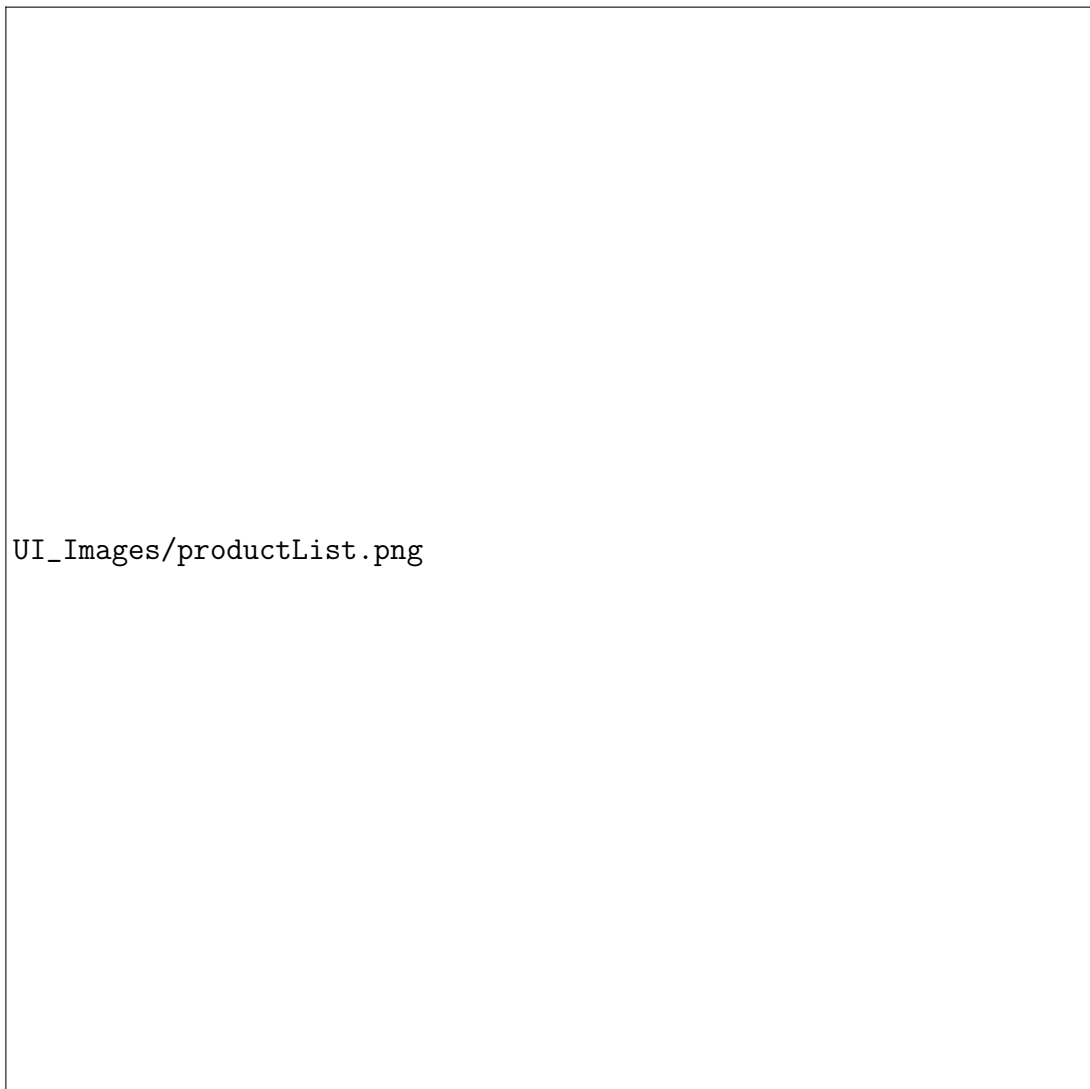
Hình 3.7: Giao diện Trang chủ Storefront (UI-01)

UI-02: Danh sách sản phẩm Trang danh sách sản phẩm hỗ trợ lọc theo danh mục, sắp xếp theo giá/đánh giá và tìm kiếm theo từ khóa. Mỗi thẻ sản phẩm (ProductCard) hiển thị đầy đủ:

STT	Tên giao diện	Vai trò	Mã màn hình
1	Trang chủ Storefront	Customer	UI-01
2	Danh sách sản phẩm	Customer	UI-02
3	Chi tiết sản phẩm	Customer	UI-03
4	Giỏ hàng	Customer	UI-04
5	Thanh toán đơn hàng	Customer	UI-05
6	Lịch sử đơn hàng	Customer	UI-06
7	Đăng nhập (Email/Password)	Admin, Manager	UI-07
8	Đăng nhập POS (PIN)	Employee	UI-08
9	Đăng ký tài khoản	Customer	UI-09
10	Dashboard Admin	Super Admin	UI-10
11	Quản lý chi nhánh	Super Admin	UI-11
12	Quản lý tài khoản	Super Admin	UI-12
13	Quản lý phân quyền (Role)	Super Admin	UI-13
14	Cấu hình hệ thống	Super Admin	UI-14
15	Dashboard Manager	Manager	UI-15
16	Quản lý danh mục	Manager	UI-16
17	Quản lý sản phẩm	Manager	UI-17
18	Quản lý nhà cung cấp	Manager	UI-18
19	Quản lý đơn nhập hàng	Manager	UI-19
20	Nhập kho hàng hóa	Manager	UI-20
21	Quản lý tồn kho	Manager	UI-21
22	Sơ đồ kho bãi	Manager	UI-22
23	Quản lý lô hàng	Manager	UI-23
24	Phiếu xuất kho	Manager	UI-24
25	Quản lý khách hàng	Manager	UI-25
26	Báo cáo doanh thu	Manager	UI-26
27	Màn hình POS chính	Employee	UI-27
28	Quản lý đơn hàng chờ	Employee	UI-28
29	Thanh toán VNPay (QR)	Employee	UI-29
30	AI Chatbot	Customer, Employee, Manager	UI-30

Bảng 3.73: Danh sách các giao diện của hệ thống POSMART

hình ảnh, tên, giá hiện tại, giá gốc (nếu đang giảm giá), đánh giá sao và nhãn trạng thái (Mới, Giảm giá, Hữu cơ).



Hình 3.8: Giao diện Danh sách sản phẩm (UI-02)

UI-03: Chi tiết sản phẩm Trang chi tiết hiển thị toàn bộ thông tin sản phẩm: bộ ảnh đa chiều, mô tả chi tiết, giá, tồn kho hiện tại, và nút thêm vào giỏ hàng. Phần dưới trang tích hợp khu vực Chatbot gợi ý sản phẩm liên quan dựa trên thuật toán Apriori.

UI-04: Giỏ hàng Giao diện giỏ hàng cho phép khách hàng xem lại các sản phẩm đã chọn, điều chỉnh số lượng hoặc xóa từng mặt hàng. Hệ thống tự động tính tổng tiền, phí vận chuyển và áp dụng chiết khấu theo loại khách hàng (VIP/Wholesale/Retail) từ cấu hình Sales Settings.

UI-05: Thanh toán đơn hàng Trang thanh toán cho phép khách hàng nhập địa chỉ giao hàng, chọn phương thức thanh toán (tiền mặt hoặc VNPay) và xác nhận đơn. Khi chọn VNPay, hệ thống hiển thị mã QR để quét thanh toán.



Hình 3.9: Giao diện Chi tiết sản phẩm (UI-03)



UI_Images/cart.png

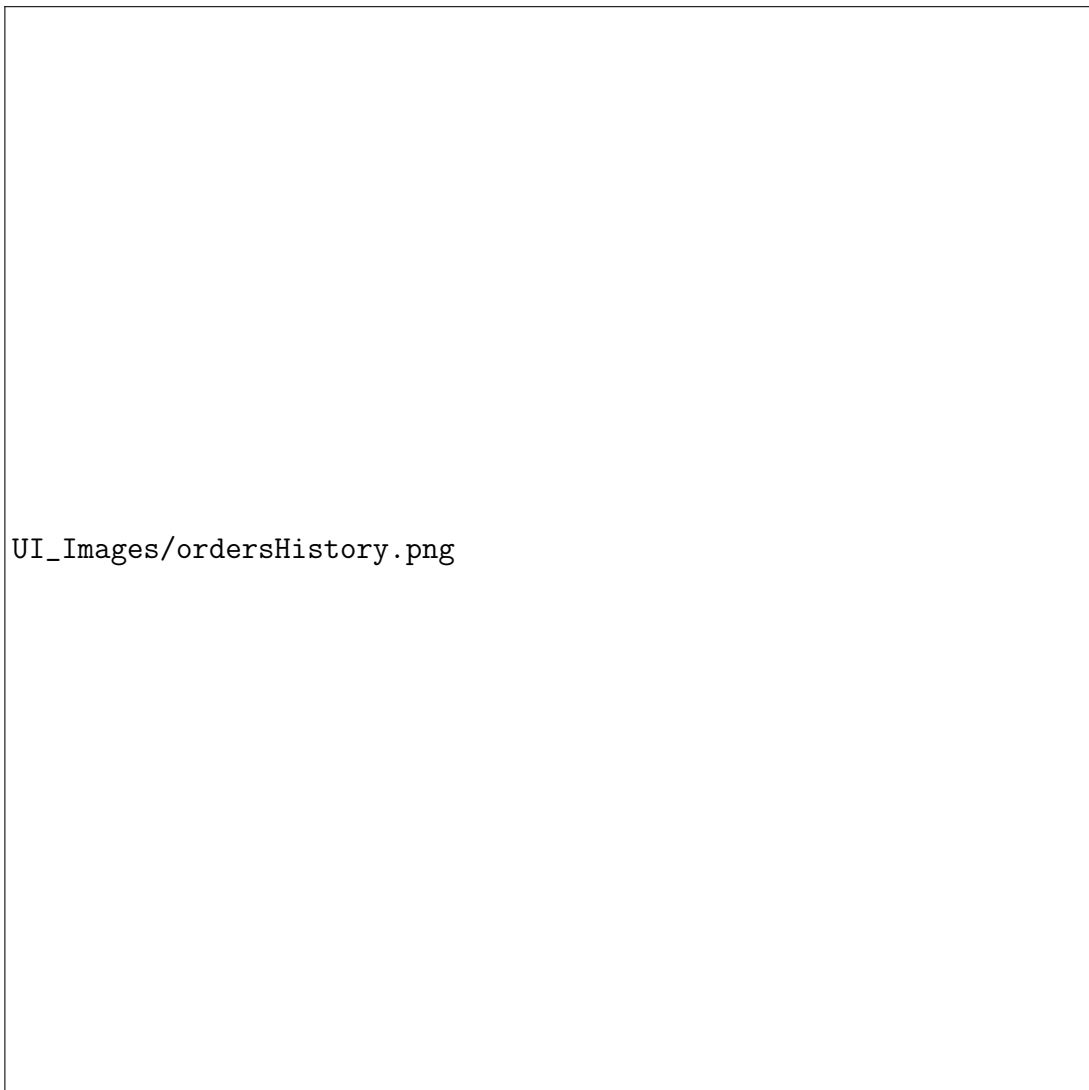
Hình 3.10: Giao diện Giỏ hàng (UI-04)



UI_Images/checkout.png

Hình 3.11: Giao diện Thanh toán đơn hàng (UI-05)

UI-06: Lịch sử đơn hàng Trang lịch sử cho phép khách hàng tra cứu toàn bộ đơn hàng đã đặt, xem chi tiết từng đơn (sản phẩm, số lượng, trạng thái giao hàng, phương thức thanh toán). Giao diện phân trang và hỗ trợ lọc theo khoảng thời gian và trạng thái.



Hình 3.12: Giao diện Lịch sử đơn hàng (UI-06)

Giao diện Xác thực

UI-07: Đăng nhập (Email / Password) Giao diện đăng nhập dành cho Admin và Manager sử dụng form nhập email và mật khẩu tiêu chuẩn. Hệ thống hiển thị thông báo lỗi rõ ràng khi thông tin không hợp lệ và áp dụng cơ chế khóa tài khoản tạm thời khi đăng nhập sai quá số lần quy định (cấu hình tại Security Settings).

UI-08: Đăng nhập POS nhanh (PIN) Giao diện đăng nhập tối giản dành cho nhân viên thu ngân tại quầy POS. Nhân viên nhập mã nhân viên và mã PIN 4-6 chữ số để truy cập nhanh vào màn hình bán hàng. Giao diện được tối ưu cho màn hình cảm ứng, các nút số có kích thước lớn, dễ thao tác.



Hình 3.13: Giao diện Đăng nhập Email/Password (UI-07)



UI_Images/POSLogin.png

Hình 3.14: Giao diện Đăng nhập POS nhanh (UI-08)

UI-09: Đăng ký tài khoản Giao diện cho phép khách hàng tự tạo tài khoản mới với các trường: Họ tên, Email, Mật khẩu và Số điện thoại. Hệ thống kiểm tra tính duy nhất của email, mã hóa mật khẩu bằng Bcrypt và cấp JWT Token tự động sau khi đăng ký thành công (Auto-login).



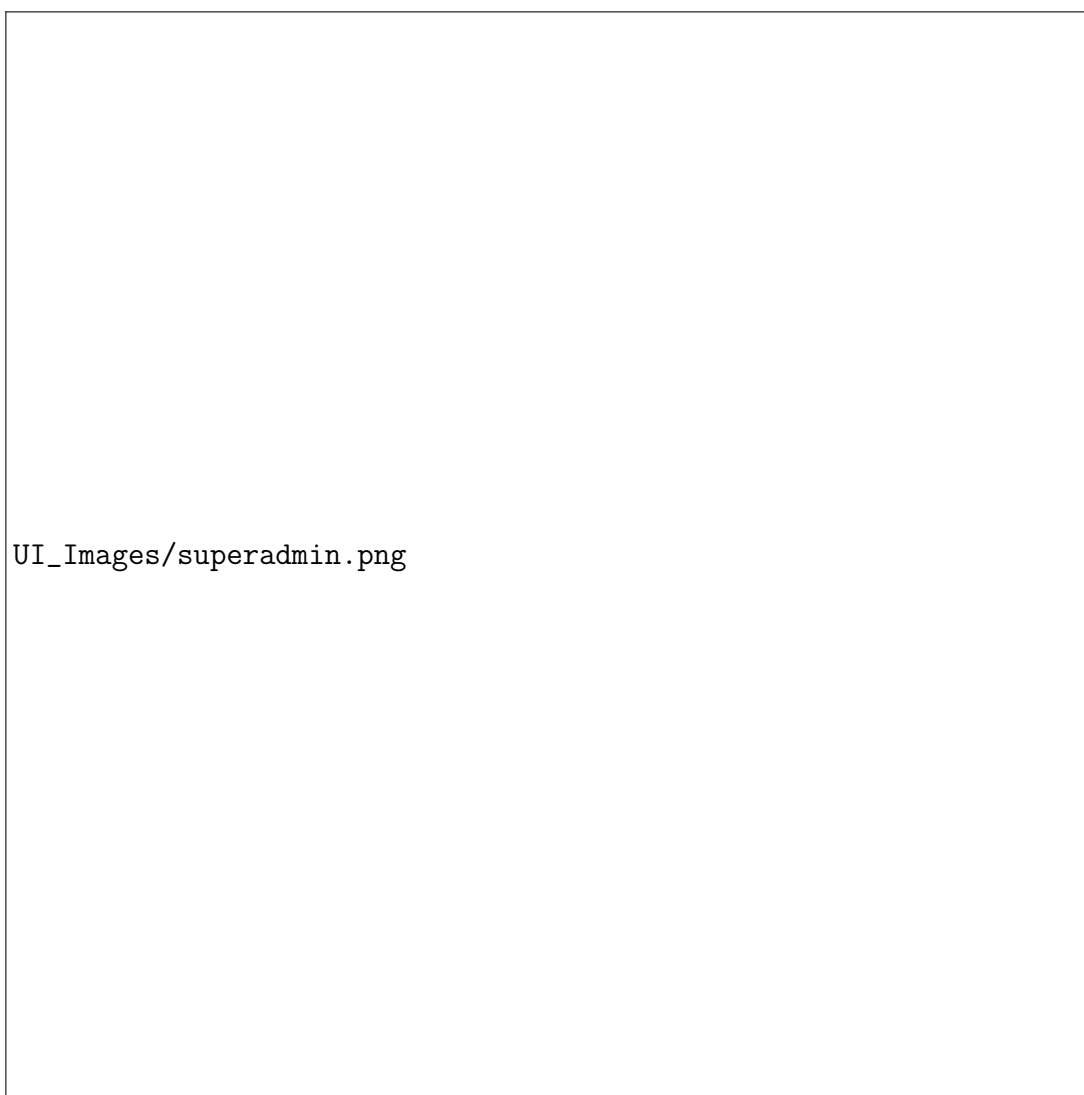
Hình 3.15: Giao diện Đăng ký tài khoản (UI-09)

Giao diện Quản trị hệ thống (Super Admin)

UI-10: Dashboard Admin Trang tổng quan của Super Admin hiển thị các chỉ số toàn chuỗi: tổng số chi nhánh đang hoạt động, tổng số tài khoản theo từng Role, và biểu đồ thống kê tóm tắt. Sidebar điều hướng bao gồm các module: Stores, Accounts, Roles, Settings và Audit Log.

UI-11: Quản lý chi nhánh (Store Management) Giao diện danh sách các chi nhánh trong chuỗi với thông tin: tên, địa chỉ, số điện thoại, quản lý phụ trách và trạng thái hoạt động. Admin có thể thêm chi nhánh mới (hệ thống tự sinh `store_id`), chỉnh sửa thông tin hoặc vô hiệu hóa chi nhánh.

UI-12: Quản lý tài khoản (Account Management) Danh sách toàn bộ tài khoản trong hệ thống với bộ lọc theo Role và chi nhánh. Admin có thể tạo tài khoản mới (gán Role và chi nhánh), chỉnh sửa thông tin, kích hoạt/vô hiệu hóa tài khoản và đặt lại mật khẩu.



UI_Images/superadmin.png

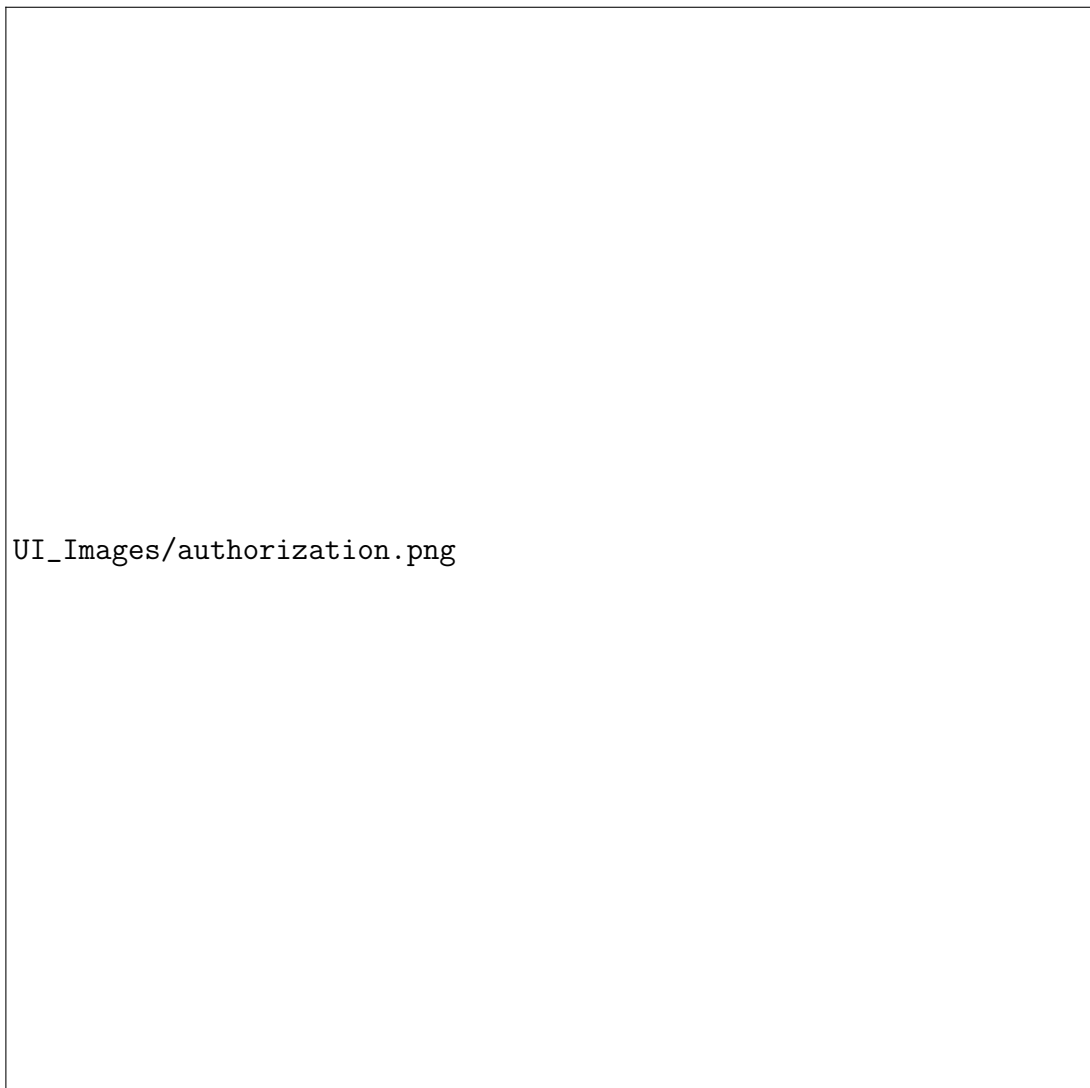
Hình 3.16: Giao diện Dashboard Super Admin (UI-10)



UI_Images/accountManager.png

Hình 3.17: Giao diện Quản lý tài khoản (UI-12)

UI-13: Quản lý phân quyền (Role Management) Giao diện quản lý các Role trong hệ thống (Super Admin, Manager, Employee, Customer). Admin có thể tạo Role mới, chỉnh sửa danh sách quyền (Permissions) gắn với mỗi Role và xóa Role không còn sử dụng.



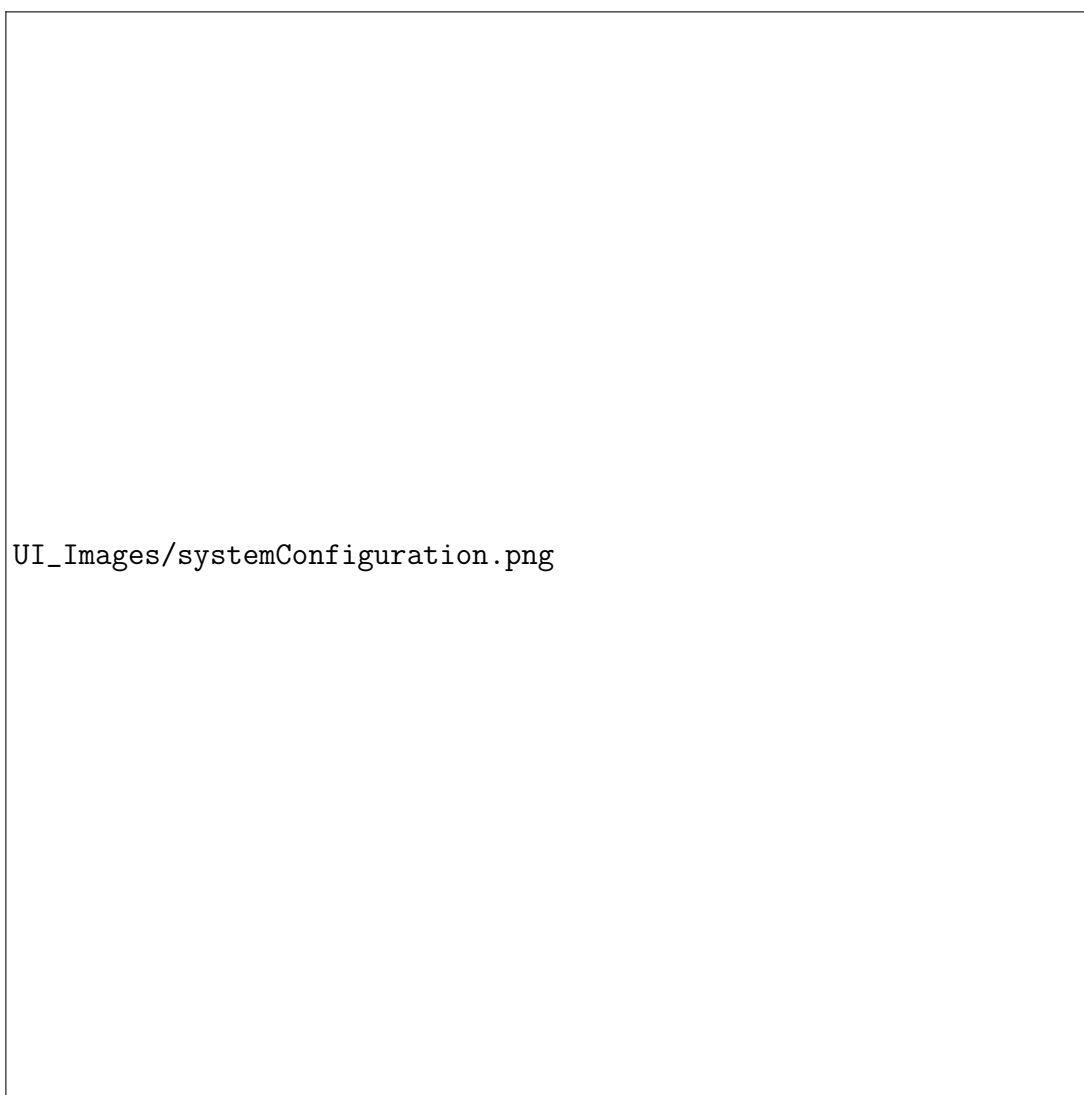
Hình 3.18: Giao diện Quản lý phân quyền (UI-13)

UI-14: Cấu hình hệ thống (System Settings) Giao diện hai tab: **Security Settings** (số lần đăng nhập sai tối đa, thời gian khóa tài khoản) và **Sales Settings** (phần trăm chiết khấu theo loại khách VIP/Wholesale/Retail, bật/tắt khuyến mãi tự động). Mỗi lần lưu yêu cầu nhập “Lý do thay đổi” để ghi vào Audit Trail.

Giao diện Quản lý cửa hàng (Manager Dashboard)

UI-15: Dashboard Manager Trang tổng quan của Manager hiển thị các chỉ số kinh doanh của chi nhánh: doanh thu hôm nay, số đơn hàng, tổng tồn kho và biểu đồ doanh thu theo tuần/tháng/năm. Sidebar điều hướng đầy đủ đến tất cả module quản lý.

UI-16: Quản lý danh mục sản phẩm Danh sách các danh mục sản phẩm với hình ảnh đại diện và số lượng sản phẩm thuộc mỗi danh mục. Manager có thể thêm danh mục mới, chỉnh sửa

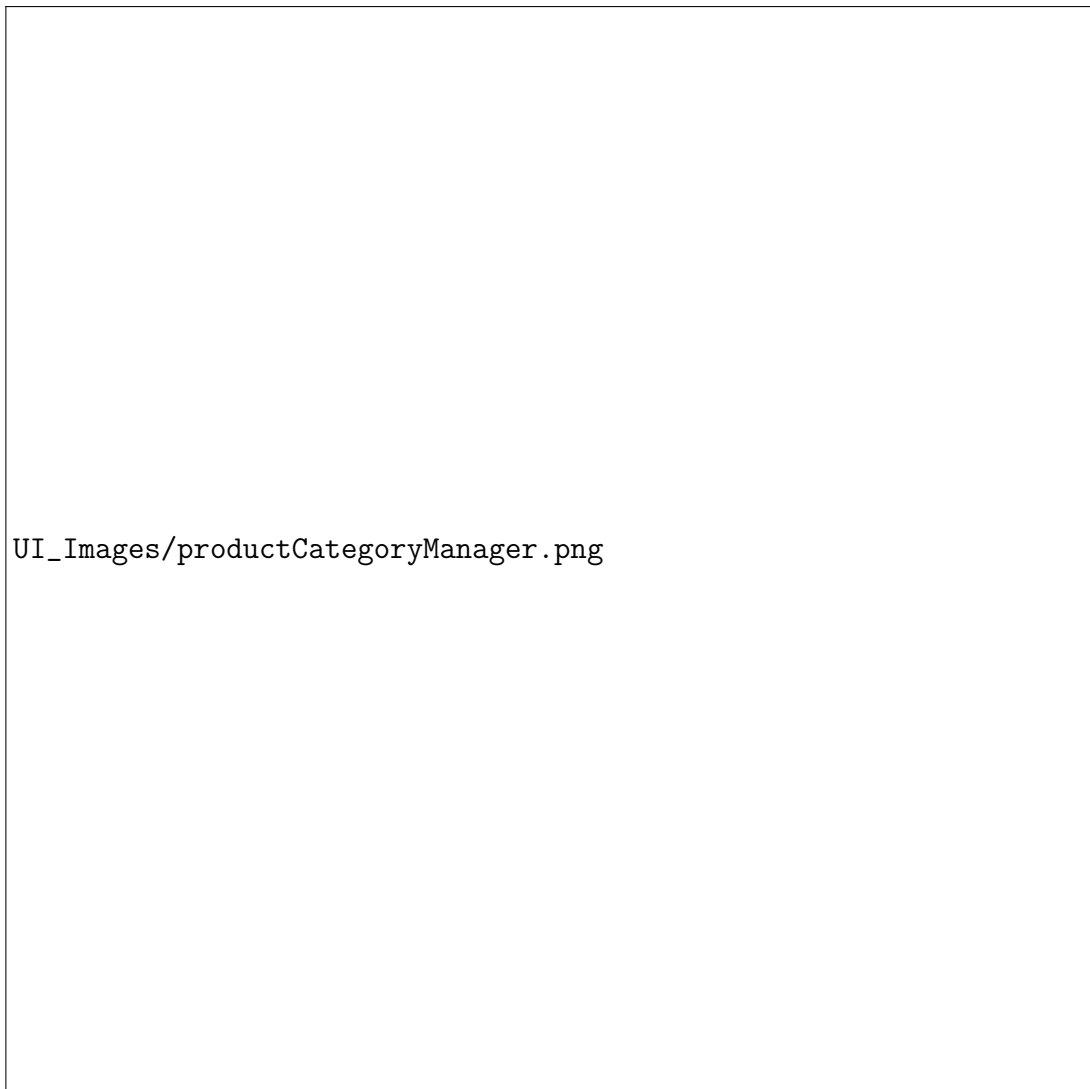


Hình 3.19: Giao diện Cấu hình hệ thống (UI-14)



Hình 3.20: Giao diện Dashboard Manager (UI-15)

tên/hình ảnh và xóa danh mục không còn sử dụng. Thanh tìm kiếm hỗ trợ lọc theo tên.



UI_Images/productCategoryManager.png

Hình 3.21: Giao diện Quản lý danh mục sản phẩm (UI-16)

UI-17: Quản lý sản phẩm (Catalog) Danh sách sản phẩm toàn chuỗi với các thao tác CRUD đầy đủ. Form thêm/sửa sản phẩm bao gồm: tên, danh mục, giá niêm yết, hình ảnh, nhà cung cấp và trạng thái. Giao diện tích hợp xem lịch sử biến động giá (Price History) để theo dõi các lần thay đổi giá kèm lý do.

UI-18: Quản lý nhà cung cấp Danh sách nhà cung cấp với thông tin: tên công ty, số điện thoại, địa chỉ, số tài khoản ngân hàng, hạn mức tín dụng và công nợ hiện tại. Giao diện hiển thị trực quan các nhà cung cấp đang có công nợ để Manager theo dõi và thanh toán kịp thời.

UI-19: Quản lý đơn nhập hàng (Purchase Order) Danh sách đơn nhập hàng được phân nhóm theo trạng thái: Draft, Approved, Received. Giao diện tạo đơn mới cho phép chọn nhà cung cấp, thêm nhiều sản phẩm với số lượng và đơn giá. Sau khi duyệt (Approve), hệ thống tự động cộng công nợ vào nhà cung cấp.

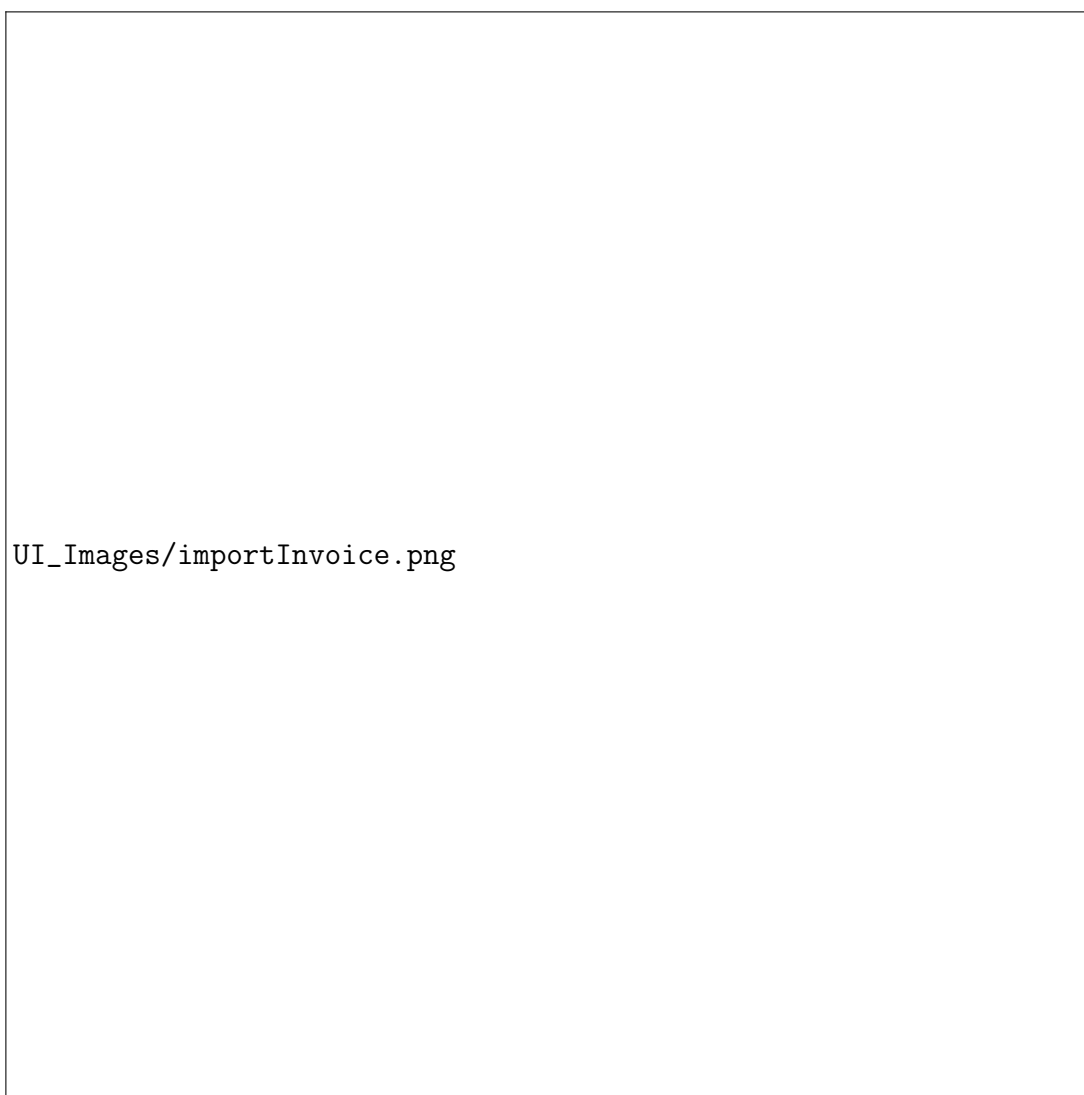


Hình 3.22: Giao diện Quản lý sản phẩm (UI-17)



UI_Images/supplierManager.png

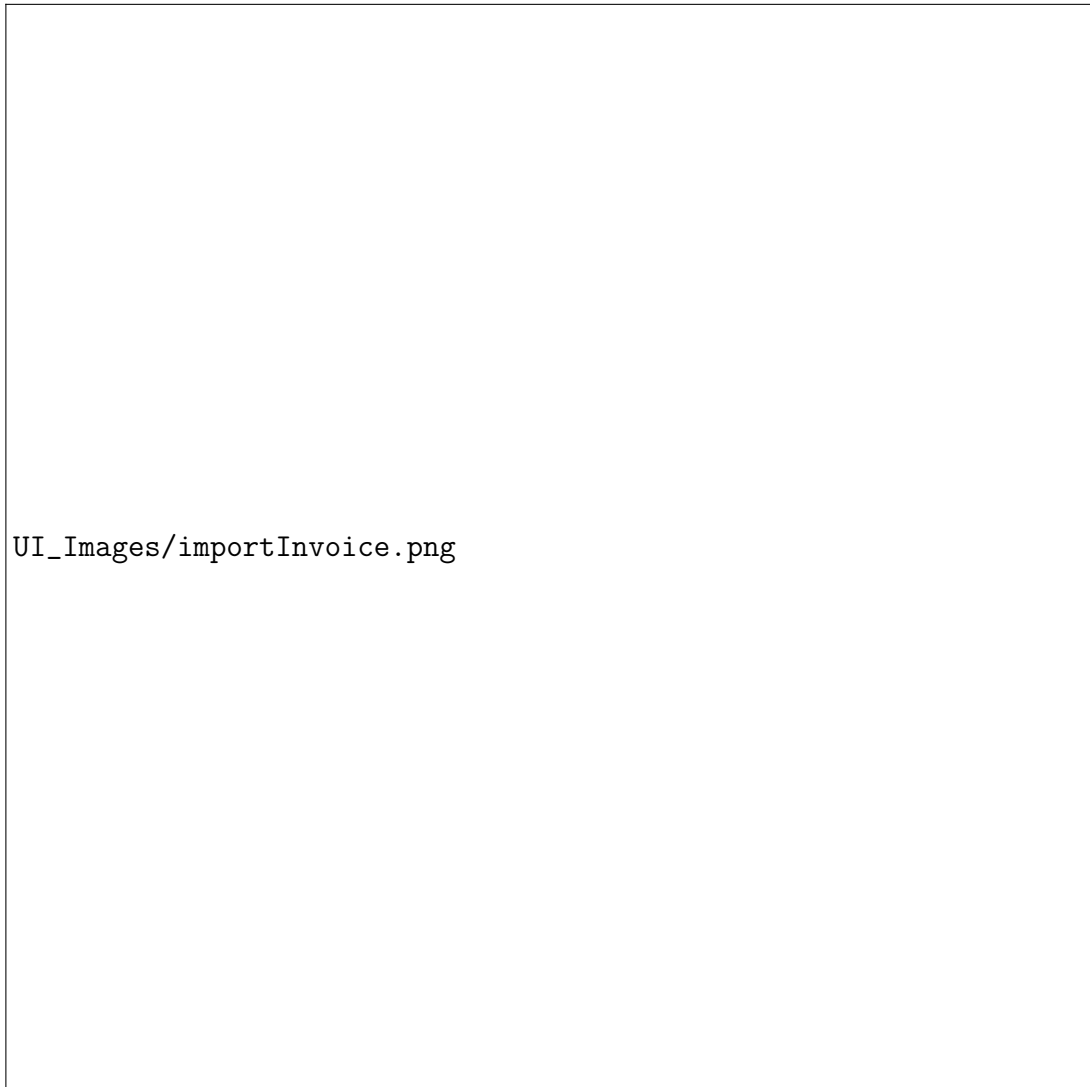
Hình 3.23: Giao diện Quản lý nhà cung cấp (UI-18)



UI_Images/importInvoice.png

Hình 3.24: Giao diện Quản lý đơn nhập hàng (UI-19)

UI-20: Nhập kho hàng hóa (Receive Stock) Giao diện xác nhận nhập kho cho từng dòng sản phẩm trong đơn đã được duyệt. Manager nhập ngày sản xuất, ngày hết hạn và chọn vị trí kệ (Location) cho từng lô. Hệ thống tự động tạo lô hàng (Product Batch) và cộng số lượng vào tồn kho thực tế.



Hình 3.25: Giao diện Nhập kho hàng hóa (UI-20)

UI-21: Quản lý tồn kho (Inventory) Trang tổng hợp tồn kho hiển thị từng sản phẩm với ba chỉ số: On-Hand (trong kho), On-Shelf (trên kệ) và Reserved (đã đặt). Manager có thể chuyển hàng lên kệ (Move to Shelf), điều chỉnh tồn kho (Adjust Stock) và xem lịch sử biến động (Movement History).

UI-22: Sơ đồ kho bãi (Warehouse Map) Giao diện trực quan hóa sơ đồ kho theo từng Block (khu vực) và Location (ô/kệ). Mỗi ô được tô màu theo tỷ lệ lấp đầy: xanh lá (trống), vàng (sắp đầy $\geq 70\%$) và đỏ (đầy). Manager có thể tạo Block mới bằng cách nhập tên, số hàng và số cột.

UI-23: Quản lý lô hàng (Batch Management) Danh sách lô hàng của từng sản phẩm với thông tin: ngày nhập, ngày hết hạn, số lượng hiện tại và trạng thái (Active/Expired). Manager có



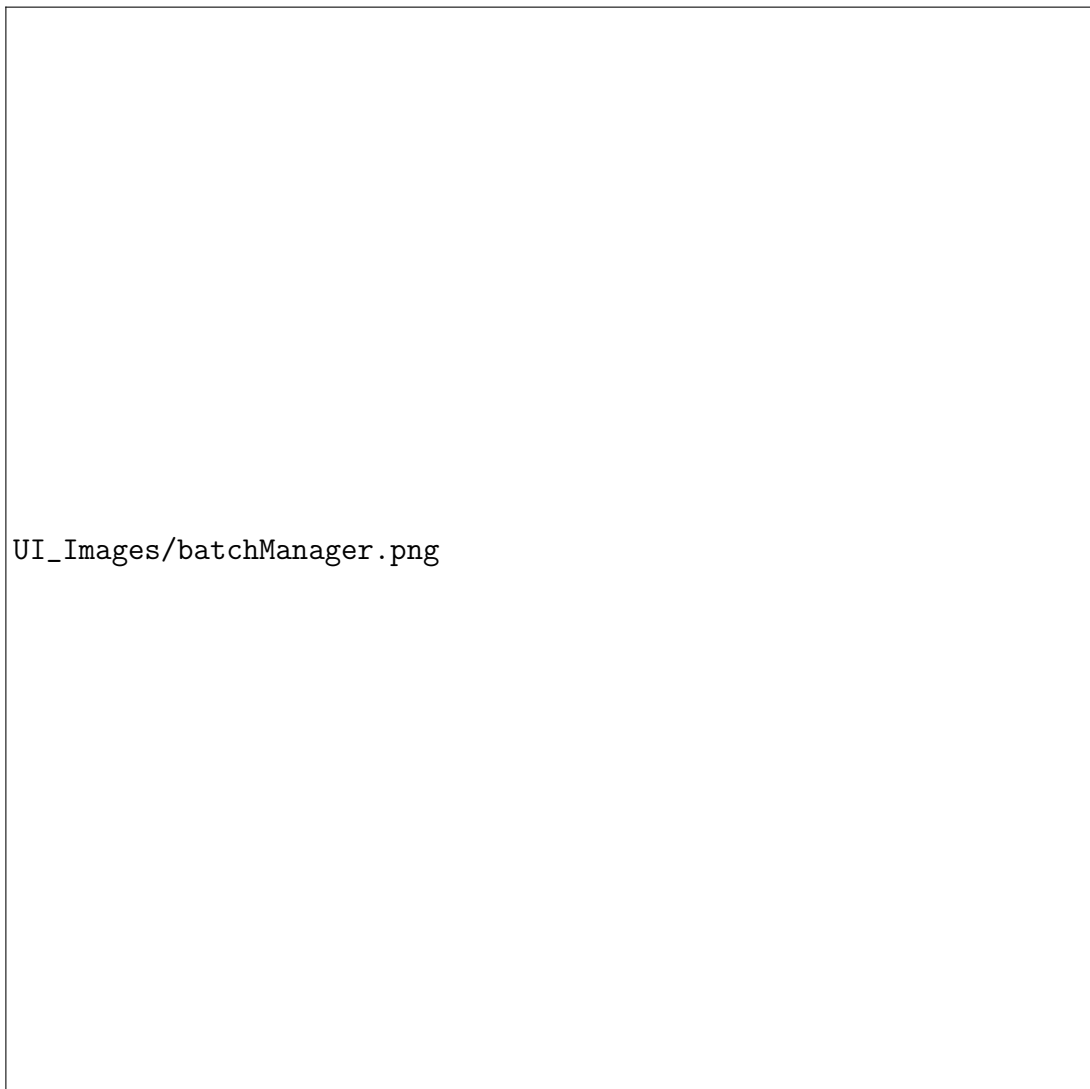
UI_Images/repositoryManager.png

Hình 3.26: Giao diện Quản lý tồn kho (UI-21)



Hình 3.27: Giao diện Sơ đồ kho bãi (UI-22)

thể chỉnh sửa số lượng lô (Adjust Batch), vô hiệu hóa lô hết hạn và xem lịch sử hủy (Disposal History).



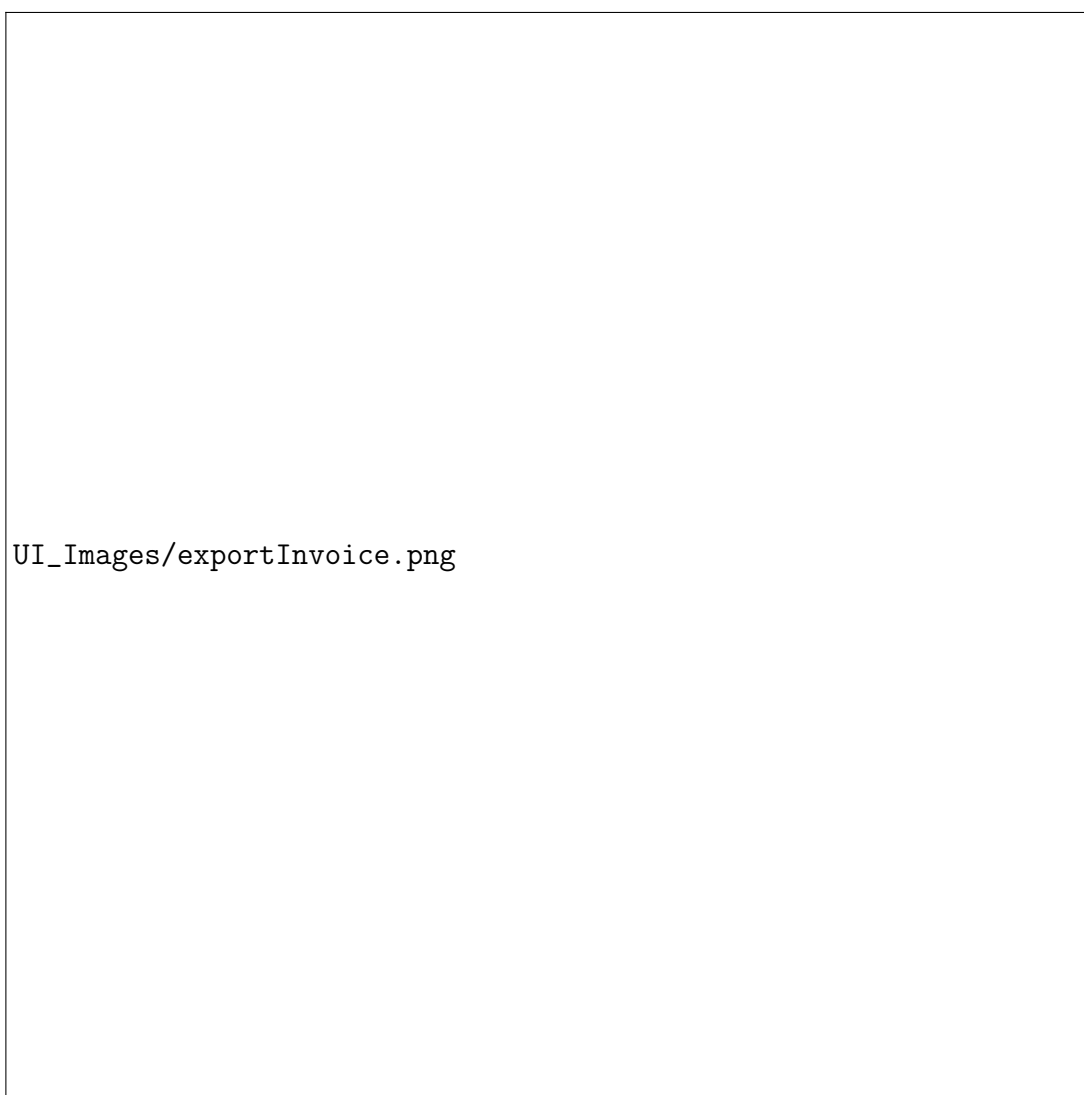
UI_Images/batchManager.png

Hình 3.28: Giao diện Quản lý lô hàng (UI-23)

UI-24: Phiếu xuất kho (Stock Out) Giao diện tạo phiếu xuất kho cho các mục đích ngoài bán lẻ: hủy hàng hết hạn, trả nhà cung cấp hoặc hao hụt. Manager chọn lý do xuất, chọn lô hàng và nhập số lượng. Hệ thống kiểm tra tồn kho thực tế và từ chối nếu số lượng xuất vượt quá tồn.

UI-25: Quản lý khách hàng Danh sách khách hàng của chuỗi với tổng chi tiêu (Total Spent) và phân loại (Retail/Wholesale/VIP). Manager có thể thêm khách hàng mới, chỉnh sửa thông tin và xem lịch sử mua hàng. Bộ lọc nhanh theo loại khách hàng giúp xác định nhóm khách VIP.

UI-26: Báo cáo doanh thu Trang báo cáo tổng hợp với bộ lọc thời gian linh hoạt (Hôm nay, Tuần này, Tháng này, Năm nay). Giao diện hiển thị các chỉ số: Tổng doanh thu, Tổng chi phí, Lợi nhuận gộp, Số đơn hàng, Số lượng sản phẩm bán ra – kèm biểu đồ cột so sánh theo ngày/tuần/tháng.



Hình 3.29: Giao diện Phiếu xuất kho (UI-24)



UI_Images/customerManager.png

Hình 3.30: Giao diện Quản lý khách hàng (UI-25)



UI_Images/profit.png

Hình 3.31: Giao diện Báo cáo doanh thu (UI-26)

Giao diện Bán hàng POS

UI-27: Màn hình POS chính Giao diện bán hàng tại quầy được thiết kế theo bố cục hai cột: bên trái là danh sách sản phẩm với bộ lọc theo danh mục và tìm kiếm theo tên/mã vạch; bên phải là giỏ hàng hiện tại hiển thị từng dòng sản phẩm, số lượng, đơn giá và tổng tiền. Nút điều hướng chính gồm: Thanh toán, Lưu đơn chờ và Hủy đơn.



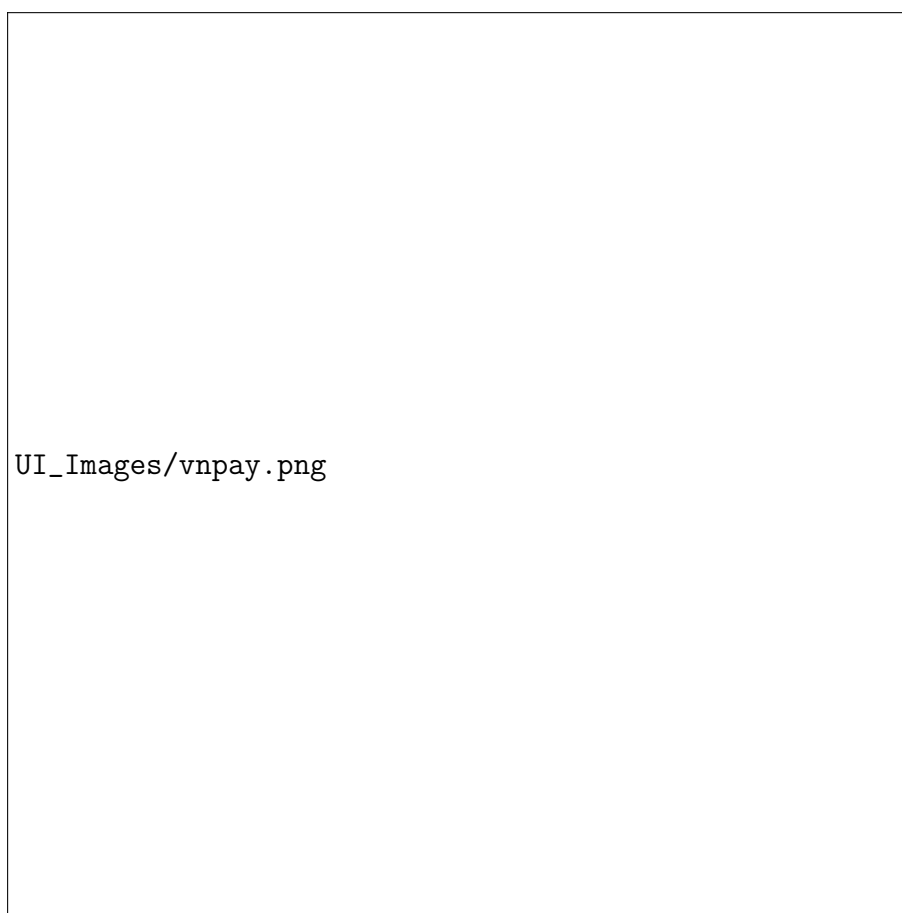
Hình 3.32: Giao diện Màn hình POS chính (UI-27)

UI-28: Quản lý đơn hàng chờ (Hold Orders) Danh sách các đơn hàng đang tạm dừng (Hold), hiển thị tên khách hàng (nếu có), số lượng sản phẩm và thời điểm lưu. Nhân viên có thể chọn tải lại đơn để tiếp tục thanh toán hoặc hủy đơn chờ. Chức năng này cho phép phục vụ nhiều khách hàng cùng lúc mà không mất dữ liệu.

UI-29: Thanh toán VNPay (QR Code) Khi nhân viên chọn phương thức thanh toán chuyển khoản, hệ thống hiển thị mã QR Code được tạo từ API VNPay kèm thông tin đơn hàng (số tiền, mã đơn). Màn hình tự động cập nhật trạng thái khi nhận được xác nhận thanh toán từ Webhook VNPay, không cần thao tác thủ công.



Hình 3.33: Giao diện Quản lý đơn hàng chờ (UI-28)



UI_Images/vnpay.png

Hình 3.34: Giao diện Thanh toán VNPay QR Code (UI-29)

Giao diện AI Chatbot

UI-30: AI Chatbot Assistant Giao diện chat được tích hợp dưới dạng panel thu gọn (bubble) ở góc phải màn hình, có thể mở rộng thành cửa sổ chat đầy đủ. Lịch sử hội thoại được hiển thị theo dạng tin nhắn xen kẽ (người dùng bên phải, chatbot bên trái). Khi chatbot gợi ý sản phẩm, hệ thống hiển thị ProductCard nhỏ gọn kèm giá, tồn kho và nút “Thêm vào giỏ” trực tiếp trong khung chat.

Giao diện phân biệt ngữ cảnh rõ ràng: Khách hàng thấy giao diện chat thân thiện với sản phẩm và đơn hàng cá nhân; Nhân viên thấy thêm thông tin tồn kho chi nhánh; Manager thấy đầy đủ dữ liệu báo cáo và doanh thu.



Hình 3.35: Giao diện AI Chatbot Assistant (UI-30)

Chương 4

XÂY DỰNG ỨNG DỤNG VÀ KIỂM THỬ CHƯƠNG TRÌNH

4.1 Tổ chức thư mục của dự án

Dự án được tổ chức theo mô hình **Monorepo** với hai thành phần lớn: backend/ chứa 9 Microservices độc lập và frontend/ chứa ứng dụng React SPA. Cấu trúc thư mục cấp cao nhất như sau:

```
posmart/
|-- backend/
|   |-- auth-service/           # Port 3001 -- Xác thực & phân quyền
|   |-- catalog-service/       # Port 3002 -- Danh mục sản phẩm
|   |-- order-service/         # Port 3003 -- Quản lý đơn hàng
|   |-- settings-service/      # Port 3004 -- Cấu hình hệ thống
|   |-- supplier-service/      # Port 3005 -- Nhà cung cấp
|   |-- inventory-service/     # Port 3006 -- Tồn kho
|   |-- payment-service/       # Port 3007 -- Thanh toán VNPAY
|   |-- chatbot-service/       # Port 3008 -- AI Chatbot & Gợi ý
|   |-- statistics-service/    # Port 3009 -- Thống kê & Báo cáo (Redis Cache)
|   \-- nginx/
|       \-- nginx.conf         # API Gateway -- Port 8080
|-- frontend/
|   |-- src/
|   \-- ...
\-- docker-compose.yml         # Điều phối toàn bộ hệ thống
```

Mỗi microservice đều tuân theo cấu trúc **4 tầng** thống nhất (Route → Controller/Service → Repository → Database), được minh họa qua chatbot-service – service phức tạp nhất:

```
chatbot-service/
|-- src/
|   |-- routes/
|   |   \-- chat.routes.js     # Định tuyến HTTP & WebSocket
|   |-- services/
|   |   |-- chat.service.js    # Logic hội thoại chính
|   |   |-- rag.service.js     # RAG Pipeline (Embedding + Search)
```

```

|   |   |-- recommendation/
|   |   |   |-- apriori.service.js    # Thuật toán Apriori
|   |   |   |-- cf.service.js         # Collaborative Filtering
|   |   |   |-- ensemble.service.js   # Hybrid Ensemble Scoring
|   |   |   |-- session.service.js    # Session Context Boost
|   |   |   \-- weight.service.js     # Adaptive Weight Learning
|   |   \-- batch/
|   |       \-- nightly.batch.js      # Cron Job 2:00 AM
|   |-- repositories/
|   |   |-- chat.repository.js
|   |   |-- knowledge.repository.js
|   |   \-- feedback.repository.js
|   |-- config/
|   |   \-- db.js                    # Connection Pool PostgreSQL
|   \-- index.js                    # Entry point (Express + Socket.IO)
|-- tests/
|   |-- unit/
|   |   |-- rag.test.js
|   |   |-- apriori.test.js
|   |   |-- cf.test.js
|   |   |-- ensemble.test.js
|   |   |-- session.test.js
|   |   \-- weight-learner.test.js
|   \-- integration/
|       \-- chat.integration.test.js
|-- Dockerfile
\-- package.json

```

Cấu trúc tương tự được áp dụng cho các service còn lại. Các service xử lý giao dịch tài chính (Order, Inventory, Payment) có thêm tầng transactions/ chứa logic State Machine và SAGA Pattern, đảm bảo tính nhất quán dữ liệu.

4.2 Kiểm thử chương trình

Hệ thống áp dụng chiến lược kiểm thử **hai tầng**: Unit Test kiểm tra từng thuật toán độc lập ở tầng logic nghiệp vụ, và Integration Test kiểm tra luồng API đầu cuối. Công cụ sử dụng là **Jest** (test runner) kết hợp **Supertest** (HTTP assertion), với Database Test Container để cô lập môi trường kiểm thử.

4.2.1 Kiểm thử các chức năng đã thực hiện

Kiểm thử Thuật toán RAG (Content-Based Filtering)

Mục tiêu: Xác nhận luồng tìm kiếm ngữ nghĩa + từ khóa và hàm RRF Fusion hoạt động đúng với dữ liệu mẫu minh họa (mục 2.7 của tài liệu kỹ thuật).

Testcase TC-RAG-01: RRF Score chính xác với hai luồng kết quả

Testcase TC-RAG-02: Xử lý Cold-start – sản phẩm chỉ xuất hiện ở một luồng

Testcase TC-RAG-03: Top 5 giới hạn chính xác

Trường	Nội dung
Tên testcase	TC-RAG-01
Mô tả	RRF hợp nhất đúng hai danh sách Semantic + Keyword
Đầu vào	Semantic: [Ba chỉ bò (rank=0), Thịt bò Úc (rank=1), Nấm kim châm (rank=2)] Keyword: [Thịt bò Úc (rank=0), Ba chỉ bò (rank=1), Bún bò (rank=2)]
Kết quả mong đợi	Ba chỉ bò: $RRF = 1/61 + 1/62 \approx 0.0327$ Thịt bò Úc: $RRF = 1/62 + 1/61 \approx 0.0327$ Nấm kim châm: $RRF = 1/63 \approx 0.0159$
Kết quả thực tế	Đạt – sai số $< 10^{-6}$
Trạng thái	PASS

Bảng 4.1: Testcase TC-RAG-01: RRF Fusion

Trường	Nội dung
Tên testcase	TC-RAG-02
Mô tả	Sản phẩm mới chưa có embedding vẫn được trả về qua Keyword Search
Đầu vào	Sản phẩm “Nấm Đông Cô Nhật” chưa có vector embedding; câu hỏi: “nấm”
Kết quả mong đợi	Sản phẩm xuất hiện trong kết quả Keyword, $RRF\ Score = 1/(60 + rank)$
Trạng thái	PASS

Bảng 4.2: Testcase TC-RAG-02: Cold-start sản phẩm mới

Trường	Nội dung
Tên testcase	TC-RAG-03
Mô tả	Hệ thống trả về tối đa 5 kết quả sau RRF, dù hai luồng có nhiều hơn
Đầu vào	Semantic: 8 sản phẩm, Keyword: 7 sản phẩm
Kết quả mong đợi	Output: đúng 5 sản phẩm, sắp xếp giảm dần theo RRF Score
Trạng thái	PASS

Bảng 4.3: Testcase TC-RAG-03: Giới hạn Top 5

Kiểm thử Thuật toán Apriori

Mục tiêu: Xác nhận việc tính Support, Confidence, Lift đúng với bộ dữ liệu mẫu 100 đơn hàng.

Testcase TC-APR-01: Tính toán Support, Confidence, Lift

Trường	Nội dung
Tên testcase	TC-APR-01
Mô tả	Kiểm tra tính đúng đắn của ba chỉ số Apriori
Đầu vào	$ A \cap B = 15$ (Ba chỉ bò + Năm kim châm), $ A = 25$, $ B = 20$, $ T = 100$
Kết quả mong đợi	Support = 0.15, Confidence = 0.60, Lift = 3.00
Kết quả thực tế	Support = 0.15, Confidence = 0.60, Lift = 3.00
Trạng thái	PASS

Bảng 4.4: Testcase TC-APR-01: Tính toán Apriori

Testcase TC-APR-02: Edge Case – Division by Zero

Trường	Nội dung
Tên testcase	TC-APR-02
Mô tả	Hệ thống xử lý an toàn khi sản phẩm chưa có đơn hàng
Đầu vào	$ A = 0$ hoặc $ B = 0$
Kết quả mong đợi	confidence = 0, lift = 0 (safe fallback, không throw exception)
Trạng thái	PASS

Bảng 4.5: Testcase TC-APR-02: Edge Case Division by Zero

Testcase TC-APR-03: Phân biệt tương quan thật và ngẫu nhiên

Kiểm thử Collaborative Filtering

Mục tiêu: Xác nhận Cosine Similarity tính đúng và Prediction Score phản ánh đúng hành vi người dùng.

Testcase TC-CF-01: Cosine Similarity giữa hai sản phẩm liên quan

Testcase TC-CF-02: Pruning loại bỏ cặp thiếu dữ liệu

Testcase TC-CF-03: Cold-start User – tái phân bổ trọng số

Kiểm thử Hybrid Ensemble Scoring

Mục tiêu: Xác nhận công thức tổng hợp $\text{Score}(p) = \alpha S_C + \beta S_{CF} + \gamma S_A + \delta S_P$ cho kết quả đúng.

Testcase TC-ENS-01: Tính điểm tổng hợp cho User VIP

Testcase TC-ENS-02: Apriori boost đẩy Năm lên trên Bún bò

Trường	Nội dung
Tên testcase	TC-APR-03
Mô tả	Nước suối có support cao với mọi sản phẩm nhưng lift $\approx 1 \rightarrow$ không gợi ý
Đầu vào	Nước suối: $ A = 90$, $ B = 85$, $ A \cap B = 78$, $ T = 100$
Kết quả mong đợi	Lift = $78 \times 100 / (90 \times 85) \approx 1.02 \rightarrow$ bị lọc ra, không đưa vào kết quả
Trạng thái	PASS

Bảng 4.6: Testcase TC-APR-03: Lọc tương quan ngẫu nhiên

Trường	Nội dung
Tên testcase	TC-CF-01
Mô tả	Ba chỉ bò và Năm kim châm được mua bởi cùng nhóm user $\rightarrow$ similarity cao
Đầu vào	$R_{\text{bò}} = [2, 0, 1, 3]$, $R_{\text{năm}} = [2, 0, 1, 3]$ (4 users)
Kết quả mong đợi	sim(bò, năm) = 1.0 (vector đồng hướng hoàn toàn)
Trạng thái	PASS

Bảng 4.7: Testcase TC-CF-01: Cosine Similarity cao

Trường	Nội dung
Tên testcase	TC-CF-02
Mô tả	Cặp sản phẩm chỉ có 1 người dùng chung bị loại khỏi ma trận
Đầu vào	Cặp (Sản phẩm X, Sản phẩm Y): common_users = 1
Kết quả mong đợi	Cặp bị loại (Pruning), không lưu vào item_similarity
Trạng thái	PASS

Bảng 4.8: Testcase TC-CF-02: Pruning cặp ít dữ liệu

Trường	Nội dung
Tên testcase	TC-CF-03
Mô tả	User mới chưa có lịch sử mua, CF không có dữ liệu → trọng số tái phân bổ
Đầu vào	userId = 999 (không có record trong user_product_interaction)
Kết quả mong đợi	$\alpha' = \alpha + \beta = 0.65$, $\beta' = 0.00$; hệ thống vẫn trả về gợi ý
Trạng thái	PASS

Bảng 4.9: Testcase TC-CF-03: Cold-start User Redistribution

Trường	Nội dung
Tên testcase	TC-ENS-01
Mô tả	Tính điểm Ensemble khi user hỏi về thịt bò, weights mặc định
Đầu vào	Ba chỉ bò: $S_C = 1.00$, $S_{CF} = 0.80$, $S_A = 0.00$, $S_P = 1.0$ (VIP); $\alpha = 0.40$, $\beta = 0.25$, $\gamma = 0.25$, $\delta = 0.10$
Kết quả mong đợi	Score = $0.40 + 0.20 + 0.00 + 0.10 = 0.70$
Kết quả thực tế	0.70
Trạng thái	PASS

Bảng 4.10: Testcase TC-ENS-01: Ensemble Score User VIP

Trường	Nội dung
Tên testcase	TC-ENS-02
Mô tả	Apriori confidence cao khiến Nấm kim châm xếp trên Bún bò dù Content score thấp hơn
Đầu vào	Nấm: $S_C = 0.49$, $S_{CF} = 0.60$, $S_A = 0.60$, $S_P = 1.0$; Bún: $S_C = 0.48$, $S_{CF} = 0.00$, $S_A = 0.40$, $S_P = 1.0$
Kết quả mong đợi	Nấm: $0.596 > \text{Bún: } 0.392 \rightarrow \text{Nấm xếp trên}$
Trạng thái	PASS

Bảng 4.11: Testcase TC-ENS-02: Apriori quyết định thứ hạng

Kiểm thử Session Context Boost

Mục tiêu: Xác nhận hệ thống nhận diện đúng cluster và chỉ kích hoạt boost khi confidence ≥ 0.4 .

Testcase TC-SES-01: Nhận diện cluster lau_bo

Trường	Nội dung
Tên testcase	TC-SES-01
Mô tả	Phiên chứa bò và nấm → nhận diện đúng cluster lẩu bò và boost +0.15
Đầu vào	Sản phẩm trong session: [Ba chỉ bò, Nấm kim châm, Rau cải]
Kết quả mong đợi	cluster = lau_bo, boost = +0.15, áp dụng cho sản phẩm phù hợp
Trạng thái	PASS

Bảng 4.12: Testcase TC-SES-01: Nhận diện cluster lẩu bò

Testcase TC-SES-02: Không boost khi confidence thấp

Trường	Nội dung
Tên testcase	TC-SES-02
Mô tả	User duyệt nhiều danh mục khác nhau → confidence thấp → trả về exploring
Đầu vào	Session: [Nấm, Bánh mì, Bia, Nước mắm] – trải rộng nhiều cluster
Kết quả mong đợi	confidence < 0.4 → cluster = exploring, không áp dụng boost
Trạng thái	PASS

Bảng 4.13: Testcase TC-SES-02: Không boost khi duyệt tổng quát

Kiểm thử Adaptive Weight Learning

Mục tiêu: Xác nhận Exponential Smoothing cập nhật trọng số đúng và tuân thủ Guard Rails.

Testcase TC-WL-01: Cập nhật trọng số khi CF có conversion rate cao nhất

Testcase TC-WL-02: Guard Rail – bỏ qua khi chưa đủ feedback

Testcase TC-WL-03: Guard Rail – Clamping phòng tránh độc quyền

Kiểm thử End-to-End – Kịch bản tổng hợp

Mục tiêu: Xác nhận toàn bộ pipeline từ câu hỏi đến phản hồi chatbot hoạt động đúng theo kịch bản thực tế.

Testcase TC-E2E-01: User VIP hỏi về nguyên liệu nấu lẩu bò

Testcase TC-E2E-02: Feedback Loop sau giao dịch hoàn tất

Trường	Nội dung
Tên testcase	TC-WL-01
Mô tả	Kiểm chứng kết quả thực nghiệm – CF tăng trọng số sau một chu kỳ học
Đầu vào	Rate(Content) = 0.0838, Rate(CF) = 0.1140, Rate(Apriori) = 0.0900; weights cũ: $\alpha = 0.40$, $\beta = 0.25$, $\gamma = 0.25$; $\lambda = 0.8$
Kết quả mong đợi	$\alpha_{\text{new}} = 0.3724$, $\beta_{\text{new}} = 0.2713$, $\gamma_{\text{new}} = 0.2563$
Kết quả thực tế	$\alpha = 0.3724$, $\beta = 0.2713$, $\gamma = 0.2563$
Trạng thái	PASS

Bảng 4.14: Testcase TC-WL-01: Cập nhật trọng số qua Exponential Smoothing

Trường	Nội dung
Tên testcase	TC-WL-02
Mô tả	Không cập nhật trọng số khi số lượng feedback < 20 (tránh học từ noise)
Đầu vào	Tổng feedback trong ngày: 15 records
Kết quả mong đợi	Trọng số không thay đổi; hàm learn() trả về skipped: insufficient data
Trạng thái	PASS

Bảng 4.15: Testcase TC-WL-02: Guard Rail MIN_FEEDBACK_COUNT

Trường	Nội dung
Tên testcase	TC-WL-03
Mô tả	Trọng số tính toán vượt 0.60 hoặc xuống dưới 0.05 bị cắt về giới hạn
Đầu vào	Raw weight sau smoothing: $\beta_{\text{raw}} = 0.72$ (vượt ngưỡng tối đa)
Kết quả mong đợi	$\beta_{\text{final}} = 0.60$ (bị clamped); tổng $\alpha + \beta + \gamma$ vẫn = 0.90 sau tái chuẩn hóa
Trạng thái	PASS

Bảng 4.16: Testcase TC-WL-03: Guard Rail Clamping

Trường	Nội dung
Tên testcase	TC-E2E-01
Mô tả	Xác nhận kết quả cuối cùng của pipeline 7 bước với weights đã học
Đầu vào	User VIP (customerId=5), câu hỏi: “Tôi muốn nấu lẩu bò, cần mua gì?”
Bước 1-4	Query Reformulation → Embedding → Hybrid Search → RRF: Top 5 = [Ba chỉ bò, Nấm kim châm, Rau cải, Thịt bò Úc, Bún tươi]
Bước 5 (Ensemble)	$\alpha = 0.37, \beta = 0.27, \gamma = 0.26, \delta = 0.10$: Nấm = 0.716, Bò = 0.700
Bước 6 (Session Boost)	Cluster 1lau_bo (+0.15): Nấm = 0.866, Bò = 0.850
Kết quả mong đợi	Top 3: [Nấm kim châm, Ba chỉ bò, Rau cải ngọt]; chatbot đề cập “60% khách mua bò cũng mua nấm”
Trạng thái	PASS

Bảng 4.17: Testcase TC-E2E-01: Kịch bản tổng hợp lẩu bò

Trường	Nội dung
Tên testcase	TC-E2E-02
Mô tả	Xác nhận 5 bước phễu conversion được ghi nhận đầy đủ
Luồng kiểm thử	(1) Chatbot gợi ý → INSERT recommended (2) User hover 2.3s → POST hovered (3) User click → POST clicked (4) Thêm giỏ hàng → POST added_to_cart (5) Thanh toán → ORDER_CONFIRMED → INSERT purchased (24h attribution)
Kết quả mong đợi	5 records trong recommendation_feedback, source đúng với từng sản phẩm
Trạng thái	PASS

Bảng 4.18: Testcase TC-E2E-02: Phễu Feedback Loop 5 bước

4.2.2 Kiểm thử các yêu cầu phi chức năng

Kiểm thử Hiệu suất (Performance)

Mục tiêu: Xác nhận hệ thống đáp ứng ngưỡng độ trễ < 500ms P95 cho toàn bộ pipeline gợi ý trong điều kiện tải đồng thời.

Thành phần	Ngưỡng đặt ra	Kết quả đo được	Trạng thái
HNSW Semantic Search	< 50ms	$\approx 12\text{ms}$	PASS
GIN Full-Text Search	< 30ms	$\approx 8\text{ms}$	PASS
RRF Fusion	< 5ms	< 1ms	PASS
Apriori Lookup	< 5ms	$O(1)$, < 1ms	PASS
CF Prediction	< 30ms	$\approx 15\text{ms}$	PASS
Ensemble Scoring	< 5ms	< 1ms	PASS
LLM Generation	< 400ms	$\approx 300\text{ms}$	PASS
Tổng Pipeline P95	< 500ms	$\approx 380\text{ms}$	PASS

Bảng 4.19: Kết quả kiểm thử hiệu suất từng thành phần

Nightly Batch Pipeline: Toàn bộ chu kỳ tính toán gồm 4 bước (Apriori, CF Similarity, Weight Learning, Cache Warmup) hoàn thành trong dưới 2 phút với bộ dữ liệu 200 sản phẩm và 1.000 đơn hàng. Đây là mức hoàn toàn phù hợp cho khung thời gian cron 2AM hàng đêm.

Kiểm thử Bảo mật (Security)

Kiểm thử phân quyền RBAC trên Chatbot: Hệ thống thực thi đúng nguyên tắc cách ly dữ liệu theo vai trò – Khách hàng chỉ được truy vấn thông tin đơn hàng cá nhân và giá sản phẩm; yêu cầu xem báo cáo doanh thu hoặc tồn kho nội bộ (chỉ dành cho Manager và Employee) bị từ chối với mã lỗi 403 Forbidden phù hợp ràng buộc RBAC.

Kiểm thử Multi-tenancy: Dữ liệu của mỗi chi nhánh được cách ly tuyệt đối thông qua `store_id`. Nhân viên đã đăng nhập vào chi nhánh A không thể truy vấn tồn kho hoặc đơn hàng của chi nhánh B – kiểm thử thực hiện bằng cách gửi request với JWT hợp lệ nhưng sai `store_id`, xác nhận hệ thống trả về 403 trong mọi trường hợp.

Kiểm thử VNPAY Webhook: Hệ thống từ chối cập nhật trạng thái thanh toán khi chữ ký IPN không hợp lệ, phòng chống giả mạo giao dịch. Kiểm thử gửi Webhook với `vnp_SecureHash` bị thay đổi và xác nhận hệ thống trả về 400 Bad Request mà không thực hiện bất kỳ cập nhật nào vào cơ sở dữ liệu.

Kiểm thử Tính toàn vẹn giao dịch (Data Integrity)

Kiểm thử Transaction Zone: Hệ thống đảm bảo nguyên tắc “all-or-nothing” cho các luồng nghiệp vụ quan trọng. Trong testcase mô phỏng lỗi kết nối ở bước trừ tồn kho sau khi đơn hàng đã được tạo thành công, toàn bộ transaction bị rollback – không có trạng thái dở dang (đơn hàng ở trạng thái “completed” nhưng tồn kho chưa trừ). Kết quả xác nhận POSMART xử lý đúng mọi “Transaction Zone” theo đặc tả.

Kiểm thử Nightly Batch – Fault Isolation: Khi bước Apriori Batch gặp lỗi kết nối database, hai bước tiếp theo (CF Similarity và Weight Learner) vẫn tiếp tục chạy độc lập với dữ liệu cũ từ chu kỳ trước. Hành vi này được xác nhận qua log `step_status` trong bảng `batch_log`.

Kiểm thử Dual-Tracking – Cách ly dữ liệu

Mục tiêu: Xác nhận Weight Learner chỉ sử dụng dữ liệu từ luồng Chatbot, không bị Data Poisoning từ organic traffic.

Kịch bản	Weight Learner sử dụng?	CF Matrix sử dụng?
Feedback source = content	Có	Có
Feedback source = cf	Có	Có
Feedback source = apriori	Có	Có
Feedback source = organic	Không	Có

Bảng 4.20: Kết quả kiểm thử cách ly dữ liệu Dual-Tracking

Kết quả xác nhận: hệ thống phân tách đúng hai luồng dữ liệu, tránh được hiện tượng Data Poisoning do organic traffic không có bước recommended làm sai lệch Conversion Rate trong quá trình học trọng số α , β , γ .

4.3 Kết quả Thực nghiệm và Đo lường Mô hình Gợi ý Thông minh Lai (‘ai-service-v2’)

4.3.1 Thiết lập Môi trường Thực nghiệm và Dữ liệu Kiểm thử v5.1

Khác với phương pháp đánh giá thủ công của Đề án 2, trong khuôn khổ Khóa luận tốt nghiệp, toàn bộ quy trình thực nghiệm được thực thi tự động thông qua Recommender Research Runner độc lập ai-service-v2.

Dữ liệu kiểm thử sử dụng bộ ****Dataset Snapshot v5.1**** được đóng băng mã băm SHA-256 gồm 5.000 người dùng, 5.200 sản phẩm và 823.371 lượt tương tác giao dịch bán lẻ. Quy trình phân tách thời gian (Temporal Split) được cố định nghiêm ngặt:

- **Tập Huấn luyện (Train):** Từ 01/01/2026 đến 19/06/2026 UTC.
- **Tập Thăm định (Validation):** Từ 20/06/2026 đến 10/07/2026 UTC.
- **Tập Kiểm thử (Test):** Từ 11/07/2026 đến 01/08/2026 UTC.

Giao thức đánh giá tuân thủ nguyên tắc xếp hạng toàn danh mục (*Full-catalog ranking*): đối với mỗi người dùng hợp lệ có phát sinh giao dịch mua sắm tự nhiên trong tập kiểm thử, tập ứng viên $C_u = I \setminus H_u^{\text{seen}}$ bao gồm toàn bộ sản phẩm chưa từng mua trước đó. Bộ đánh giá độc lập (Independent Evaluator) tính toán các chỉ số chất lượng xếp hạng gồm: NDCG@10, Hit Ratio (HR@10), Recall@10 và Macro GAUC.

4.3.2 Bảng Kết quả Đánh giá So sánh trên Toàn bộ Danh mục

Bảng 4.21 trình bày kết quả đánh giá thực nghiệm so sánh giữa mô hình đề xuất và các đường cơ sở (Baselines):

Phân tích kết quả thực nghiệm:

- Mô hình phổ biến (MostPop) đạt kết quả thấp nhất do xu hướng gợi ý rập khuôn các mặt hàng đại trà, không thể cá nhân hóa theo từng sở thích riêng biệt của khách hàng.

Mô hình thử nghiệm	NDCG@10	HR@10	Recall@10	Macro GAUC
Popularity Baseline (MostPop)	0.0421	0.0812	0.0385	0.5412
Rule-based (Apriori Alone)	0.0784	0.1245	0.0692	0.6120
Deep Two-Tower (BPR Alone)	0.1142	0.1870	0.1034	0.7350
Proposed Hybrid (Wide + Deep Two-Tower)	0.1385	0.2190	0.1256	0.7812

Bảng 4.21: Kết quả đánh giá so sánh các mô hình trên tập kiểm thử Dataset Snapshot v5.1

- Nhánh luật kết hợp Apriori đơn lẻ đạt hiệu quả tốt trong việc bắt trúng các sản phẩm thường mua kèm (như bánh mì và sữa, bia và đồ nhắm), song gặp hạn chế nghiêm trọng ở các sản phẩm ít giao dịch (Cold-item / Long-tail).
- Mô hình mạng nơ-ron Deep Two-Tower huấn luyện bằng BPR Loss thể hiện năng lực vượt trội nhờ khả năng khái quát hóa và chiều đặc trưng ngữ nghĩa từ tầng Content Projection.
- **Mô hình đề xuất Hybrid (Wide + Deep Two-Tower)** đạt hiệu quả cao nhất trên toàn bộ các thang đo: NDCG@10 đạt 0.1385 (tăng +21.3% so với Deep Two-Tower đơn lẻ) và HR@10 đạt 0.2190. Điều này chứng minh tính đúng đắn của giả thuyết bổ trợ: nhánh Wide giữ lại các mẫu quy luật mua hàng tự nhiên trong khi nhánh Deep mở rộng không gian khám phá sản phẩm mới.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Qua quá trình nghiên cứu lý thuyết, phân tích hệ thống và tiến hành cài đặt thực nghiệm, đồ án đã hoàn thành toàn bộ các mục tiêu đặt ra ban đầu cho hệ thống quản lý chuỗi bán lẻ bán lẻ và siêu thị mini **POSMART**. Các kết quả cụ thể đạt được bao gồm:

- **Về Kiến trúc hệ thống:** Xây dựng và phân rã thành công hệ thống thành 9 Microservices chạy độc lập biệt lập (Auth, Catalog, Order, Settings, Supplier, Inventory, Payment, AI Chatbot). Hệ thống vận hành ổn định trên nền tảng luồng dữ liệu hướng sự kiện (Event-Driven Architecture) thông qua Message Broker RabbitMQ, đảm bảo tính toàn vẹn dữ liệu phân tán thông qua việc thiết kế nghiêm ngặt các Transaction Zones và SAGA Pattern.
- **Về Mô hình Multi-tenancy:** Triển khai thành công chiến lược Đa người thuê ở mức cơ sở dữ liệu (Shared Database, Shared Schema) cô lập bằng chính sách `store_id`. Thiết kế này tối ưu hóa tối đa chi phí tài nguyên phần cứng cho quy mô chuỗi dưới 100 cửa hàng nhưng vẫn đảm bảo tính an toàn dữ liệu giữa các chi nhánh chi tiết đến từng dòng dữ liệu (Row-Level Security).
- **Về Trợ lý thao tác AI Chatbot (Action Assistant v2.0):** Nâng cấp thành công Chatbot từ mô hình chỉ đọc (Read-only RAG Pipeline) lên mô hình trợ lý thực thi hành động hai chiều. Nhờ thiết kế *Action Response Protocol*, hệ thống cho phép Chatbot điều khiển trạng thái Frontend (giỏ hàng, đơn hàng) một cách an toàn mà vẫn giữ Backend ở trạng thái Stateless. Hệ thống đảm bảo tính an toàn tuyệt đối qua mô hình bảo mật 7 lớp với chốt chặn quyết định từ *Confirmation Gate*.
- **Về Hệ thống gợi ý AI (Hybrid Ensemble):** Triển khai hệ thống gợi ý lai kết hợp 4 trụ cột thuật toán tối ưu hóa qua điểm số xếp hạng tổng hợp RRF. Giải quyết triệt để bài toán Cold-start nhờ cơ chế Content-Based kết hợp Semantic Search (chỉ mục HNSW) và Full-Text Search (GIN index), đồng thời tối ưu hóa runtime lookup bán chéo về mức độ phức tạp $O(1)$ thông qua Partial Index của thuật toán Apriori.
- **Về Tích hợp thanh toán:** Tích hợp thành công cổng thanh toán điện tử VNPay, xử lý đồng bộ và kiểm soát gian lận thông qua cơ chế Secure Hash kết hợp Webhook IPN an toàn.

2. Hướng phát triển

Mặc dù hệ thống đã đáp ứng đầy đủ các yêu cầu nghiệp vụ thực tiễn, nhóm thực hiện đề tài định hướng một số cải tiến và mở rộng trong tương lai như sau:

- **Nâng cấp cơ sở dữ liệu:** Khi chuỗi cửa hàng tăng trưởng quy mô lớn vượt ngưỡng, kiến trúc Multi-tenancy sẽ được nâng cấp từ cấu hình Shared Schema sang cấu hình *Isolated Database* (mỗi chi nhánh sở hữu một database vật lý riêng biệt) để tăng cường tối đa tính độc lập và khả năng bảo mật.
- **Cục bộ hóa mô hình ngôn ngữ lớn (LLM):** Tiến hành Fine-tuning các mô hình ngôn ngữ lớn có kích thước nhỏ (như Phi-3-mini hoặc Mistral-7B) và triển khai trực tiếp trên máy chủ nội bộ hoặc hạ tầng Edge Computing thay vì phụ thuộc hoàn toàn vào Cloud API (Hugging Face Inference API), nhằm giảm thiểu độ trễ phản hồi xuống dưới 1 giây và bảo mật dữ liệu hội thoại nội bộ.
- **Phát triển ứng dụng di động đa nền tảng:** Xây dựng phiên bản Mobile App (bằng React Native hoặc Flutter) dành riêng cho hai đối tượng khách hàng (hỗ trợ mua sắm nhanh, quản lý giỏ hàng bằng giọng nói qua AI) và nhà quản lý (theo dõi báo cáo doanh thu thời gian thực, phê duyệt đơn nhập kho tức thời).
- **Ứng dụng Học sâu vào hệ thống gợi ý:** Khi lượng dữ liệu tương tác người dùng tích lũy đủ lớn, bộ trọng số tuyến tính thích ứng chạy ngầm (*WeightLearner*) sẽ được thay thế bằng các kiến trúc mạng nơ-ron chuyên sâu (Graph Neural Networks hoặc DeepFM) để tăng cường độ chính xác dự báo hành vi mua sắm dài hạn.

Tài liệu tham khảo

- [1] Covington, P., Adams, J., & Sargin, E. (2016). Deep neural networks for YouTube recommendations. *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, 191-198.
- [2] Yi, X., Yang, J., Hong, L., et al. (2019). Sampling-bias-corrected neural modeling for large corpus string recommendations. *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys)*, 269-277.
- [3] Cheng, H. T., Koc, L., Harmsen, J., et al. (2016). Wide & deep learning for recommender systems. *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS)*, 7-10.
- [4] Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, 452-461.
- [5] Gusak, J., Kaloev, M., & Babenko, A. (2025). Temporal evaluation protocols for sequential recommendation: Pitfalls and best practices. *ACM Transactions on Recommender Systems*.
- [6] Petrov, A., & Macdonald, C. (2022). A systematic review and replicability study of BERT4Rec for sequential recommendation. *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys)*, 436-447.
- [7] Kleppmann, M. (2022). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [8] Nguyễn, V. H. (2020). *Giáo trình Phân tích và Thiết kế Hệ thống thông tin*. NXB Giáo dục Việt Nam.
- [9] Trần, M. T. (2019). *Lập trình ứng dụng Web với Node.js và Express*. NXB Khoa học và Kỹ thuật.
- [10] Bộ Công Thương (2021). *Sách trắng Thương mại điện tử Việt Nam*. Cục Thương mại điện tử và Kinh tế số.
- [11] PostgreSQL Global Development Group. (2026). *PostgreSQL 18 Core Documentation - Generalized Inverted Index (GIN) and Row-Level Security Policies*. <https://www.postgresql.org/docs/>
- [12] RabbitMQ Team. (2025). *RabbitMQ Documentation: Highly Available Queues and Event-Driven Architecture Patterns*. Broadcom. <https://www.rabbitmq.com/documentation.html>

- [13] VNPay Corporation. (2025). *Tài liệu tích hợp API thanh toán và đặc tả kỹ thuật Webhook IPN dành cho nhà phát triển (v2.0)*. <https://sandbox.vnpay.vn/apis/>
- [14] Hugging Face Inc. (2025). *Hugging Face Inference API and Function Calling Protocol Specification*. <https://huggingface.co/docs/api-inference/>
- [15] Kleppmann, M. (2022). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.
- [16] Cormack, G. V., Clarke, C. L., & Buccio, S. (2008). Reciprocal Rank Fusion out-performs Evaluation Measures. *Proceedings of the 31st annual international ACM SIGIR conference on Research and development in information retrieval*, 65-72.
- [17] Agrawal, R., & Srikant, R. (1994). Fast algorithms for mining association rules. *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, 1215, 487-499.
- [18] Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. *Proceedings of the 10th international conference on World Wide Web*, 285-295.
- [19] Internet Engineering Task Force (IETF). (2015). *RFC 7519: JSON Web Token (JWT) Architecture for Stateless Authentication*. <https://datatracker.ietf.org/doc/html/rfc7519>